COVER LETTER OF PAPER 804

Dear Meta Reviewer and Referees:

We have substantially revised the paper following the Referees' suggestions. For the convenience of Referees, changes for addressing the concerns of R1, R3 and R4 are color-coded in purple, blue and orange, respectively. We would like to thank the Referees for insightful comments and for supporting this work!

Below please find our responses to the reviews.

Response to the comments of the Meta Reviewer.

[Meta summary & recommendation]

- The reviewers discussed the authors' response and revision plan and agreed that the paper should undergo a revision. All three reviewers were satisfied with the revision plan and did not have additional requests.

- The reviewers did not request any changes to the revision plan.

[A] Many thanks! We have fully implemented the revision plan to address all reviewers' comments throughout the revised paper.

Response to the comments of Referee #1.

[R1O1 & D7 & D11] *- The introduction motivates DocChecker as addressing the stated challenges and then describes DocChecker's features, but the new contribution of this paper was not clear to me. There is a paragraph explaining how the algorithms improve upon prior work, implying that the contribution is a new algorithm for mining graph contradiction rules. If so, it would be good to state this explicitly, explain the shortcomings of existing rules, using specific examples, and motivate why new rules are required to capture real-world contradictions.*

- Page 3, to make this paper self-contained, it would help to explain what GCRs are and show examples of how the new proposed rules are different and more suitable for real-world contradictions. Also clarify what 'sparse' means in this problem.

[A] Thanks! We have summarized the novelty (Sections 1 & 2, pp. 2–3). This work goes beyond a system integration of graph analytics and NLP models. We formulate textual contradiction detection as a *rule-based reasoning problem over text-derived graphs*, where rules unify logical predicates, ML predictions and NLP classifiers, to uniformly catch intra- and inter-document contradictions with both interpretability and scalability beyond prior NLI-based or graph rule-based approaches. Based on the rules, we develop novel algorithms for learning-based rule discovery and contradiction detection.

We have also clarified that "sparse" contradictions (a tiny fraction of entity-attribute pairs) induce a large search space, making traditional support-based methods ineffective (p. 2 & p. 8).

We have expanded the comparison between GCRs and RICOs (Section 3.2, p. 4 and Appendix A): RICOs extend GCRs with (a) NLP predicates on text spans, (b) edge-level attributes and reasoning, and (c) contextual and temporal constraints, enabling structural reasoning and richer comparisons for real-world contradictions. Despite incorporating ML/NLP predicates, evaluation remains well-defined: predicates are treated as Boolean oracles, and rule satisfaction follows the induced truth assignment, preserving logical semantics.

We illustrate (Example 2, pp. 4–5) why prior rules fail without text predicates, edge attributes, and temporal/contextual reasoning.

[R1O1 & D15] *The title of section 6 is "Discovering cleaning rules" which does not align with the rest of the paper where the focus is on contradictions. If the connection between contradictions and graph data cleaning is important then it should be stated clearly and the implications should be explained. For example, can* DocChecker *extend to knowledge graph cleaning?*

[A] Thanks! We have revised the title of Section 6 to "Fast Discovery of RICOs" (p. 8). A remark in Section 6.3 (p. 9) further clarifies that DocChecker focuses on contradiction detection (*i.e.,* conflicting attributes or relations for the same entity). Since existing methods (*e.g.,* RICO-based tracing, knowledge-based, or LLM-based approaches) handle the subsequent repair phase, DocChecker complements prior graph cleaning techniques and can extend to knowledge graph cleaning when it is coupled with a repair module.

[R1D1] *On page 1, the paper gives some stats about contradictions in national newspapers in 2017-2019. Are there any more recent stats? For instance, I am wondering if this problem is getting worse given the increasing volume of auto-generated content.*

[A] As suggested, we have updated Section 1 (p. 1) with recent statistics to emphasize the prevalence of document contradictions. Specifically, 21% of news stories [20] and at least 3.3% of facts in the 2025 Wikipedia snapshot are contradictory. Furthermore, AI hallucinations make the problem worse, where 48% to 82% of AI-generated texts contain hallucinations, and factual consistency drops to 21% in certain AI-generated datasets [104].

[R1D2] *Page 1, clarify what it means for early work to frame contradiction detection as text classification or textual entailment. Also clarify what "rather shallow" means and why methods struggle with cross-document conflicts? Is it because cross-document entity resolution is harder? Examples might help.*

[A] We have clarified these concepts by example in Section 1 (p. 1). Prior approaches treat contradiction detection as text classification (contradiction or not) relying on shallow signals (*e.g.,* lexical overlap, negation, antonymy) rather than semantic reasoning, and thus miss implicit cross-document conflicts beyond entity resolution (ER). For instance, detecting the contradiction between "Company X was founded in 1998" and "Company X held a board meeting in 1995" requires both ER and temporal reasoning. Moreover, NLI-style models lack a shared structure (*e.g.,* a unified entity–relation graph) for multi-hop reasoning across documents.

[R1D3] *Page 1, I was confused about the sentence "When contradictions arise, LLMs tend to default to parametric knowledge [15] or favor majority entities backed by larger volumes [125], rather than reasoning over conflicting evidence." I thought this paragraph was talking about contradiction detection, but this sentence sounds like it's talking about contradiction correction using the LLM's parametric knowledge. It would be good to clearly distinguish between what seems like different problems.*

[A] Thanks! We have clarified (Section 1, p. 1) the limitations of *LLM-based contradiction detection*, where the accuracy is only 51.6% [84]. Given conflicting statements, LLMs often fail to identify contradictions objectively and tend to favor parametric knowledge [16] over conflicting evidence [72]. This motivates our design:

LLMs are used only for candidate rule generation, while detection relies on explicit rule-based reasoning over structured graphs.

[R1D4] *Challenge 1: I assume "contextual inference" refers to entity resolution, so inferring that "the vaccine" in both sentences is referring to the same vaccine? If so, state this explicitly.*

[A] Thanks! We have clarified in Section 1 (Challenge 1, p. 1) that contextual inference requires more than Entity Resolution (ER), where logical deduction from context is also necessary. For example, the statements "The vaccine is safe for children" and "The vaccine has not been tested on minors" may seem compatible, but they become contradictory under the contextual assumption that any claim of safety must be supported by clinical testing.

[R1D5 & D13 & D17] *- Page 1, The "A graph-based framework" paragraph states "learn RICOs over samples of G", which lacks context and motivation. For example, why learn over samples and not all of G, or how are these samples chosen?*

- Page 5, "Rule discovery" and "Contradiction detection" paragraphs, it's still unclear to me how the graph is sampled to ensure correct rules. The "Ground truth" sentence says that facts are validated by users or experts, but user involvement is not clearly stated or illustrated in the paper.

- Page 9, clarify the source of ground truth. Is it included in every dataset used in the experiments? Also, how big are these datasets?

[A] Thanks! We have adopted a sampling strategy [31] to rule mining, which extracts subgraphs related to target entities (Section 5.1, p. 6). With a controllable parameter σ, this approach ensures that RICOs discovered from the samples retain support and confidence within bounded loss in terms of σ. In other words, it reduces the search space and cost while retaining valuable RICOs. This is to accommodate the prohibitive cost of rule discovery and contradiction analysis in large-scale graphs (*i.e.,* large collection of documents).

We have clarified (Section 4, p. 6) that ground truth is obtained from expert annotations and trusted sources. All our evaluation datasets provide annotated contradictions; dataset sizes are reported in Section 7 (Table 2, p. 10). While the experiments use labeled data, DocChecker does not rely on full supervision: RICOs can be initialized from small labeled subsets or trusted sources and refined via weak signals (*e.g.,* consistency constraints or high-confidence predictions), generalizing beyond fully supervised settings.

[R1D6] *Page 2, I couldn't understand the paragraph "The algorithms improve upon prior work" due to the use of undefined terms like "learning-based rule generator", "levelwise traversal", "contradiction-relevant", "downstream performance metrics", "sparse contradictions".*

[A] In Section 1 (p. 2), we have clarified these concepts by comparison with classical methods: (a) *Learning-based discovery vs. levelwise traversal*: instead of Apriori-style enumeration with exponential cost, we use an LLM guided by domain signals to generate high-quality rule candidates; (b) *Contradiction-relevant subset (D')*: we restrict discovery to conflict-prone regions, avoiding full-graph scans; (c) *Metrics vs. support*: since support treats sparse contradictions (see **[R1O1]**) as noise, we assess RICOs using contradiction detection accuracy. Together, these replace exhaustive enumeration with guided generation and tailor evaluation to sparse contradiction signals.

[R1D8] *Page 2, it wasn't clear to me what "*DocChecker *provides verifiable evidence" means. An example would help to understand what constitutes evidence and what it means to verify it.*

[A] Thanks! We have revised Case 1 in Example 2 (p. 4) to clarify that "verifiable evidence" consists of supporting facts and a rule-based reasoning trace. For rule φ_1, the evidence includes facts Γ (two award descriptions matched by $\mathcal{M}$) and the rule application trace: the rule reveals conflicting honorees (Nobel vs. Shannon), and the chase (p. 6, Appendix F) with external knowledge ($T(s, s', \text{factual})$) infers "Nobel" as the true honoree, and identifies "Shannon" as a contradiction. Thus, each detected contradiction is accompanied by both its supporting facts and reasoning steps.

[R1D9] *In section 2, clarify event-level and dialogue/QA scenarios. Examples would help. Also clarify "zero-shot inference, multi-hop reasoning and LLM-based inference". I interpreted zero-shot as LLM-based inference without any examples in the prompt.*

[A] Thanks! We have added examples (Section 2, p. 2) to illustrate event-level conflicts (*e.g.,* a summit reported in both Paris and Berlin) and dialogue/QA contradictions (*e.g.,* an agent claiming "I am a strict vegetarian" and "I love eating steak" in dialogues).

We have also clarified the terms in LLMs and NLP (p. 2), where zero-shot inference is reasoning without task-specific training examples, multi-hop reasoning is the process of combining multiple pieces of information to derive the conclusion, and LLM inference is using pre-trained LLMs to predict without extra training.

[R1D10] *Page 2, give examples of "NLP predicates" and "interpretable logical explanations", clarify "differentiable preprocessing".*

[A] Thanks! We have expanded Case 7 in Example 2 (p. 5) to illustrate how RICO φ_7 catches contradictions using both an NLP predicate ($\mathcal{M}(x_1.\text{time}, y_1.\text{time})$) and an interpretable logical explanation of inconsistent treatments ($x_2.\text{treatment} \neq y_2.\text{treatment}$).

We have also clarified that differentiable methods [63, 85] enable joint optimization of ML models and preprocessing by making preprocessing trainable via gradients, allowing end-to-end learning.

[R1D12] *Page 5, "Graph conversion" paragraph, there is not much detail on the graph conversion process, just the use of pertained ML models and LLM. I think this step is not trivial and requires steps like entity/pronoun resolution, so it would be good to describe it in more detail.*

[A] Thanks! We have detailed graph conversion (Section 5.1, p. 6), including coreference and entity resolution, and clarified how pre-trained models and LLMs extract entities, relations and attributes to construct G_D via node attributes and edge predicates. We have also included incremental graph conversion (Section 5.3, pp. 7–8). These steps ensure that G_D preserves the structural and semantic information required for contradiction detection and reasoning.

[R1D14] *Page 6, "user-specified entity set E", clearly state the inputs to the system, especially what users are supposed to provide.*

[A] Thanks! As suggested, we have clarified (Section 4, p, 5) that the input of DocChecker includes the document set D and an optional target entity set $\mathcal{E}$, where the set $\mathcal{E}$ filters out irrelevant documents to improve both efficiency and relevance.

[R1D16] *Page 8, clarify "binary classification accuracy". What is the classification problem here?*

[A] Thanks! In Section 6.1 (p. 8), we have formulated contradiction detection as a binary classification task (contradictory vs. non-contradictory) to evaluate the quality of learned RICOs. Each RICO encodes a specific contradiction type (see **[R3O1 c)]**), while the classifier (*e.g.,* RoBERTa-large) is used only to decide whether a contradiction exists. Thus, multi-class classification is unnecessary.

More specifically, we rank rule candidates by the accuracy gain measured by $f_a(\mathcal{D}_{\text{filter}}, \mathcal{D}) = \text{Acc}(\mathcal{D}_{\text{filter}}) - \text{Acc}(\mathcal{D})$, which measures how much a RICO improves the classifier's performance.

[R1D18] *In the experiments, it would be nice to see a qualitative study with examples of contradictions found by* DocChecker *but missed by baselines. This would help to show which types of contradictions are handled poorly by prior work and where* DocChecker *excels.*

[A] Thanks! As suggested, we have added a case study (Exp-4) in Section 7 (p. 13) to demonstrate complex contradictions (*e.g.,* temporal and factual inter-document contradictions) that DocChecker correctly detects but baselines miss. We have also explained why DocChecker works while the baselines fail. As these conflicts cannot be detected by sentence-level NLI models, the case study highlights the advantages of graph-based RICO reasoning approach.

As limitations, we note that contradictions requiring purely subjective or ambiguous interpretation remain challenging.

Many thanks for your support and insightful suggestions!

Response to the comments of Referee #3.

[R3O1 a)] *The paper doesn't talk in detail about the LLM cost for each LLM call within a rule - e.g., for X document pairs - an average of Y LLM calls were made within every rule. Cost per document would be a really useful number to know for deployment at scale.*

[A] Thanks! We have clarified (Section 6.3, p. 9) that LLMs are only utilized during the offline graph conversion and rule discovery phase. During this stage, the search space is confined to pruned patterns, which enables rapid convergence with limited number of LLM calls. For inference, RICO employs lightweight models (*e.g.,* GAT, SBERT, GCN; see p. 4, Section 3.2); it requires no LLM invocations.

We have also added LLM token usage for graph conversion and rule discovery (Table 2, p. 10), and reported both time and token consumption for incremental maintenance (Table 7, p. 13). Overall, LLM usage is confined to offline stages, while online inference is lightweight and efficient without making LLM calls.

[R3O1 b) & d)] *- b) There's an inherent cost involved in maintaining rules, knowledge graph/enrichment. Also, it's not clear whether small changes in the document require the graph join to be done again.*

- d) streaming updates consequences - knowledge graphs are rebuilt time to time, but mostly the knowledge is incremental or coming in a streaming fashion. The cost of rebuilding every time along with learning rules could become expensive with scale. It would be great to cover this in the paper to some extent.

[A] Thanks! We have clarified that DocChecker supports incremental maintenance instead of full recomputation (Section 4, p. 6; Section 5.3, pp. 7–8; and Section 6.2, p. 9; Appendices B.1–B.3, E). Specifically, (a) graph construction updates only affected subgraphs; (b) rule mining refines existing RICOs using new samples; and (c) mapping updates contradictions for new instances. Updates are localized to modified entities or relations, avoiding full recomputation while remaining equivalent to recomputation on affected subgraphs.

As shown by Proposition 3 (p.8, Appendix E), incremental updates preserve correctness since all updates are localized to affected subgraphs, and rule evaluation is equivalent to recomputation over the updated graph. This follows from the locality of pattern matching and the invariant of contradiction detection with RICOs.

We have also evaluated incremental maintenance (Exp-5, Table 7, Section 7, p. 13). When updating 100 out of 530-825 documents, DocChecker's localized updates significantly reduce cost, getting 77.6% lower latency and 78.3% fewer tokens in graph conversion. Rule generation time also drops from 1,135.3s to 50.1s, with meta-learning refinement completed in only 4.3% of the original time.

[R3O1 c)] *It's not clear what happens when multiple contradictory rules are fired. Do we do rule conflict resolution or produce a matrix/signals of contradictions per rule.*

[A] As suggested, we have clarified in Section 4 (p. 6) that while an entity or relation may participate in multiple rules, each attribute triggers at most one contradiction type, since the types defined in Section 2 are mutually exclusive. Thus, while multiple contradictions may arise for an entity, each corresponds to a distinct attribute, avoiding ambiguity. The reasoning follows a chase procedure with the Church-Rosser property (p. 6, Appendix F.2), ensuring that the final set of contradictions is independent of rule order and conflict-free. This unified graph-based reasoning also enables cross-document inference beyond pairwise NLI methods.

[R3O2 a)] *In practice, a lot of documents could have unstructured diversified text (discussions, blogs, opinions, etc.) - the evaluation is presented on relatively clean/structured benchmarks. To make a strong case for the paper, real-world, maybe topic-aware contradiction detection could help make the system more extensible and applicable for other scenarios.*

[A] Thanks! We have revised Section 7 (p. 9) to clarify that DocChecker was indeed evaluated on domain-specific datasets sourced from diverse real-world scenarios (see Appendix J), which exhibit heterogeneous and unstructured textual characteristics. More specifically, these datasets span legal contracts (ContractNLI, CaseHOLD), economic reports (ECON), news reports (ContraDoc, DOCNLI), and narrative stories (StorySumm). By incorporating a rich variety of genres and writing styles, these datasets reflect real-world documents along with versatile contradictions.

[R3O2 b)] *It's not clear if the authors have spent time trying different prompts when doing the comparison with GPT. A much more graph-focused (spo style formatting, linking, etc.) prompt could possibly show improvements.*

[A] Thanks! We have clarified in Section 7 (pp. 9–10) that the GPT baselines have already integrated comprehensive prompt designs, including SPO-style formatting and explicit entity linking (detailed in Appendix G). We have also added evaluations of various enhanced prompts in Appendices G.3–G.5. Our findings demonstrate

that the advanced prompting strategies, such as graph-augmented SPO, multi-stage CoVe, and counterfactual auditing, yield marginal improvements, and LLMs still fall much behind DocChecker.

More specifically, GPT-4o trails DocChecker by 44.67% in Type MR and 22.19% in inter-document detection, while the joint reasoning performance of Claude-3.7 is nearly half of DocChecker due to imprecise entity linking. Case studies (p. 13) also show that despite these prompt enhancements, LLMs often struggle with inter-sentence temporal linkage and lack the multi-entity alignment required to resolve complex conflicts. In contrast, DocChecker relies on graph-based entity binding for robust contradiction detection.

[R3O2 c)] *The paper claims to win over cross-document contradictions but they are not studied in detail, and no error analysis is provided. This should be expanded imo.*

[A] Thanks! We have clarified that cross-document contradiction detection is enabled by the graph join phase (Section 5.2), which fuses document-level graphs into a unified graph for entity alignment and global reasoning. In contrast, NLI operates on isolated sentence pairs and lack the structure for inter-document inference.

The ablation study (Exp-3, Section 7, pp. 12–13) validates that removing the graph join causes a sharp accuracy drop (from 85.2% to 63.0%), proving that structural alignment is critical. The contradiction analysis (Exp-4, p. 13) explains how DocChecker addresses the limitations of LLMs. Specifically, in order to mitigate the attention dilution that LLMs experience over long sequences [89], DocChecker aligns isolated texts into a unified graph structure, enabling precise logical reasoning across distant entities. This approach allows our method to accurately identify intra- and inter-document contradictions on ContraDoc. In these long-document scenarios, which account for 28.4% of the cases in ContraDoc, LLMs underperform DocChecker by 15.2% in accuracy due to long-range dependency failures. By maintaining robust performance on these conflicts, DocChecker validates the need for graph-based reasoning for detecting complex inter-document contradictions.

[R3O2 d)] *In many knowledge graph systems, entity resolution/linking holds the key to success, and these metrics start to kick in more at scale (millions of entities). How does this impact cross-document contradiction detection - does it contribute to a low score? How many cases of false positive contradictions are tied to bad entity resolution?*

[A] Thanks! Yes, entity resolution (ER) is crucial for inter-document detection, as incorrect linking (*e.g.,* two mentions of "Elon Musk") may produce spurious contradictions (Example 5). We mitigate this by linking entities via ϵ-edges rather than merging nodes, preserving document-level context. This highlights the importance of accurate entity alignment for reliable contradiction detection.

To quantify the impact of ER errors, we have added an ablation study in Exp-3 (Table 6, pp. 12–13). Results show that inaccurate ER is a primary source of false positives: disabling SBERT and HGCN increases FPR by 21.2% and reduces accuracy by 16.7% (from 85.2% down to 68.5%). This confirms that ϵ-edge linking is critical for distinguishing true document-level contradictions.

[R3O2 e)] *Expansion on meta-learning block - it would be nice to see how and what part of this process is core/critical and contributes to higher accuracy.*

[A] As suggested, we have added the ablation study to Exp-3 of Section 7 (pp. 12–13), which analyzes the impact of each components in the RICO discovery process on rule quality and efficiency.

We report the following. (1) Regarding LLM-based rule generation, replacing the LLM-driven initialization with a levelwise strategy drops the accuracy by 38.0%, which shows the critical role of LLM in capturing complex semantic preconditions. (2) For meta-learning feedback, disabling the iterative few-shot guidance decreases accuracy and F1 by 16.6% and 19.9%, respectively, which highlights the need for pattern refinement. (3) For rule verification, removing the filtering produces inconsistent rules, which slashes the accuracy to 54.7% and verifies the efficacy of failure counter in pruning noisy candidates. This confirms that rule generation, refinement and verification are all essential for high accuracy.

Moreover, we have analyzed the parallel rule discovery mechanism and demonstrated the impact on efficiency (Figures 4(g)–(h), p. 12). As the number of processors increases from 1 to 16, the system exhibits near-linear scalability. More specifically, the runtime of rule learner falls drastically (*e.g.,* from 2,465s to 408s on ContraDoc) since parallel workers effectively mask LLM latency. DocChecker demonstrates robust parallel scalability, which reduces total execution time from 7,410s to 974s on ContraDoc.

Many thanks for your support and constructive suggestions!

Response to the comments of Referee #4.

[R4O1 & Q1] *- Rules discovered in the rule discovery stage are assessed every three iterations – could the authors provide a reasoning behind the choice of the number of iterations?*

- Please justify the choice of three iterations in rule discovery.

[A] Many thanks! We have revised Section 6.2 (pp. 8–9) to treat k as a configurable parameter rather than a constant. Setting k as 3 works well in practice, which is supported by prior research on iterative LLM refinement [94] where LLM outputs and feedback typically stabilize within 2 to 4 iterations. We have also observed that the performance remains stable given $2 \leq k \leq 4$, which indicates that our method is rather insensitive to this parameter. Thus, setting $k = 3$ strikes a practical balance between the quality of rule refinement and overall computational cost.

[R4O2] *While scalability of* DocChecker *has been provided, its efficiency has not been compared against the provided baselines. This absence of efficiency results raises a question on the applicability and benefit of* DocChecker.

[A] Thanks! As suggested, we have added an experiment to Exp-1 in Section 7 (p. 11) to compare the time and token consumption of DocChecker against all baselines under identical settings.

As shown in Table 4, DocChecker achieves high efficiency and low cost. On Task 1 (16.3k tokens), it runs in 1.92s, much faster than Llama-3-70B (5.68s). On Tasks 2–3, it incurs minimal overhead (0.13s, 0.0 tokens), whereas Gemini-3-Pro requires 3.63s and 2.4K additional tokens. These show that DocChecker's graph-based design avoids heavy long-context prompting, substantially reducing latency and token usage, and enabling high-throughput processing.

Many thanks for your support and helpful suggestions!

Detecting Contradictions Within and Across Documents

Paper ID: 804

ABSTRACT

This paper presents DocChecker, a system for detecting textual contradictions. Given a document collection $\mathcal{D}$, DocChecker identifies conflicting properties or relationships assigned to the same entity, both within and across documents. It adopts a graph-based paradigm: entities and their relations are extracted, represented as a property graph G, joined across documents, and enriched with domain knowledge. DocChecker introduces a class of rules that detect vertex- and edge-level contradictions in PTIME, unifying logical reasoning, ML predictions and NLP tools. We present how graphs are extracted, joined and enriched, and propose a parallel rule discovery algorithm with new generation, evaluation and validation strategies tailored for sparse contradictions. Using real-life documents, we empirically verify that DocChecker detects intra- and inter-document contradictions with an average F1-score of 0.900, up to 0.935, and it scales to large collections $\mathcal{D}$ in parallel.

1 INTRODUCTION

A contradiction is a condition in which an entity is simultaneously assigned two or more properties or relationships that lead to a logical inconsistency, *i.e.,* they are logically or semantically incompatible at the same time, such that their joint validity violates domain constraints or undermines internal consistency [17, 84, 117].

Documents contain various contradictions. Examples include: (a) event/entity inconsistencies, *e.g.,* "All donors are between 20–45 years old" vs. "Lisa, who donates her kidney, is 70"; (b) conflicting opinions, *e.g.,* "The project is a breakthrough" vs. "The project had no impact"; and (c) mutually exclusive relations/knowledge, *e.g.,* "Jane and Tom are a married couple" vs. "Jane is Tom's sister" (cf. [84]). Such contradictions may occur within a single document (*intra-document*) or across related documents (*inter-document*).

Contradictions in documents are common: 21% of news stories contain contradictions [20], and at least 3.3% of facts in the 2025 Wikipedia snapshot are inconsistent. Furthermore, growing amount of AI-generated content makes this problem worse due to hallucinations: 48%–82% of generated text is affected, and factual consistency can drop to as low as 21% in some datasets [104].

The prevalence of textual inconsistencies highlights the need for their detection, since such contradictions can undermine core tasks including information retrieval, named entity resolution (NER), retrieval-augmented generation (RAG), reading comprehension, medical text analysis, and LLM fine-tuning [117, 127, 133, 136].

Early work casts contradiction detection as sentence-pair classification (contradiction or not) [15, 21, 23, 59, 65, 112, 138] or textual entailment (checking whether a premise contradicts a hypothesis) [21, 97, 112, 138]. However, such NLI-based methods are "rather shallow" [62, 93, 140], as they rely on lexical overlap, negation and antonymy, not semantic reasoning. Moreover, cross-document conflicts require more than entity resolution (ER), often involving temporal or causal reasoning; *e.g.,* "Company X was founded in 1998" contradicts "Company X held a board meeting in 1995", since establishment must precede meetings, an inference beyond ER. Such conflicts are often missed by NLI models [84]. Even state-of-the-art (SOTA) LLMs are only 51.6% accurate [84] in conflict detection, due to noise and hallucinations [143], as they tend to favor parametric knowledge [16] over identifying conflicting evidence [72].

Challenges. Detecting textual contradictions is challenging.

(1) Context dependence. Contradictions may hinge on context that shifts within or across documents [17, 84]. While ER aligns entity mentions, contextual inference is necessary to catch the inconsistencies. For instance, "the vaccine is safe for children" and "the vaccine has not been tested on minors" seem consistent. However, they become contradictory given the knowledge that any safety claim must be supported by clinical testing.

(2) Linguistic variation. Contradictions may be masked by paraphrasing, negation, or figurative language [7, 113, 117]. The same claim can be phrased in multiple ways; syntactic differences ("X increases risk" vs. "risk is higher with X") or pragmatic nuances ("may cause" vs. "is known to cause") can obscure semantic opposition.

(3) Domain specificity. In specialized domains, *e.g.,* medicine, law, or science, contradictions depend on expert knowledge of terminology, causality and regulatory logic [135]. For example, "aspirin prevents blood clots" and "aspirin increases bleeding risk" are both true, but they reflect opposing medical implications of the same mechanism; one statement highlights the benefit of aspirin, the other the risk.

(4) Temporal dynamics. Information evolves over time, and statements that were once consistent may later contradict due to policy changes, scientific updates, or data revisions [84, 92, 116, 117]. For example, "masks are unnecessary" (early 2020) and "masks are required" (late 2020) differ not by error but by temporal context.

(5) Scalability. As documents and corpora grow longer, exhaustive pairwise comparison of statements becomes prohibitively expensive [17, 135]. Practical contradiction detection must therefore scale with large, evolving corpora without sacrificing accuracy.

These challenges raise several questions: (1) How can contradictions of different types be detected in a unified way, capturing contextual cues, linguistic variation, and temporal change? (2) How can external domain knowledge be incorporated to boost accuracy when dealing with specialized terminology and context? (3) How can systems efficiently and consistently detect contradictions both within a document and across multiple documents at scale?

DocChecker. To answer these questions, we develop DocChecker, a system for detecting diverse types of contradictions across a document collection $\mathcal{D}$, integrating contextual reasoning, domain knowledge, and scalable analysis. It offers the following features.

(1) Rules (Section 3). Central to DocChecker is a class of Rules for Identifying COntradictions (RICOs) over text and graph data. RICOs encode entities and relations as graph patterns, and integrate logical reasoning with ML/NLP methods to detect contradictions. They (a) combine reasoning and prediction to catch context and temporal change, (b) link graph vertices to document spans and use NLP

tools to resolve linguistic variation, and (c) infer conflicts in both attributes and relations, catching diverse textual inconsistencies.

(2) A graph-based framework (Section 4). Given a document collection $\mathcal{D}$, DocChecker adopts a graph-based paradigm in five stages: (a) convert each document into a property graph, enriching entities and relations with domain knowledge when available; (b) fuse the graphs by aligning vertices referring to the same entity, producing a unified graph G that brings distant statements into structural correspondence; (c) learn RICOs over entity-centric subgraphs sampled from G, reducing the search space while preserving patterns relevant to target entities [31]; (d) detect contradictions via rule-based parallel reasoning over G; and (e) trace detected conflicts back to their textual origins. This collapses contradictions scattered across long or multiple documents into aligned conflicts, enabling unified intra- and inter-document detection through multi-hop reasoning.

(3) Scalable algorithms (Section 6). We develop a suite of graph analytic algorithms underlying DocChecker, including (a) a graph join algorithm to fuse extracted graphs into a unified graph G and enrich it with domain knowledge from KG, (b) Learner, an algorithm for learning RICOs from (samples of) G, and (c) contradiction detection algorithms for identifying inconsistencies in G. These algorithms are designed to scale to long documents and large collections.

The algorithms improve upon prior work. Taking Learner as an example, it (a) replaces costly levelwise traversal [68], which enumerates preconditions step by step, with an LLM that directly generates candidate rules; (b) restricts rule generation to a contradiction-relevant subset $\mathcal{D}'$ where conflicts occur, avoiding full-dataset scans; (c) replaces classical support [37] with downstream performance metrics that suit sparse contradictions (*i.e.,* rare entity-attribute conflicts that are treated by support-based methods as noise); (d) introduces two thresholds (contradiction coverage and accuracy improvement) to ensure broad coverage of contradictions in $\mathcal{D}$; and (e) aligns entities and relations from $\mathcal{D}'$ with triples in G, using their attributes to generate RICO predicates.

(4) Effectiveness (Section 7). Using real-life documents, we empirically verify DocChecker's accuracy and scalability: (a) it achieves SOTA results on 15 datasets, including 93.1% F_1 on SNLI [15] and 87.8% accuracy on ContractNLI [75]; (b) it excels at long-range reasoning, with 76.81% (intra-) and 86.96% (inter-document) accuracy, beating GPT-4o (51.89%/65.06%); (c) in document-level self-contradiction detection, it provides verifiable evidence (0.962 F_1), exceeding Claude-3.7 and GPT-5.2; and (d) it scales with document size and entity count in parallel, reducing runtime from 7,410.11s to 974.44s with 16 processors, enabling large-scale deployment.

Novelty. Our key contributions are: (1) RICOs: Rules integrating logical, ML and NLP predicates over text-derived graphs to catch semantic contradictions beyond prior graph rules; (2) DocChecker: A graph-based framework for contextual reasoning across documents via graph construction, enrichment and fusion; (3) Learner: A learning-based rule discovery algorithm that departs from levelwise enumeration; and (4) Detector: A PTIME algorithm for parallel intra- and inter-document contradiction detection.

2 BACKGROUND AND RELATED WORK

This section reviews background and previous work.

Documents. Assume a countably infinite set of symbols W for tokens. A document D is represented as a finite token sequence $\langle w_1, w_2, \ldots, w_{|D|} \rangle$, where each token $w_i \in W$ denotes a word, punctuation mark or symbol. We assume the sequence is both syntactically valid and semantically coherent. A *substring* of D is a contiguous sequence of tokens $\langle w_i, \ldots, w_j \rangle$ for $1 \leq i \leq j \leq |D|$.

Contradictions. Given documents D and D', a *contradiction* arises if there exist substrings $s \subseteq D$ and $s' \subseteq D'$ that refer to the same entity or relationship o, but assign mutually exclusive values to at least one attribute or property of o under the domain semantics. If $D = D'$, it is *intra-document*. If $D \neq D'$, it is *inter-document*.

Prior research in the NLP community [84, 110, 117] classifies contradictions into seven distinct types, with examples shown in Table 1: (1) *Factual*: contradictions requiring external world knowledge to verify; (2) *Content* (misinformation, emotion, opinion, semantic, and lexical types): conflicts in the attributes of entities or relationships; (3) *Causal*: contradictions where the result does not align with the cause under a clear causal relationship; (4) *Negation*: explicit negation or antonymy (*e.g.,* use of "not," "never," or antonyms); (5) *Relational*: conflicts in entity relationships, such as reversed, missing, or mismatched links; (6) *Numeric*: contradictions involving mismatched or out-of-range numbers; and (7) *Temporal*: contradictions caused by conflicting statements across different time points.

Detection. Given a document collection $\mathcal{D}$, *contradiction detection* identifies all contradictory substring pairs (s, s') where $s \in D$ and $s' \in D'$, for $D, D' \in \mathcal{D}$, and outputs their contradiction types T.

Research on contradiction detection spans social media [80, 81, 83], financial reporting [25] and biomedical literature [5, 6, 108, 110]. Early work focused on sentence-level contradictions framed as text classification or NLI tasks [15, 21, 23, 59]. Later studies extend to event-level contradictions (*e.g.,* conflicting reports placing a summit in Paris vs. Berlin) and inconsistent dialogue responses (*e.g.,* an agent claiming "I am a strict vegetarian" but later "I love eating steak") [24, 51, 71, 100, 144]. Recent work addresses document-level detection via zero-shot inference (*i.e.,* performing contradiction detection without task-specific training examples), multi-hop reasoning (*i.e.,* the process of combining multiple pieces of information to derive the conclusion), and LLM-based methods (*i.e.,* using pre-trained LLMs to predict without extra training) [84, 97, 112, 117, 138, 139]. Benchmarking across GPT-3.5, GPT-4, PaLM-2, and LLaMA-2 shows GPT-4 leading in accuracy and evidence localization [84]. However, gaps still remain: LLMs lack interpretability, suffer from hallucinations, and often struggle with subjective or long-range contradictions [84, 117].

Unlike prior methods, this work proposes a graph-based paradigm for textual contradiction detection: (1) entities and relationships of interest are extracted into graphs, enabling uniform detection of diverse contradictions over both vertices and edges in long reasoning; (2) graphs are "joined" together and enriched with domain knowledge, capturing domain semantics and uniformly catching both intra- and inter-document contradictions; (3) vertices and edges are bound to substrings and coupled with NLP predicates, handling linguistic variation and ambiguity; (4) logic and ML predicates are integrated to reason about inconsistencies, account for context dependence and temporal dynamics, and provide interpretable logical explanations; (5) the process can be conducted in PTIME.

Types	Sentence 1	Sentence 2	RICOs
(1) Factual	"The Nobel Prize was indeed established in 1901 to commemorate Alfred Nobel"	"Hinton received the Nobel Prize in 2024, which was established in 1901 to honor Shannon's contributions to the field of information."	Rule φ_1
(2) Content	"William was engaged while reading the paper on computer architecture"	"William found the content of the paper on computer architecture to be extremely boring"	Rule φ_2
(3) Causal	"David's video, which was published on YouTube in 2024, was permanently removed that year due to violence."	"Taylor came across this video published by David while browsing YouTube in 2025."	Rule φ_3
(4) Negation	'On April 20, 2025, the router experienced a brief routing disruption at 2:00 PM"	"The router did not report any routing disruptions throughout April 20, 2025."	Rule φ_4
(5) Relational	"The owner of the brand store, Howard, and his wife Sheila have a harmonious relationship after marriage."	"Howard and his wife Emma have a beloved little daughter named Sheila"	Rule φ_5
(6) Numeric	"Howard's brand store sold 29 pairs of women's high heels on June 15, 2025."	"Howard is concerned about the lack of any sales on June 15, 2025."	Rule φ_6
(7) Temporal	"Around 1860, Nobel and other researches believed that nitroglycerin could not treat heart disease"	"After Nobel's death, further research revealed that nitroglycerin is effective for treating heart disease"	Rule φ_7

Table 1: Examples of Contradictions and Corresponding Detection Rules

Inconsistency detection. Inconsistency detection has been extensively studied in relational [4, 9, 12, 22, 33, 34, 40, 43, 61, 95, 119, 125] and graph data [35, 37, 38, 41, 45]. Early efforts focused on consistent query answering and constraint-based repair [9, 22, 33, 34], and later extended to ML-based conflict detection and entity resolution [4, 40, 61, 95, 119]. For graphs, research explored dependencies, association rules, rule discovery and data repair [35, 38, 41, 45].

Data cleaning has also been integrated into ML training pipelines, including interactive tools [47], AutoML systems [49], provenance debugging [50, 132], differentiable preprocessing [63, 85], iterative repair [77, 78], boosting-based [76], fairness-aware [120], weak supervision [106], and self-supervised methods [91]. Recent work unifies cleaning with integration, fairness, and federated learning [57, 73, 120], and assesses its influence on ML performance [86].

DocChecker differs from prior studies as follows. (a) Unlike traditional methods for relational tables or semi-structured graphs, it addresses textual contradictions, resolving conflicts in documents and handling linguistic ambiguity absent in earlier work. (b) It expends graph cleaning rules (GCRs) [32, 37] to support expressive contradiction detection in text-derived graphs (Section 3.2). (c) It uniformly detects intra- and inter-document contradictions by leveraging graph join and fusion to integrate multi-document evidence and trace detected conflicts back to their textual sources. (d) We also develop new rule discovery methods. Since prior discovery methods are ineffective for sparse tasks like contradiction detection, we propose new generation, evaluation and validation strategies for learning rules effectively in sparse settings (Sections 1 and 6).

3 CONTRADICTION IDENTIFYING RULES

This section first extends the notion of dual patterns of [32] (Section 3.1), then defines RICO (Section 3.2), and finally settles the complexity of contradiction detection with RICOs (Section 3.3).

3.1 Dual Graph Patterns

Graphs. A *graph* is $G = (V, E, L, F_A)$, where (a) V is a finite set of vertices; (b) E is a finite set of labeled edges, with each $e = (v, l, v') \in E$ denoting an edge from v to v' labeled l; (c) L assigns a label to each vertex; and (d) F_A assigns to each vertex $v \in V$ (resp. edge $e \in E$) a tuple of attributes $(A_1 = a_1, \ldots, A_n = a_n)$, written as $v.A_i = a_i$ (resp. $e.A_i = a_i$). Unlike relational data, vertices and edges may carry heterogeneous attributes without a fixed schema.

As opposed to graphs studied in prior work [32, 35, 37, 45] where only vertices are associated with attributes, we consider graphs in which both vertices and edges carry attributes (properties).

Paths. A *path* ρ from vertex v_0 in a graph $G = (V, E, L)$ is a sequence $(v_0, l_0, v_1, \ldots, v_{n-1}, l_{n-1}, v_n)$ such that $(v_{i-1}, l_{i-1}, v_i) \in E$ for all $i \in [1, n]$. We focus on *simple* paths, where no vertex repeats. For an edge $(u, l, v) \in E$, v is a *child* of u, and u is a *parent* of v.

Stars. A *star pattern* is $Q[x_0, \bar{x}, \bar{e}_x] = (V_Q, E_Q, L_Q)$, where (1) V_Q (reps. E_Q) is the set of pattern vertices (resp. edges); (2) L_Q assigns labels to vertices in V_Q; (3) $\bar{x}$ and $\bar{e}_x$ are lists of distinct vertex and edge variables, each uniquely denoting an element in V_Q and E_Q; (4) $x_0 \in \bar{x}$ is the designated *center*; and (5) Q is *star-shaped*: each vertex $x \in \bar{x}$ has a unique path from x_0 and, if $x \neq x_0$, at most one child.

Dual patterns. A *dual pattern* is $\langle Q_x[x_0, \bar{x}, \bar{e}_x], Q_y[y_0, \bar{y}, \bar{e}_y]\rangle$, denoted $\boldsymbol{Q}[x_0, y_0]$, where Q_x and Q_y are disjoint star patterns. Intuitively, $\boldsymbol{Q}$ couples two entities x_0 and y_0, each with its own feature leaves, so that their properties can be compared pairwise.

Matches. A *match of a star pattern Q in a graph G* is a homomorphism h from the pattern vertices and edges in Q to G such that (a) for every $u \in V_Q$, $L_Q(u) = L(h(u))$; and (b) for every edge $(u, l, u') \in E_Q$, $(h(u), l, h(u')) \in E$ holds in G. Similarly, a *match of a dual pattern* $\boldsymbol{Q}[x_0, y_0] = \langle Q_x, Q_y\rangle$ in G is a homomorphism h from $V_{Q_x} \cup V_{Q_y}$ and $E_{Q_x} \cup E_{Q_y}$ to V and E of G, respectively.

Observe that $h(x_0)$ and $h(y_0)$ may be disconnected in G, which enables comparison of entities without direct links and with differing topological structures in a schemaless graph.

Example 1: Figure 1 depicts seven dual patterns $\boldsymbol{Q}_1$–$\boldsymbol{Q}_7$, each with two centers (x_0, y_0) and multiple paths. For example, in $\boldsymbol{Q}_1$, a path is (y_0, honor, y_1); and in $\boldsymbol{Q}_5$, a path is $(x_0, \text{owner}, x_1, \text{wife}, x_2)$. □

It is in PTIME to check whether a dual pattern $\boldsymbol{Q}$ has a match in a graph G [37]. In contrast, it is NP-complete to decide whether there exists a match of generic patterns in G (cf. [52]).

3.2 Rules for Identifying Contradictions

We next define RICOs, starting with their predicates.

Predicates. A *predicate* q of RICOs over a document collection $\mathcal{D}$ and a dual pattern $\boldsymbol{Q}[x_0, y_0] = \langle Q_x[x_0, \bar{x}, \bar{e}_x], Q_y[y_0, \bar{y}, \bar{e}_y]\rangle$ is:

$$q \quad ::= \quad S(x, s) \mid S(e_x, s) \mid \mathcal{M}(x.\bar{A}, y.\bar{B}) \mid \mathcal{M}(e_x.\bar{A}, e_y.\bar{B}) \mid x.A \oplus y.B \mid z.A \oplus c \mid e_x.A \oplus e_y.B \mid e_z.A \oplus c \mid T(s, s', \tau) \mid \tau(x.A, y.B) \mid \tau(e_x.A, e_y.B).$$

Here s is a *string variable* for a substring in a document; $x \in \bar{x}$ and $y \in \bar{y}$ (resp. $e_x \in \bar{e}_x$ and $e_y \in \bar{e}_y$) are *vertex variables* (resp. *edge variables*) in Q_x and Q_y, respectively; $z \in \bar{x} \cup \bar{y}$ (resp. $e_z \in \bar{e}_x \cup \bar{e}_y$); c is a constant in U; A and B are attributes in Υ; τ is a contradiction

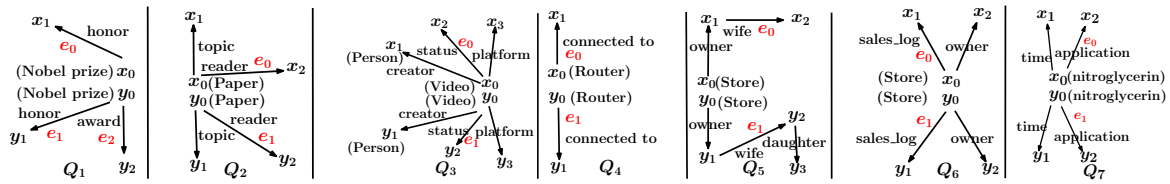


Figure 1: Example dual patterns

type such as negation or casual; and $\oplus$ is in $\{=, \neq, <, \leq, >, \geq\}$.

The predicates are classified as follows:

- Substring binding: $S(x, s)$ (resp. $S(e_x, s)$) binds a substring s to a vertex x (resp. edge e_x).
- Textual classifier: $T(s, s', \tau)$ is a NLP/NLI/LLM model that classifies whether substrings s and s' are in contradiction of type τ.
- ML predicate: $\mathcal{M}$ is a pretrained ML model returning a Boolean value (*e.g.,* prediction strength above a threshold), where $x.\bar{A}$ (resp. $e_x.\bar{A}$) denotes a list of attributes on vertex x (resp. edge e_x).
- Comparison: $x.A \oplus y.B$ and $z.A \oplus c$ (resp. $e_x.A \oplus e_y.B$, $e_z.A \oplus c$) compare values of attributes, where $\oplus \in \{=, \neq, <, \leq, >, \geq\}$.
- Type: $\tau(x.A, y.B)$ is true if and only if (iff) $x.A$ and $y.B$ are in contradiction of type τ (similarly for $\tau(e_x.A, e_y.B)$).

Textual classifiers $T(s, s', \tau)$ are pretrained NLP/NLI/LLM models, *e.g.,* (1) SBERT [107] detects attribute-level and negation/antonym contradictions; (2) NumNet [105] handles numeric reasoning; (3) knowledge_graph_maker [30] extracts relations for relational contradictions; (4) Flan-T5-base [18] captures temporal dynamics; (5) GPT-5.2 with retrieval [101] addresses factual contradictions; and (6) Prompted GPT-5.2 [101] detects causal and broad content, misinformation, emotion, opinion, semantic and lexical inconsistencies.

ML predicates $\mathcal{M}(x.\bar{A}, y.\bar{B})$ and $\mathcal{M}(e_x.\bar{A}, e_y.\bar{B})$ can be implemented as pretrained graph-based models, including (1) graph-based NLI classifiers (*e.g.,* GraphSAGE [58], GAT [124]) that embed structural information to detect entailment, neutrality, and contradiction between vertices and edges, (2) graph semantic models (*e.g.,* GraphBERT [142], GCN [74]) that capture event similarity and contextual dependencies among vertices and edges, and (3) graph-based prompt-driven models (*e.g.,* HGT [67]) that extend transformers to infer temporal and causal relationships in graphs.

Rules. A Rule for Identifying COntradictions (RICO) φ has the form:

$$Q[x_0, y_0](X \rightarrow q_0),$$

where $Q[x_0, y_0] = \langle Q_x[x_0, \bar{x}, \bar{e}_x], Q_y[y_0, \bar{y}, \bar{e}_y] \rangle$ is a dual pattern, X is a conjunction of predicates and q_0 is a predicate, both over $\mathcal{D}$ and Q. Moreover, (1) each binary predicate in X is defined on either two leaves x and y in Q_x and Q_y or on their incident edges e_x and e_y, and (2) each leaf and its edge carry at most one binary predicate.

We refer to Q and $X \rightarrow q_0$ as the *pattern* and *dependency* of φ, and to X and q_0 as *precondition* and *consequence* of φ, respectively.

Intuitively, restrictions (1) and (2) above balance expressivity with efficiency following [37] (see Section 3.3). We separate Q and X to (a) visualize the topological features of x_0 and y_0 via Q, and (b) accelerate evaluation by exploiting the locality of pattern matching.

To see the expressivity of RICOs, we compare them with graph cleaning rules (GCRs) [37]. While GCRs are restricted to vertex-level equality and basic similarity, RICOs introduce (1) NLP classifiers for semantic conflicts, (2) edge-level reasoning, (3) structural inference for contextual dependencies, and (4) rich comparison logic (*e.g.,* inequalities) to catch complex numeric and temporal contradictions (see Appendix A in [1] for comparison with prior rules).

Example 2: We illustrate how learned RICOs detect the seven contradiction types in Table 1, using the dual patterns in Figure 1.

(1) *Factual contradiction.* RICO φ_1 : Q_1 $[x_0, y_0]$ (x_0.title = Nobel Prize$\wedge y_0$.title = Nobel Prize$\wedge \mathcal{M}(x_0$.description, y_0.description)$\wedge$ y_2.name = Hinton $\wedge$ e_2.time=2024 $\wedge$ e_2.award field = physics $\wedge$ x_1.name = Nobel $\wedge$ y_1.name=Shannon $\wedge$ $S(e_0, s)$ $\wedge$ $S(e_1, s')$ $\wedge$ $T(s, s',$ factual) $\rightarrow$ y_1.name = Nobel). It identifies the factual contradiction in case (1) of Table 1. The contradiction arises because the two awards have nearly identical descriptions (detected by the NLI model $\mathcal{M}(x_0$.description, y_0.description)) but list different honorees (x_1.name = Nobel vs. y_1.name = Shannon). Using external knowledge and LLM ($T(s, s',$ factual)), the rule infers y_1.name = Nobel. GCRs fail as they (a) lack substring binding ($S(e, s)$) for text localization; (b) skip edge attributes (*e.g.,* e_2.time); and (c) restrict comparisons to equality without NLI models (T).

As noted in Section 4, DocChecker's chase process captures textual bindings and logical conflicts (*e.g.,* x_1.name $\neq$ y_1.name) via φ_1. Exporting its consequence as a truth value provides users with explicit, verifiable evidence for inspection. This evidence consists of the matched supporting facts (the match of NLI model) and the rule application trace, which together explain why the claims conflict.

(2) *Content contradiction.* RICO φ_2: Q_2 $[x_0, y_0]$ ($S(e_0, s) \wedge S(e_1, s') \wedge$ $\mathcal{M}(x_1$.field, y_1.field) $\wedge$ x_2.name = William $\wedge$ y_2.name = William $\wedge$ e_0.rating $\neq$ e_1.rating $\rightarrow$ emotion(e_0.rating, e_1.rating)). This rule finds an emotion contradiction on edge attributes (case (2) of Table 1), where the same reader (x_2.name = William $\wedge$ y_2.name = William) has conflicting comments (engaged versus boring) on the same paper (identified by $\mathcal{M}(x_1$.field, y_1.field)). GCRs fail here since they lack edge attributes, inequality predicates ($\neq$), and the text binding ($S(e, s)$) to capture content conflicts.

(3) *Causal contradiction.* RICO φ_3: Q_3 $[x_0, y_0]$ ($S(x_2, s) \wedge S(y_2, s')$ $\wedge$ x_1.name = David $\wedge$ y_1.name = David $\wedge$ x_3.name = y_3.name $\wedge$ x_2.details = removed $\wedge$ e_0.removed_time $\leq$ e_1.viewing_time $\wedge$ y_2.details = viewable $\rightarrow$ causal(x_2.details, y_2.details)). The rule catches a causal contradiction (case (3) of Table 1), as a video cannot be accessible (y_2.details = viewable) once it has been removed (x_2.details = removed $\wedge$ e_0.removed_time $\leq$ e_1.viewing_time). GCRs fail here because they lack temporal inequalities ($\leq$), ignore edge timestamps (e_0, e_1), and restrict consequences to equality, which preclude semantic predicates such as causal.

(4) *Negation contradiction.* RICO φ_4: Q_4 $[x_0, y_0]$ ($S(e_0, s) \wedge S(e_1, s') \wedge$ x_1.ip = y_1.ip $\wedge$ e_0.date = e_1.date $\wedge$ e_0.interrupt = true $\wedge$ e_1.interrupt = false $\rightarrow$ negation(e_0.interrupt, e_1.interrupt)). This rule catches a negation contradiction on edge properties (case (4) of Table 1). Here Q_x identifies that the router reported an interruption (e_0.interrupt = true) at 2:00 PM on April 20, 2025, whereas Q_y indicates no interruption (e_1.interrupt = false) on the same day. GCRs fail to capture this as they lack negation predicate in consequences, edge attribute comparison that aligns events, and substring binding ($S(e, s)$) which verifies semantic intents like "interruption".

(5) *Relational contradiction.* RICO φ_5: $Q_5[x_0, y_0]$ $(S(e_0, s) \wedge S(e_1, s') \wedge x_1.\text{name} = y_1.\text{name} \wedge x_2.\text{name} \neq y_2.\text{name} \wedge e_0.\text{time} = e_1.\text{time} \wedge T(s, s', \text{relation}) \rightarrow \text{relational}(e_0.\text{spouse}, e_1.\text{spouse}))$. This rule catches a relational contradiction that Howard cannot be married to both Sheila and Emma at the same time (case (5) of Table 1). Note that classifier $T(s, s', \text{relation})$ captures context dependence: the contradiction becomes clear only when the contexts of Q_x and Q_y are considered together, showing that Howard cannot have two different wives simultaneously ($e_0.\text{time} = e_1.\text{time}$). GCRs fail here since they lack inequality ($\neq$) between entities and edge-level temporal alignments ($e_0.\text{time} = e_1.\text{time}$) to verify simultaneity.

(6) *Numeric contradiction.* RICO φ_6: $Q_6[x_0, y_0]$ $(S(e_0, s) \wedge S(e_1, s') \wedge x_2.\text{name} = y_2.\text{name} \wedge e_0.\text{sold items} = \text{women's high heels} \wedge e_0.\text{sale quantity} = 29 \wedge e_1.\text{sold items} = \text{women's high heels} \wedge e_1.\text{sale quantity} = 0 \wedge e_0.\text{date} = e_1.\text{date} \rightarrow \text{numeric}(e_0.\text{sale_quantity}, e_1.\text{sale_quantity}))$. The rule catches a numerical contradiction on the same item that was sold by a store on the same day: sentence 1 reports 29 women's high heels were sold, whereas sentence 2 claims 0 (case (6) of Table 1). GCRs fail here because they cannot handle with numeric edge attributes, lack numeric predicates in consequence to express quantitative conflicts.

(7) *Temporal contradiction.* RICO φ_7: $Q_7[x_0, y_0]$ $(S(x_2, s) \wedge S(y_2, s') \wedge \mathcal{M}(x_1.\text{time}, y_1.\text{time}) \wedge x_2.\text{treatment} \neq y_2.\text{treatment} \rightarrow \text{temporal}(x_2.\text{treatment}, y_2.\text{treatment}))$. Here $\mathcal{M}$ is an ML model that determines chronological ordering, returning true if $x_1.\text{time}$ precedes $y_1.\text{time}$, *e.g.,* Flan-T5-base model. In this case, $x_1.\text{time}$ is around 1860, while $y_1.\text{time}$ falls after Nobel's death. The rule captures a temporal contradiction: while sentence 1 states that nitroglycerin could not treat heart disease during Nobel's lifetime, sentence 2 claims it was effective after his death (case (7) of Table 1). GCRs fail here as they lack text binding and temporal predicates.

The RICO φ_7 integrates NLP predicates with logical interpretability: LLM-based $\mathcal{M}(x_1.\text{time}, y_1.\text{time})$ infers temporal order, while a temporal predicate detects conflicting treatments ($x_2.\text{treatment} \neq y_2.\text{treatment}$). We also employ differentiable methods [63, 85] to jointly optimize ML models and preprocessing by making preprocessing trainable via gradients, allowing end-to-end learning. □

Semantics. Consider RICO $\varphi = Q[x_0, y_0](X \rightarrow p_0)$. A *valuation* of φ is a match h of $Q[x_0, y_0]$ in graph G, extended with bindings of string variables to substrings in document collection $\mathcal{D}$.

We say that valuation h *satisfies* a predicate q, written $h \models q$, if (a) $q = S(x, s)$: vertex $h(x)$ is bounded to substring $h(s)$ in $\mathcal{D}$ (similarly for $q = S(e_x, s)$); (b) $q = x.A \oplus y.B$: $h(x).A \oplus h(y).B$ holds (similarly for $z.A \oplus c$, $e_x.A \oplus e_y.B$ and $e_z.A \oplus c$); (c) $q = \mathcal{M}(x.\bar{A}, y.\bar{B})$: the ML model $\mathcal{M}$ predicts true on features $h(x).\bar{A}$ and $h(y).\bar{B}$ (similarly for $q = \mathcal{M}(e_x.\bar{A}, e_y.\bar{B})$); (d) $q = T(s, s', \tau)$: the textual classifier T assigns contradiction type τ to substrings $h(s)$ and $h(s')$; (e) $q = \tau(x.A, y.B)$: the contradiction between vertex attributes $h(x).A$ and $h(y).B$ is classified as type τ (similarly for $q = \tau(e_x.A, e_y.B)$).

For precondition X, we write $h \models X$ if valuation h satisfies *all* the predicates in X. We write $h \models X \rightarrow q_0$ if $h \models X$ implies $h \models q_0$. We say that graph G and documents $\mathcal{D}$ *satisfy* a RICO φ, written $(G, \mathcal{D}) \models \varphi$, if for all valuations h of $Q[x_0, y_0]$ over G and $\mathcal{D}$, $h \models X \rightarrow q_0$. Similarly, $(G, \mathcal{D}) \models \Sigma$ if $(G, \mathcal{D}) \models \varphi$ for every $\varphi \in \Sigma$.

3.3 The Complexity of Contradiction Detection

We study the *validation problem*, stated as follows.

- Input: A graph G, a document collection $\mathcal{D}$, and a set Σ of RICOs.
- Question: Does $(G, \mathcal{D}) \models \Sigma$?

This problem studies the complexity of contradiction detection.

The validation problem is coNP-hard for GARs [35] but solvable in PTIME for GCRs [32]. Despite being more expressive than both (by supporting edge predicates, substring bindings, and textual classifiers), RICOs still preserve tractability. This contrasts with general graph rules on arbitrary patterns, which are typically intractable. Thus, RICOs strike a balance between expressiveness and efficiency, enabling scalable contradiction detection.

Theorem 1: *The validation problem is in* PTIME *for* RICOs. □

Proof sketch: An algorithm for RICO validation proceeds as follows: for each φ, it checks whether there exists no valuation h of Q over $(G, \mathcal{D})$ such that $h \models X$ but $h \not\models q_0$. (1) It first locates matches of leaves satisfying X but violating q_0. (2) It then deduces matches of other vertices in a bottom-up manner. The procedure runs in PTIME, since evaluating predicates such as $S(x, s)$, $T(s, s', \tau)$, $\mathcal{M}(x.\bar{A}, y.\bar{B})$, $\mathcal{M}(e_x.\bar{A}, e_y.\bar{B})$, $\tau(x.A, y.B)$ and $\tau(e_x.A, e_y.B)$ is polynomial, and star-pattern matching is also in PTIME [32]. □

4 AN OVERVIEW OF THE SYSTEM

As shown in Figure 2, DocChecker takes a document collection $\mathcal{D}$ and an optional target entity set $\mathcal{E}$ as input. It catches intra- and inter-document contradictions in five sequential steps as follows.

(1) Graph conversion. For each document D in $\mathcal{D}$, DocChecker converts D into a graph G_D using pretrained ML models and LLM, extracting only vertices and edges for entities in $\mathcal{E}$ to exclude irrelevant objects. It records each entity's position in D so that contradictions in G_D can be mapped back to the text. Logically, it defines a relation $S(x, s)$ linking vertices to substrings; physically, this yields a mapping f_{index} from G_D's vertices to entity positions in D.

In order to incorporate specialized knowledge in the domain and enrich the converted graph, DocChecker joins each G_D and knowledge graph KG when available. The join is conducted only on entities in $\mathcal{E}$, *i.e.,* it extracts from KG only relevant knowledge of users' interest. This yields an enriched graph, also denoted as G_D.

(2) Graph fusion. To synthesize the interplay of multiple statements across sentences, sections and documents, DocChecker fuses all graphs G_D into a single graph G. It is carried out by computing the "join" of G_D's, such that vertices across different graphs are "connected" in G if they refer to the same real-world entity.

(3) Rule discovery. Taking a sample of the fused and enriched graph G, DocChecker learns a set Σ of high-quality RICOs with document-specific patterns and domain-specific context, in parallel.

(4) Contradiction detection. DocChecker then applies the set Σ of RICOs to G in parallel, to detect contradictions of vertices and edges in G, intra-document and inter-document in a unified process. It guarantees the parallel scalability [79] to scale with large $\mathcal{D}$.

(5) Document marking. Using the mapping f_{index} from graphs to documents, DocChecker maps the detected contradictions back to the documents of $\mathcal{D}$, by marking the corresponding text in position.

Below we outline the process of steps 4 and 5 (see details in [1]).

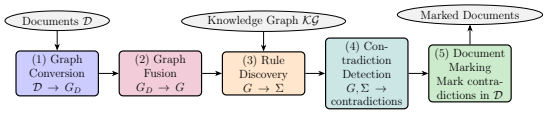


Figure 2: The workflow of DocChecker

We will present steps 1-2 and 3 in Sections 5 and 6, respectively.

Contradiction detector. Step 4 of DocChecker detects conflicts by chasing graph G with RICOs Σ, called Detector (see [1] for details).

Facts. For each attribute $x.A$ (resp. $e_x.A$), the system maintains relations $[x.A]_\otimes$ (resp. $[e_x.A]_\otimes$), where $\otimes \in \{=, \neq, <, \leq, >, \geq\}$. Each $[x.A]_\otimes$ records the attributes and constants that satisfy the $\otimes$ relation with $x.A$, as well as contradiction-type labels; these are derived from a set Γ containing both the initial ground truth Γ_0 and facts generated during the chase. All $[x.A]_\otimes$ (resp. $[e_x.A]_\otimes$) are valid, *e.g.,* no conflicting constants $c_1 \neq c_2$ may coexist in $[x.A]_=$.

Ground truth. Ground truth Γ_0 is a set of consistent facts used as the baseline for reasoning. It is collected from trusted sources and checked by experts (*e.g.,* DBpedia [10]), and fixes key entities and attributes (*e.g.,* setting $[x.A]_=$ to constants). A contradiction is detected only when a RICO finds a conflict that violates the facts.

The chase. For a RICO $\varphi : Q[x_0, y_0](X \to q_0)$ and valuation h satisfying premise X, if current facts $\mathcal{F}$ validate X, consequence q_0 is added to $[x.A]_\otimes$, yielding extended set $\mathcal{F}'$ (*i.e.,* Γ). Formally, a *chase step* of graph G with Σ at a set $\mathcal{F}$ of facts is $\mathcal{F} \Rightarrow_{(\varphi,h)} \mathcal{F}'$, which applies φ under the valuation h to produce the updated fact set $\mathcal{F}'$. That is, the chase expands facts Γ with validated facts in the process. A *chasing sequence* ξ is a sequence $\mathcal{F}_0, \ldots, \mathcal{F}_n$, where $\mathcal{F}_0 = \Gamma_0$ and each $\mathcal{F}_{i-1} \Rightarrow_{(\varphi,h)} \mathcal{F}_i$ is valid, *i.e.,* it incurs no conflicts.

Chase ξ terminates when no rule can extend $\mathcal{F}_n$. When ξ terminates, we refer to the set $\mathcal{F}_n$ of facts as the result of ξ.

Church-Rosser property. The chase ensures deep contradiction detection. Under correct rules and accurate predicates, only correct facts are deduced. Further, the chase is Church-Rosser, *i.e.,* the fixpoint is unique regardless of the order in which rules are applied [2].

Proposition 2: *The chase over G with Σ is Church-Rosser.* □

Proof sketch: The proof has two steps. (1) *Finiteness*: Chase steps add facts to a finite number of attributes, bounded by $O(|\Gamma||G| + |G|^2)$. (2) *Convergence*: If two terminal results differ, a missing fix would apply to both, contradicting terminality (see [1]). □

Parallel chasing. Detector implements the chase using the Bulk Synchronous Parallel (BSP) model [123], using one master and K workers. The graph G and document collection $\mathcal{D}$ are partitioned across K workers: (1) each document graph G_D and its document D are colocated on the same worker; (2) all dual-star patterns $Q[x_0, y_0]$ sharing the same x_0 (resp. y_0) and their contexts are placed in the same partition to ensure locality; (3) when G_D or D is large, the associated $Q[x_0, y_0]$ and contexts are further partitioned for load balancing; and (4) each rule φ is dispatched only to the workers containing its corresponding $Q[x_0, y_0]$. Workers execute in parallel, and the master aggregates all newly derived $[x.A]_=$ (resp. $[e_x.A]_=$) into Γ.

We show that the algorithm is *parallelly scalable* [79]: its runtime is $t(\mathcal{D}, G, \Sigma)/K$, where $t(\mathcal{D}, G, \Sigma)$ is the cost of the sequential algorithm, and K is the number of processors (see [1] for proof).

Conflict of rules. Although multiple rules may trigger simultaneously on the same entity or relationship, there is no conflict in the deduction, since contradiction types are mutually exclusive (Section 2) and each rule predicts at most one type. Moreover, the reasoning is proved Church-Rosser [1], which guarantees a unique fixpoint that is independent of rule application order.

Incremental detection. In response to updates, Detector employs incremental chase by restricting the rules to an affected area (AFF) determined by updates ΔG. This localized deduction is equivalent to a global computation (see Appendix B.3 in [1] for details).

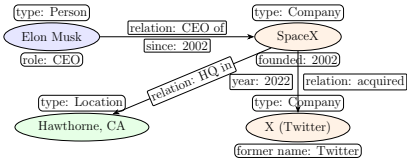


Figure 3: Example graph G_D

Document marking. To map graph-level contradictions in G back to their textual origins in $\mathcal{D}$, step 5 of DocChecker maintains, for each triple (h, r, t) in every dual star $Q[x_0, y_0]$, a mapping $f_{\text{index}}(u) : u \mapsto [s_u, e_u]$, where $u \in \{h, t\}$ and $[s_u, e_u]$ is the sentence span in $\mathcal{D}$ containing the entity mention. These spans provide the exact text context for annotating contradictions. If a statement conflicts with multiple others, all involved spans are marked via $f_{\text{index}}(u)$, enabling fine-grained multi-point conflict tracking.

5 GRAPH CONVERSION AND FUSION

This section presents steps 1-2 of DocChecker: graph conversion (Section 5.1), fusion (Section 5.2) and maintenance (Section 5.3).

5.1 Graph Conversion

Graph conversion transforms each document $D \in \mathcal{D}$ into a graph G_D via LLM-based generation, and enriches the resulting vertex and edge attributes using an external knowledge graph (KG).

LLM-based graph conversion. Following the sampling strategy in [31], DocChecker filters out data irrelevant to entities of $\mathcal{E}$. With a controllable parameter σ, it ensures that RICOs discovered from the samples retain support and confidence with bounded loss (as a function of σ). In other words, it reduces the search space and computational cost while preserving valuable RICOs.

The conversion of D into the property graph G_D follows a three-stage pipeline to ensure semantic consistency. (1) A pretrained model [141] is employed to identify explicit entity mentions and resolves pronouns to antecedent nouns, ensuring that all relations anchor to specific entities. (2) Cross-document entities are matched, where an LLM (*e.g.,* GPT-5.2 [101]) is used to understand versatile natural languages and cluster similar mentions into canonical entities in $\mathcal{E}$, which prevents redundant nodes. (3) The LLM extracts attributes and relationships from documents, and maps them to node attributes and edge predicates. This translates unstructured text into G_D that preserves logical constraints in documents.

Example 3: Given document D *"Elon Musk, the CEO of SpaceX (founded in 2002), acquired X (formerly Twitter) in 2022. SpaceX, a private company, has its headquarters in Hawthorne, CA"* and the entity set $\mathcal{E}$ = {Elon Musk, SpaceX, X (Twitter), Hawthorne, CA}, the NER model extracts the relevant contexts: (1) Elon Musk, the CEO of SpaceX, (2) SpaceX (founded in 2002), acquired X (formerly Twitter) in 2022, and (3) SpaceX has its headquarters in Hawthorne, CA. These contexts are formatted as a prompt (given in [1]) and used to instruct the LLM, which generates the property graph G_D in Figure 3, capturing the entities and their relationships. □

Graph enrichment. To recover information omitted by LLMs during graph generation [69, 102, 128] and to supplement documents D that are not self-contained in specialized domains, we enrich each generated graph G_D with knowledge from a curated KG (*e.g.,* Wikidata). Given G_D, the KG, and entity set $\mathcal{E}$, we augment G_D with domain-relevant facts about entities in $\mathcal{E}$. Enrichment consists of two steps: (1) entity linking and disambiguation, and (2) selective extraction of KG attributes and relations, combining efficient retrieval with context-based disambiguation and no manual effort.

Entity Linking. We adopt an automatic two-stage strategy to map each entity node $e_i \in \mathcal{E}$ in G_D to its unique identifier q_i in the KG. *(1) Candidate retrieval.* For each entity e_i, the Wikidata API (`wbsearchentities`) is queried automatically via standardized templates [126], which supports high throughput [96]. This returns a candidate set $\mathcal{Q}_i = \{q_1, \ldots, q_k\}$ of the top-k most relevant identifiers ($k = 10$ yields recall of over 90% [29, 64]). Each candidate is attached with its description and aliases from the KG. This step restricts the search space to a compact and accurate set, which provides high scalability to thousands of entities per document [114]. *(2) Contextual disambiguation.* Then, the best match of e_i from $\mathcal{Q}_i$ is selected as follows. Taking $C(e_i)$ and $\text{Desc}(q_j)$ as inputs, a SBERT model [107] embeds a context vector $\mathbf{v}_{C(e_i)}$ for e_i and a description vector $\mathbf{v}_{\text{Desc}(q_j)}$ for each candidate q_j in $\mathcal{Q}_i$, where $C(e_i)$ is the KG descriptions for the two-hop neighboring entities of e_i, and $\text{Desc}(q_j)$ is the description of q_j. Then the best matching identifier q^* for e_i is selected with the highest cosine similarity of vectors: $q^* = \arg\max_{q_j \in \mathcal{Q}_i} \frac{\mathbf{v}_{C(e_i)} \cdot \mathbf{v}_{\text{Desc}(q_j)}}{\|\mathbf{v}_{C(e_i)}\| \, \|\mathbf{v}_{\text{Desc}(q_j)}\|}$. Thus, the system resolves ambiguities without manual review. Note that only neighbors within two-hops are considered since they provide enough information for disambiguation with limited cost [29, 55].

Selective extraction. Given G_D with linked KG identifiers, attributes and relations are retrieved via SPARQL to enrich the graph. To ensure relevance, a domain label (*e.g.,* finance, biomedicine) is inferred for each entity using GPT-5.2 [101]. Retrieval is also restricted to a two-hop neighborhood to be efficient and context-aware, and only domain-specific predicates are retained. Selective extraction preserves essential evidence while reducing noise.

After these two stages, we obtain the enriched graph (also denoted by G_D), which encodes D with augmented KG knowledge.

Example 4: Consider the node John_Smith labeled `Person` in G_D. The enrichment first links the node to Wikidata entity Q123456 and retrieves attributes occupation : Software_Engineer, citizenship : United_States and educated_at : Stanford_Univ. These attributes are attached to the John_Smith, enabling richer semantic reasoning (*e.g.,* matching by education or nationality). □

Complexity. The graph conversion complexity is $O(NL)$, where N is the number of documents and L aggregates document, prompt, graph lengths, and enriched entities ($L_D, L_P, L_G, |\mathcal{E}|$), parallelizable to $O(NL/K)$ across K processors (refer to Appendix D.1 in [1]).

5.2 Graph Fusion

Graph fusion identifies nodes in different G_D's that refer to the same real-world entity, links them with a special ϵ-edge, and merges all G_D's into a unified graph G. Below we first revise the notion of graph join [36] and then extend it to introduce the graph fusion.

Graph join. Graph join identifies entities referring to the same real-world object [36]. Prior methods merge such entities into a single node by unifying their attributes and relations [27], which obscures the provenance of attributes and relations and makes it difficult to trace contradictory facts back to their source documents. Hence we develop a provenance-preserving hybrid alignment method.

Local graph embedding. We use an HGNN-based model (*e.g.,* R-GCN [111]) to learn embeddings $\mathbf{h}_u$ and $\mathbf{h}_v$ for nodes $u \in G_{D_1}$ and $v \in G_{D_2}$, capturing both attributes and local topology. First, SBERT encodes the raw attributes A_u into an initial vector $\mathbf{a}_u$. Then, the HGNN performs structure-aware aggregation: $\mathbf{h}_u = \sigma\Big(\sum_{r \in \mathcal{R}} \sum_{v' \in \mathcal{N}_r(u)} \frac{1}{|\mathcal{N}_r(u)|} \mathbf{W}_r \mathbf{h}_{v'} + \mathbf{W}_0 \mathbf{a}_u\Big)$, where $\mathcal{R}$ is the set of relation types, and $\mathbf{W}_r$ is the weight matrices.

Matching and ϵ-edges. Given the fused embeddings $\mathbf{h}_u$ and $\mathbf{h}_v$, we compute the similarity between nodes u and v via cosine similarity: $S(u, v) = \cos(\mathbf{h}_u, \mathbf{h}_v)$. If $S(u, v)$ exceeds a threshold τ, then u and v are connected by an ϵ-edge rather than be merged. This preserves the distinct structures and attributes of each entity node in their respective graphs G_D, which enables fine-grained analysis of their interplay during contradiction detection.

The hybrid alignment offers three benefits. (1) SBERT provides semantic similarity even when attribute texts differ literally. (2) HGNN aggregation incorporates 2-hop structural context [29], allowing alignment beyond sparse or inconsistent attributes. (3) The combination is robust to missing or heterogeneous information, linking entities that share limited context or weak structural cues.

Example 5: Consider nodes u and v from documents D_u and D_v that both refer to Elon Musk: (1) u: `{name: "Elon Musk", attribute: {"Role": "CEO"}}`, (2) v: `{name: "Elon Musk", attribute: {"Role": "CTO"}}`. Previous graph join [27] merges them into a single node: w: `{name: "Elon Musk", attribute: {"Role": "CEO", "CTO"}}`. This direct join yields conflicting attributes (*e.g.,* `Role: "CEO, CTO"`). In contrast, we connect them with an ϵ-edge: `(u, v): {label: "epsilon"}`. Our method preserves both nodes and connects them with an ϵ-edge, such that (1) `CEO` vs. `CTO` conflict remains explicit for contradiction detection; and (2) nodes u and v stay associated with their documents D_u and D_v, which is essential for document resolution. □

Graph fusion. We merge all G_D's into a unified graph G, which aligns entity references across documents to provide shared context for matching and conflict detection. Without fusion, pairwise detection on isolated G_D's may miss cross-document inconsistencies and degrade inter-document accuracy. Given $\{G_{D_i}\}_{i=1}^N$, we apply a low-complexity composition that links coreferent entities via ϵ-edges while preserving each G_{D_i}. Specifically, (a) for every pair $\langle G_m, G_n \rangle$, we identify coreferences across graph pairs and add ϵ-edges via graph join, and (b) we construct G by uniting all nodes and edges with these ϵ-edges: $G \leftarrow \Big(\bigcup_{i=1}^N G_{D_i}\Big) \cup \{\epsilon\text{-edges}\}$.

Complexity. The combined graph join and fusion process runs in $O(n \log n + |\mathcal{E}_{\text{total}}|)$, where n is the total number of nodes and $|\mathcal{E}_{\text{total}}|$ the total edges (see Appendix D.2 in [1] for details).

5.3 Incremental Maintenance of Graphs

To handle document updates, DocChecker derives an incremental algorithm $\mathcal{A}'$ from its sequential graph conversion and fusion al-

gorithm $\mathcal{A}$ using localizable computation principles [42, 44]. By restricting execution to the affected area (AFF), $\mathcal{A}'$ ensures that costs scale with the update size rather than the entire corpus.

Proposition 3: *Given $\mathcal{D}$ and an update $\Delta\mathcal{D}$, $\mathcal{A}'$ satisfies $\mathcal{D} \oplus \mathcal{A}'(\mathcal{D}, \Delta\mathcal{D}) = \mathcal{A}(\mathcal{D} \oplus \Delta\mathcal{D})$, where $\oplus$ is the application of updates.* □

This process includes incremental graph conversion, enrichment, and fusion. By localized re-computation within the AFF, the system ensures marginal overhead for dynamic updates. Appendices B.1 and E in [1] provide method details and the proof of Proposition 3.

6 FAST DISCOVERY OF RICOS

This section introduces Learner for RICO discovery (Step 3 of DocChecker). We state the discovery problem (Section 6.1), develop Learner (Section 6.2) and present its parallelization (Section 6.3).

6.1 The RICO Discovery Problem

Challenge. RICO discovery poses challenge beyond traditional rule mining [31] where quality metrics, *e.g.,* support, become invalid. As each RICO targets a specific contradiction type and typically involves only a small set of entities, most useful RICOs have low support and would be pruned by conventional high support thresholds.

Evaluation metrics. We replace support with two metrics.

Contradiction coverage: $f_c(C, \Sigma) = \frac{|\{u \in C \mid \text{detected}(u, \Sigma)\}|}{|C|}$ measures the fraction of contradictions correctly detected by rule set Σ. If $\mathcal{D}$ provides fine-grained annotations (location and type), C contains all contradiction instances $\mathcal{I}_C$, and a unit u is detected only if both its location and type are correctly identified. If $\mathcal{D}$ offers only binary labels, C consists of all contradictory segments $\mathcal{S}_C$, and u is detected if Σ correctly marks the segment as contradictory.

Accuracy improvement. We assess rule quality by measuring the improvement in binary classification accuracy (contradictory vs. normal) achieved by the set Σ of RICOs. We employ a basic classifier RoBERTa-large to obtain the binary accuracy $\text{Acc}(\mathcal{D}) = \frac{1}{|\mathcal{D}|} \sum_{s \in \mathcal{D}} \mathbb{I}(\hat{y}_s = y_s)$, where $\hat{y}_s$ and y_s are the predicted and ground-truth labels, and $\mathbb{I}(\cdot)$ is the indicator function. The improvement is quantified as $f_a(\mathcal{D}_{\text{filter}}, \mathcal{D}) = \text{Acc}(\mathcal{D}_{\text{filter}}) - \text{Acc}(\mathcal{D})$, where $\mathcal{D}_{\text{filter}} = \mathcal{D} \setminus \{s \in \mathcal{D} \mid \Sigma \text{ detects } s \text{ contradictory}\}$. Thus, f_a evaluates whether contradiction filtering effectively enhances the classifier's ability to distinguish normal texts from contradictory ones.

Problem statement. Given a document set $\mathcal{D}$ and its converted, enriched, and fused graph G, the RICO discovery problem is to find a rule set Σ from G such that Σ achieves contradiction coverage at least α and accuracy improvement at least β for bounds α and β.

6.2 Discovering RICOs by Learning

Our learning-based discovery algorithm, Learner (Algorithm 1), formulates RICO discovery as a learning-based optimization problem to improve accuracy while avoiding exponential search. By leveraging template-based prompt automation [130] and LLMs, Learner automatically generates rules that satisfy the metrics in Section 6.1.

Learner consists of the following three key modules.

Dual-star pattern extraction and pruning. This module extracts common dual-star patterns from the document graph G and employs an LLM to prune unpromising candidates. It first extracts

Algorithm 1: Algorithm Learner

Input: Graph G, corpus $\mathcal{D}$, threshold α, objectives $f_c(\cdot)$ and $f_a(\cdot)$.
Output: Rule set Σ

1 Extract generalized dual-star patterns $\mathbb{Q} = \{Q_1, \ldots, Q_n\}$;
2 Prune $\mathbb{Q}$ via LLM with context $\mathcal{D}_i$; obtain $\mathbb{Q}_s = \{Q'_1, \ldots, Q'_s\}$;
3 **for** $Q'_i \in \mathbb{Q}_s$ **do**
4 Annotate Q'_i with confidence score;
5 **if** *score* < 0.5 **then** Reflector LLM decides whether to keep Q'_i;
6 Generate predicate set $\mathcal{P} = \{P_1, \ldots, P_s\}$ for $\mathbb{Q}_s$;
7 Extract gold standard C from $\mathcal{D}$;
8 $\Sigma \leftarrow \emptyset$; $\Sigma' \leftarrow \emptyset$; iter $\leftarrow 0$; FailCount $\leftarrow 0$; $k \leftarrow 3$;
9 **while** $f_c(\Sigma, C) \leq \alpha$ ***and*** FailCount $< k$ **do**
10 $\Sigma_{\text{new}} \leftarrow \emptyset$;
11 **for** $Q'_i \in \mathbb{Q}_s$ **do**
12 Generate rules Σ_i via LLM from $\mathcal{P}$;
13 Retain $\Sigma'_i \subseteq \Sigma_i$ hitting gold standard C;
14 $\Sigma_{\text{new}} \leftarrow \Sigma_{\text{new}} \cup \Sigma'_i$;
15 $\Sigma' \leftarrow \Sigma' \cup \Sigma_{\text{new}}$;
16 Generalize predicates in Σ' (e.g., Smith.age $\rightarrow$ Person.age);
17 Use generalized Σ' as few-shot prompt to guide next iteration;
18 **if** (iter + 1) mod $k = 0$ **then**
19 Apply Σ' to $\mathcal{D}$; obtain $\mathcal{D}_{\text{clean}}$;
20 **if** $f_a(\mathcal{D}_{\text{clean}}) > f_a(\mathcal{D})$ **then**
21 $\Sigma \leftarrow \Sigma \cup \Sigma'$; FailCount $\leftarrow 0$;
22 **else**
23 FailCount $\leftarrow$ FailCount + 1;
24 $\Sigma' \leftarrow \emptyset$;
25 iter $\leftarrow$ iter + 1;
26 **return** Σ;

generalized dual-star patterns $\mathbb{Q} = \{Q_1, Q_2, \ldots, Q_n\}$ from G (line 1) via BFS starting from entity nodes connected by ϵ-edges (labeled `same_entity`) up to depth l, *e.g.,* 5 [134]. Each pattern stores only node and edge labels and structure, without attributes such as entity names or values, providing sufficient structural context for rule matching [66] while enabling generalization across entities.

Learner refines $\mathbb{Q}$ using a two-stage pruning pipeline [28] (line 2): (1) an LLM evaluates each $Q_i \in \mathbb{Q}$ against its document context $\mathcal{D}_i$ across seven contradiction types (see [1]). (2) Low-confidence cases (< 0.5) are then routed to a Reflector LLM [115] (lines 3–5) for chain-of-thought cross-validation. Only verified contradiction patterns are retained in the final set $\mathbb{Q}_s = \{Q'_1, Q'_2, \ldots, Q'_s\}$.

Predicate generation and gold standard construction. This module generalizes attributes into typed predicates and builds the gold standard for evaluation. Given $\mathbb{Q}_s$, Learner uses an LLM to generate a predicate set $\mathcal{P} = \{P_1, \ldots, P_s\}$ (line 6), formalizing attributes and relations into typed predicates (*e.g.,* comparison predicates $z.A \oplus c$ and textual classifiers $T(s_1, s_2, \tau)$) that compose rule preconditions and consequences. For evaluation, Learner derives a gold standard set C from ground-truth annotations (line 7), including binary labels, contradiction types, and evidence spans [84]. Intuitively, C is a set of constraints specifying which entity–attribute (or relation) assertions cannot co-exist, serving as the objective reference for validating rule accuracy and coverage (see [1] for details).

Three-stage discovery. Given the predicate set $\mathcal{P}$ and gold standard C, Learner initializes two empty sets, Σ' (candidate rules) and Σ

(final rules) (line 8), and then iteratively performs three steps.

Stage 1: LLM-based rule generation (lines 11–15). For each candidate pattern $Q'_i \in \mathbb{Q}_s$, the LLM generates an initial rule set Σ_i, using predicates in $\mathcal{P}$ to form preconditions and consequences. Rules contain at most n precondition predicates for a bound n and a single consequence [98, 102], balancing expressiveness and efficiency. Rules are applied to Q'_i, and only those that correctly match the gold standard C are retained as Σ'_i, which are accumulated into Σ'.

Stage 2: Meta-learning feedback (lines 16–17). To enhance generalization, Σ' is used as few-shot guidance. Before prompting, entity- and attribute-specific names are replaced with typed labels (*e.g.,* `Smith.age` $\mapsto$ `Person.age`). The LLM then derives generalized rule patterns, improving subsequent generations.

Stage 3: Verification (lines 18–25). Rules in Σ' are evaluated every k iterations, where k is configurable parameter. Following prior work [94] and our experiments, we set $k = 3$ as LLM outputs typically stabilize within 2–4 iterations, balancing generation quality and computational cost; k can be adjusted in practice. During verification, detected contradictions are mapped back to get $\mathcal{D}_{\text{filter}}$. Rules with $f_a(\mathcal{D}_{\text{filter}}) > f_a(\mathcal{D})$ are added to Σ and the failure counter resets; otherwise, they are discarded and the counter increments.

The process terminates when (a) Σ achieves contradiction coverage $f_c(\Sigma, C) \geq \alpha$, and (b) accuracy improvement $f_a(\mathcal{D}_{\text{filter}}, \mathcal{D}) \geq \beta$.

Incremental RICO *discovery.* When data is updated, Learner incrementally maintains the rule set Σ by discovering from the affected area (AFF) induced by ΔG. It updates quality metrics (f_c, f_a) via localized statistics in $O(|\Delta G|)$ and restricts LLM-driven discovery to additions, deletions and updates of entities/edges. This incremental discovery bounds the maintenance cost to the update size, which yields high performance (see Appendix B.2 in [1] for details).

Example 6: To illustrate Learner, consider a medical document $\mathcal{D}$ describing patient **John Doe**, where the admission record M_1 reports age 45 and the discharge summary M_2 reports age 65.

Phase 1. Learner identifies a dual-star centered on M_1 and M_2, linked via ϵ-edges to *John_Doe*: $Record(M_1) \xrightarrow{\epsilon} Person(P) \xleftarrow{\epsilon} Record(M_2)$. From the associated text, the LLM detects a *Numerical Contradiction* and adds the pattern to $\mathbb{Q}_s$ with confidence 0.95.

Phase 2. From this contradiction, Learner generates typed predicates (P_{age}, P_{status}, and comparison $z.A \oplus c$) and instantiates textual classifiers for "45-year-old" vs. "age is listed as 65". These are aligned with the corpus annotations to form the gold standard C.

Phase 3. (1) *Generation:* The LLM constructs a rule such as M_1.age $\neq M_2$.age $\rightarrow \tau_{\text{num}}$, which is added to Σ' after matching C. (2) *Meta-learning:* Specific values are generalized (*e.g.,* $c_1 \neq c_2$ to $x.A \neq y.B$, for unequal attributes across records of the same entity). (3) *Verification:* After periodic evaluation, filtering contradictions improves accuracy on $\mathcal{D}_{\text{filter}}$, and the rule is accepted into Σ. □

6.3 Learning RICOs in Parallel

The parallel version PLearner of Learner accelerates the computationally intensive components of the discovery loop, while preserving sequentially dependent stages such as meta-learning feedback. As detailed in [1], PLearner introduces two modules: parallel rule generation and filtering, and parallel predicate generalization.

Parallel rule generation and filtering. This is the primary source of speedup. PLearner distributes the candidate dual-star patterns $\mathbb{Q}_s$ across K workers (lines 5–8). Each worker processes its assigned Q'_i independently, generating rules via the LLM and filtering them using the gold standard. The retained rules Σ'_i are aggregated atomically into Σ_{new}, eliminating the latency of sequential LLM calls.

Parallel predicate generalization. This step (*e.g.,* replacing entity names with type labels) is done independently for each rule in Σ', needs no cross-worker coordination, and is parallelized (line 10).

We have the following (see Appendix H in [1] for a proof).

Proposition 4: PLearner *scales parallelly relative to* Learner. □

Remark. Learner focuses on contradiction detection (conflicting attributes or relations for the same entity); the identified conflicts can be repaired by existing RICO-based, knowledge-based, or LLM-based methods. Thus, Learner complements existing data repair frameworks and enables full KG cleaning when coupled with a repair module. Note that LLMs are confined to the offline rule discovery and graph conversion phase, where pruning reduces LLM calls. In contrast, RICO execution is online and employs only lightweight ML models (*e.g.,* GCN and SBERT; Section 3.2), and thus enables efficient inference without invoking LLMs.

7 EXPERIMENTAL STUDY

Using real-life documents, we empirically verify DocChecker's accuracy, scalability and its performance gains on LLM tasks.

Experimental setting. We start with the setting.

Datasets. We used 15 datasets (sizes in Table 2) for seven tasks: (1) detection of all seven contradiction types; specifically, ContraDoc [84], ECON [70], Knowledge Conflict [129], StorySumm [118], DOCNLI [138], ContractNLI [75], ConTRoL [88] and CaseHOLD [145] support inter-document tasks, and the others focus on intra-document conflicts; (2) sensitivity analysis on ContraDoc; (3) document-level self-contradiction detection (DSCD) for binary decision, ranking and retrieval on ContraDoc and ECON; (4) natural language inference (NLI) at sentence and document levels on CaseHOLD and ContractNLI; (5) domain-specific analysis of legal and economic conflicts on ECON and CaseHOLD; (6) scalability analysis of Learner, Detector and DocChecker on six representative datasets; and (7) ablation study on StorySumm.

These datasets span diverse domains, including legal contracts (ContractNLI, CaseHOLD), economic reports (ECON), news (ContraDoc, DOCNLI), and narratives (StorySumm), covering a wide range of real-world genres and writing styles.

Baselines. We compared DocChecker with 15 recent baselines. (1) *Document-level contradiction tasks.* (a) Sensitivity analysis tasks: GPT-4o [3]. (b) DSCD tasks: GPT-5.2-chat-latest [101], Claude-3-7-sonnet-latest [8], Gemini-3-pro-preview-thinking [53], Llama-3-70B [122], Qwen-Max-Latest [11], DeepSeek-V3 [87], Longformer-base-4096 [13] and RoBERTa-large-MNLI [90]. (2) *NLI tasks.* (a) Document-level: Longformer-base-4096, BART-base [82], DeBERTa-v3-base [60], DistilBERT-base [109] and RoBERTa-large-MNLI. (b) Sentence-level: BERT-base-uncased [26], RoBERTa-base [90], XLNet-base-cased [137] and RoBERTa-large-MNLI.

Prompts. To assess prompts on LLM baselines, we used six prompts

Dataset	Data size	Total	Factual	Content	Causal	Negation	Relational	Numeric	Temporal	Vertex	Edge	Acc	P	R	F_1	EHR	Avg. LLM cost
binary-FEVER-NLI	222 docs	105	58	3	1	0	30	4	9	65	40	0.789	0.895	0.869	0.882	0.853	7.2k tokens
CaseHOLD	530 docs	286	78	60	135	1	1	4	7	142	144	0.868	0.879	0.986	0.929	0.864	5.1k tokens
ConTRoL	417 docs	229	57	19	119	1	3	18	12	94	135	0.793	0.895	0.875	0.885	0.854	16.6k tokens
ContraDOC	891 docs	517	146	41	218	5	1	53	53	240	277	0.841	0.885	0.944	0.914	0.860	16.4k tokens
ContractNLI	104 docs	65	3	38	21	1	1	0	1	41	24	0.878	0.878	1.000	0.935	0.865	55.2k tokens
DOCNLI	471 docs	223	38	28	111	0	2	28	16	94	129	0.850	0.883	0.958	0.919	0.861	18.1k tokens
ECON	120 docs	363	239	13	17	9	0	71	14	323	40	0.801	0.893	0.886	0.890	0.855	21.3k tokens
Knowledge Conflict	200 docs	90	63	13	1	0	0	8	5	84	6	0.670	0.918	0.713	0.803	0.838	5.2k tokens
binary-MCTest-NLI	400 docs	217	48	36	114	3	1	5	10	89	128	0.872	0.879	0.991	0.932	0.864	14.7k tokens
MNLI	825 docs	1,309	456	168	419	101	7	85	73	709	600	0.770	0.899	0.844	0.870	0.833	18.2k tokens
Negation	899 docs	397	316	0	15	31	2	22	11	338	59	0.790	0.895	0.870	0.883	0.853	6.4k tokens
Real-life	131 docs	131	68	4	20	2	0	22	15	94	37	0.804	0.892	0.891	0.892	0.855	6.1k tokens
RTE-3	744 docs	549	245	73	120	1	9	69	32	387	162	0.836	0.886	0.937	0.911	0.817	22.5k tokens
SNLI	571 docs	913	391	269	90	12	64	29	58	689	224	0.870	0.879	0.989	0.931	0.813	17.2k tokens
StorySumm	580 docs	81	42	7	25	4	0	2	5	47	34	0.852	0.883	0.961	0.920	0.862	8.1k tokens

Table 2: Accuracy of contradiction detection and evidence retrieval across all datasets

with different strategies and tested their sensitivity (Exp-1), DSCD tasks (Exp-1), and a case study (Exp-4) (see Appendices G.3-G.5 [1]).

Ground truth. All datasets provide expert-annotated ground truth labels, including correct vs. distractor answers in QA datasets (*e.g.,* CaseHOLD, ConTRoL), three-way labels in NLI datasets (*e.g.,* SNLI, MNLI, ContractNLI), and contradiction-type labels in others (*e.g.,* binary-FEVER-NLI). See Appendices I-J in [1] for details.

Metrics. We evaluated contradiction detection, classification and evidence retrieval using: (1) *Contradiction metrics* [14, 15]: (a) Accuracy (Acc = $\frac{TP+TN}{TP+TN+FP+FN}$) to quantify the proportion of correct predictions, as a standardized metric for benchmarking our approach against NLP/ML baselines [13, 48, 60, 82, 90], where TP, TN, FP and FN denote true positives, true negatives, false positives, and false negatives, respectively; (b) Precision ($P = \frac{TP}{TP+FP}$), the ratio of true positives to all predicted positives; Recall ($R = \frac{TP}{TP+FN}$), the ratio of true positives to all actual positives; and F_1 ($= \frac{2PR}{P+R}$), the harmonic mean of precision and recall. These provide a standard and comparable basis for benchmarking against NLP/ML baselines. (2) *Evidence Hit Rate (EHR)* [121]: the probability $P(V(E) = 1 \mid J(D) = 1)$, where $J(D)$ indicates correct contradiction prediction and $V(E)$ correct evidence localization, measuring whether the model can localize contradictions once detected. (3) *Match Rate (MR)* [105]: the fraction of gold evidence tokens covered by retrieved snippets, quantifying semantic/token-level overlap with ground truth.

Data partitioning and model training. For pre-partitioned datasets, we followed the original splits. For others, we randomly divided data into training/validation/test sets with a 7:1:2 ratio. Baselines were trained, rule patterns learned, and LLM few-shot prompting performed on the training set. Hyperparameters were tuned on the validation set, and final results are reported on the test set.

Environment. DocChecker is implemented in Python. All experiments were conducted in PyTorch on a single server with four NVIDIA GPUs (32GB each) and an AMD EPYC 9754 128-core CPU. Each GPU node was allocated 24 vCPUs and 72 GB RAM, totaling 96 vCPUs and 288 GB memory. We adopted data-parallel training across all GPUs and report the average of three independent runs.

Experimental findings. We next report our findings.

Exp-1: Accuracy. We evaluate DocChecker's accuracy and robustness against SOTA LLMs and specialized models across standard benchmarks: (1) contradiction detection with location, type and count annotations (ContraDoc [84]); (2) sensitivity to contradiction position, type and domain (ContraDoc [84]); (3) DSCD [84, 117]; (4) document-level [70, 75, 138] and sentence-level NLI [15, 131].

(1) Contradiction detection. As shown in Table 2, we evaluate DocChecker on 15 datasets. To analyze structural contradiction patterns, we map the seven contradiction types onto our graph representation as: *Vertex-level (entity) contradictions*: conflicts within a single node, where attributes or facts about the same entity disagree. *Edge-level (relation) contradictions*: conflicts expressed along an edge, where the logical, causal, or directional relation between two entities is inconsistent. Mapping all contradiction types onto vertices or edges enables quantifying their structural distribution.

(a) Contradiction type distribution. Table 2 shows substantial variation in the distribution of the seven contradiction types across datasets. (i) Factual and Causal inconsistencies dominate many datasets (*e.g.,* MNLI: 456 factual, 419 causal; ContraDoc: 218 causal), and DocChecker remains stable on these categories, achieving high F_1 on SNLI (93.1%) and ContraDoc (91.4%). (ii) Relational, Numeric, and Temporal contradictions are sparser but more challenging. For instance, Relational cases appear frequently in SNLI (64) but are nearly absent in ECON, which instead contains many numeric conflicts (71). Despite such diversity, DocChecker attains 80.1% accuracy on ECON, evidencing strong robustness.

(b) Performance and generalization. (i) DocChecker generalizes well across domains, achieving high F_1 on sentence-level NLI (SNLI: 93.1%, RTE-3: 91.1%). (ii) It remains robust on document-level datasets (ContraDoc: 91.4%, DOCNLI: 91.9%), showing effective long-range modeling. (iii) Evidence Hit Rate (EHR) is consistently high (average 85.7%), including CaseHOLD (86.4%) and StorySumm (86.2%), confirming accurate graph-based evidence retrieval.

(c) Sensitivity to graph structure. DocChecker's accuracy is influenced by how contradictions manifest structurally. Performance lags on vertex-level conflict datasets (*e.g.,* Knowledge Conflict: 80.3%) than those with more edge-level ones (*e.g.,* ContraDoc: 91.4%). This shows that contradiction localization is highly dataset-dependent and underscores the need for explicit graph structure modeling to capture both attribute-level and relational conflicts.

(d) LLM cost. We measure the LLM token consumption of DocChecker in its LLM-driven stages, which are (i) graph conversion, where LLMs structure raw documents as graphs, and (ii) rule discovery, where LLMs are employed in the three-stage rule discovery process. As shown in Table 2, the average consumption is 15.9k tokens per document, ranging from 5.1k (CaseHOLD) to 55.2k (ContractNLI), which shows DocChecker's cost-effectiveness in targeted LLM application during rule inference and learning.

(2) Sensitivity analysis. We examine how contradiction position, type, and domain affect DocChecker and GPT-4o on ContraDoc.

(a) Contradiction positions. Table 3 tests performance across three contextual spans: local (intra-paragraph), intra-document (inter-

Dimension	Category	DocChecker		GPT-4o	
		Acc (%)	MR/F_1 (%)	Acc (%)	MR/F_1 (%)
Position (task 1)	Local	80.82	83.56	68.09	21.28
	Intra-document	76.81	89.04	51.89	17.92
	Inter-document	86.96	85.51	65.06	20.48
Type (task 2)	Factual	80.30	89.08	75.50	50.99
	Content	80.22	89.03	84.93	30.53
	Causal	83.74	91.15	67.88	18.49
	Negation	68.00	80.95	93.71	5.00
	Relational	85.00	91.89	91.89	3.92
	Numeric	75.25	85.87	94.04	52.63
	Temporal	72.86	84.30	94.54	52.17
Domain (task 3)	News	84.40	90.83	62.39	20.18
	Stories	84.95	95.70	72.04	30.11
	Encyclopedia	74.47	80.85	56.38	22.34

Table 3: Sensitivity analysis of DocChecker **vs. LLMs**

paragraph), and inter-document. (i) DocChecker performs strongly on local contradictions (80.82% Acc, 83.56% MR), beating GPT-4o (68.09% Acc). (ii) Accuracy drops to 76.81% for intra-document cases, reflecting the difficulty of detecting subtle cross-paragraph inconsistencies within shared context. (iii) Accuracy rises to 86.96% for inter-document contradictions, showing that graph alignment effectively captures structural divergences across sources. Across all spans, DocChecker consistently outperforms GPT-4o, whose accuracy degrades sharply on long-range contexts (down to 51.89%).

(b) Contradiction types. Table 3 examines detection difficulty across the seven contradiction types. (i) DocChecker performs best on structurally explicit categories, achieving 85.00% (relational) and 83.74% (causal), where RICO rules effectively capture clear logical dependencies. (ii) Performance is lower on semantically subtle or sequential types such as negation (68.00%) and temporal (72.86%), due to reliance on fine-grained semantic scope [99] and temporal reasoning [103], which are harder to generalize from structural patterns alone. (iii) Although GPT-4o shows high accuracy on negation and temporal (> 93%), its extremely low F_1 scores (5.00%, 52.17%) indicate severe false positives, whereas DocChecker remains balanced and reliable ($F_1 \geq 80.95\%$ for all types).

(c) Domain-specific analysis. Table 3 shows that DocChecker performs most stably on Stories (84.95% Acc.) and News (84.40%), with a high MR in Stories (95.70%), indicating reliable conflict localization. Performance is relatively lower on Encyclopedia text (74.47%) due to dense factual cross-referencing. This said, DocChecker remains consistently more robust than LLM baselines across all domains.

(3) DSCD tasks. We evaluate DocChecker on three DSCD subtasks.

(a) Binary classification. This task evaluates whether a method is able to distinguish contradictory (positive) from non-contradictory (negative) texts. As shown in Task 1 of Table 4, DocChecker achieves near-perfect accuracy, reaching $P = 1.000$ and $F_1 = 0.992$ on ContraDoc, far surpassing GPT-5.2 ($F_1 = 0.727$) and Longformer ($F_1 = 0.520$). On ECON, although the performance gap narrows (*e.g.,* $F_1 = 0.859$ by Claude-3-7), DocChecker remains superior with $F_1 = 0.965$. These results indicate that DocChecker's graph-based modeling captures long-range dependencies more effectively than Transformer architectures and remains robust across domains.

(b) Top-3 contradiction evidence. Evaluated on CONTRADOC-POS, this task measures a model's ability to localize salient evidence sentences. As shown in Task 2 of Table 4, all models achieve $P = 1.000$ due to the positive-only setting, so Recall and Accuracy indicate evidence retrieval success. DocChecker attains the best results with $F_1 = 0.962$ and Acc = 0.927, showing that graph fusion effectively highlights contradictory nodes. In contrast, GPT-5.2 and Claude-3-7

Task 1: Binary Classification (CONTRADOC / ECON)

Model	Prec.	Rec.	F1	Acc.	Time (s)	Tokens (k)
DocChecker	1.000 / 0.960	0.985 / 0.970	0.992 / 0.965	0.986 / 0.953	1.92/2.83	16.3/21.2
GPT-5.2	0.983 / 0.941	0.577 / 0.480	0.727 / 0.636	0.608 / 0.633	2.92/6.22	15.9/19.3
Claude-3-7	0.977 / 0.941	0.433 / 0.790	0.600 / 0.859	0.477 / 0.827	2.04/3.11	16.7/23.4
Llama-3-70B	0.971 / 0.971	0.340 / 0.330	0.504 / 0.493	0.393 / 0.547	5.68/3.77	16.4/20.1
Gemini-3-Pro	1.000 / 0.930	0.400 / 0.400	0.571 / 0.559	0.640 / 0.580	4.31/3.53	16.3/22.0
Longformer	0.650 / 0.353	0.433 / 0.060	0.520 / 0.103	0.520 / 0.300	0.29/0.27	–/–

Task 2: Evidence Detection (CONTRADOC-POS)

Model	Prec.	Rec.	F1	Acc.	Time (s)	Tokens
DocChecker	1.000	0.927	0.962	0.927	0.13	0.0
GPT-5.2	1.000	0.747	0.856	0.747	0.40	1,520.0
Claude-3-7	1.000	0.562	0.720	0.562	6.91	1,466.4
Llama-3-70B	1.000	0.412	0.584	0.412	28.09	1,215.4
Gemini-3-Pro	1.000	0.242	0.390	0.242	3.63	1,283.9
RoBERTa-mnli	1.000	0.322	0.413	0.322	0.09	–

Task 3: Joint Evidence Identification (CONTRADOC)

Model	$J(D)$	$J \wedge V$	EHR	Prec.	Rec.	Time (s)	Tokens
DocChecker	0.992	0.914	0.942	0.900	0.928	0.55	0.0
GPT-5.2	0.719	0.543	0.957	0.537	0.550	0.24	354.1
Claude-3-7	0.682	0.420	0.586	0.415	0.425	0.90	166.8
Qwen-Max	0.561	0.361	0.813	0.406	0.325	2.27	142.8
Gemini-3-Pro	0.625	0.400	0.950	0.346	0.475	11.9	2,379.0
DeepSeek-V3	0.644	0.356	0.684	0.394	0.325	0.62	304.7

Table 4: Performance evaluation of three DSCD subtasks

yield $F_1 = 0.856$ and $F_1 = 0.720$, frequently missing key evidence, while Gemini-3-Pro drops sharply ($F_1 = 0.390$); thus strong binary judgment does not necessarily imply reliable evidence localization.

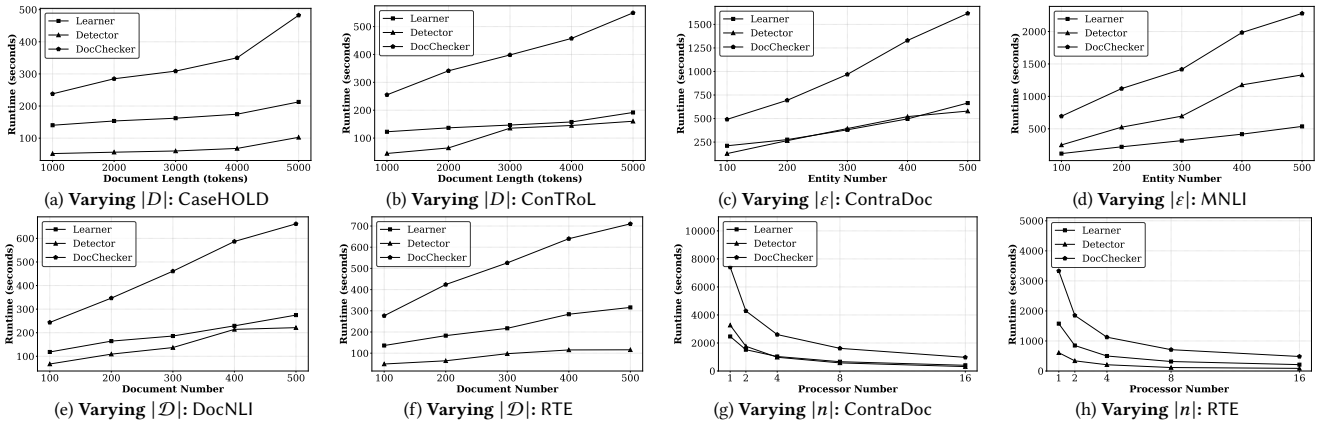
(c) Evidence identification after binary judgment. This task tests whether a method can correctly locate contradictory evidence given a positive prediction. In Task 3 of Table 4, DocChecker achieves $J(D) \wedge V(E) = 0.914$, far surpassing GPT-5.2 (0.543) and Claude-3-7 (0.420). While LLMs maintain moderate $J(D)$ (0.56–0.72), they rarely provide verifiable evidence. In contrast, DocChecker attains high EHR (0.942) with balanced Precision/Recall (0.900/0.928), showing accurate localization. This verifies that explicit graph reasoning enables reliable DSCD with verifiable evidence.

(d) Efficiency and cost. As shown in Table 4, DocChecker processes each document in 2.02s with 16.8k tokens for preprocessing and rule discovery, and incurs zero LLM cost at inference. For Task 1 (16.3k tokens), it runs in 1.92s, much faster than Llama-3-70B (5.68s); for Tasks 2 and 3, it incurs minimal overhead (0.13s and 0 tokens), while Gemini-3-Pro takes up to 3.63s and 2.4K additional tokens.

In DocChecker, LLM token costs (Table 4) stem entirely from Task 1's offline graph conversion and rule discovery. By reusing these artifacts, Tasks 2 and 3 operate as subsequent reasoning steps that incur zero additional LLM overhead. Specifically, Task 2 extracts evidence via chasing and document markings, while Task 3 aligns it with binary judgments through a subgraph consistency check. Unlike LLM baselines, DocChecker eliminates prompting costs in Tasks 2 and 3 by reusing graphs and rules, enabling highly efficient, high-throughput processing of large corpora.

(4) Document-level NLI. This task assesses ability to reason over long documents by classifying document pairs into *entailment*, *neutrality*, or *contradiction* (Table 5). DocChecker performs strongly in complex legal contexts, achieving 86.8% Acc and 0.929 F_1 on CaseHOLD, and 87.8% Acc with 0.935 F_1 on ContractNLI. Notably, DocChecker attains perfect Recall (1.000) on ContractNLI and surpasses Longformer by nearly 22% in F_1. Its consistent advantage over specialized models (*e.g.,* DeBERTa-v3-base, BART-base) highlights the effectiveness of RICO-based reasoning: by explicitly modeling document structure and integrating dispersed evidence through a unified graph, DocChecker captures long-range logical dependencies that standard neural attention often misses.

The results of sentence-level NLI tasks are consistent (see [1]).



(a) **Varying** $|D|$: CaseHOLD (b) **Varying** $|D|$: ConTRoL (c) **Varying** $|\varepsilon|$: ContraDoc (d) **Varying** $|\varepsilon|$: MNLI

(e) **Varying** $|\mathcal{D}|$: DocNLI (f) **Varying** $|\mathcal{D}|$: RTE (g) **Varying** $|n|$: ContraDoc (h) **Varying** $|n|$: RTE

Figure 4: Scalability trends across key variables

Exp-2: Scalability. We assess the scalability of Learner, Detector, and DocChecker by varying: (1) document length $|D|$, (2) entity count $|\mathcal{E}|$, (3) dataset size $|\mathcal{D}|$, and (4) processor number $|n|$.

(1) Varying document length $|D|$. We evaluated scalability when $|D|$ grows from 1k to 5k tokens on CaseHOLD and ConTRoL. Figures 4(a)–(b) show: (a) Learner takes 140.19–212.79s on CaseHOLD and 122.74–191.87s on ConTRoL, driven by the number and complexity of extracted dual-star patterns, which increase with document length. It extracts 487–577 rules (avg. 528) on CaseHOLD and 862–943 (avg. 896) on ConTRoL. (b) Detector remains stable, ranging from 51.98–102.57s on CaseHOLD and 44.87–160.44s on ConTRoL, as reasoning operates on dual-star graphs with direct span mapping via f_{index}. (c) End-to-end DocChecker takes 237.94–482.53s on CaseHOLD and 255.19–549.18s on ConTRoL, covering graph construction, fusion, rule discovery, detection and marking. Even at 5k tokens, graph construction and join account for only 21.04% (101.54s) on CaseHOLD and 16.76% (92.07s) on ConTRoL, indicating that graph abstraction decouples reasoning cost.

(2) Varying entity number $|\mathcal{E}|$. Figures 4(c)–(d) report runtime on ContraDoc and MNLI as graph size (driven by $|\mathcal{E}|$) grows from 100 to 500 nodes. (a) Learner increases from 210.74–664.56s on ContraDoc and 117.72–536.85s on MNLI, driven by denser dual-star patterns; LLM pruning remains stable. (b) Detector ranges from 127.86–579.24s on ContraDoc and 250.94–1,331.25s on MNLI. Despite growth, dual-star topology localizes matching, keeping per-contradiction latency within 278.48ms and 594.31ms, respectively. (c) End-to-end DocChecker increases from 491.11–1,616.97s on ContraDoc and 693.93–2,279.40s on MNLI. As 78.70% of real-world dual-star patterns involve < 200 entities, average runtime remains about 692s even at $|\mathcal{E}| = 500$, *i.e.*, no exponential blowup.

(3) Varying dataset $|\mathcal{D}|$. Figures 4(e)–(f) evaluate scalability as the corpus grows from 100 to 500 documents on the DocNLI and RTE datasets. (a) Learner increases moderately from 118.34s to 274.69s on DocNLI and 136.21s to 315.94s on RTE, as iterative pruning limits redundant patterns and LLM calls. (b) Detector maintains stable per-document latency (0.443s on DocNLI and 0.232s on RTE), with total runtime bounded to 221.46s (DocNLI) and 116.01s (RTE) on 16 processors, showing reasoning is localized to dual-star subgraphs rather than corpus size. (c) DocChecker rises from 243.59s to 661.61s on DocNLI and 276.63s to 710.23s on RTE, confirming that offline discovery scales with input volume while online reasoning remains effectively size-invariant for large-scale monitoring.

(4) Varying processor number $|n|$. Figures 4(g)–(h) test scalability as n ranges from 1 to 16 on ContraDoc and RTE. (a) Learner scales near-linearly, reducing runtime from 2,465s to 408s on ContraDoc and 1,577s to 212s on RTE, as parallel workers mask LLM latency. (b) Detector maintains stable per-document time (0.4s), while corpus-level runtime drops from 3,270s to 315s on ContraDoc and 609s to 88s on RTE. (c) End-to-end DocChecker drops from 7,410s to 974s on ContraDoc and 3,330s to 484s on RTE, verifying parallel scalability.

Exp-3: Ablation. Table 6 reports the impact of key DocChecker components to effectiveness (Accuracy) and efficiency (Runtime) via five ablations on StorySumm: (a) *Graph join*: Full DocChecker (with cross-document linking) vs. a no-aggregation variant. (b) *Graph enrichment*: Full DocChecker (with external KG enrichment) vs. no-enrichment variant. (c) *Entity resolution (ER)*: Full DocChecker (SBERT + HGCN) vs. no-ER variant (simulating noisy linking). (d) *Meta-learning*: Full DocChecker vs. variants without LLM-based rule generation (using traditional level-wise discovery instead, under a support threshold of 3 and a 5-hour timeout where Learner returns all rules), meta-learning feedback, or verification/filtering.

(1) Graph components. Removing graph join causes a substantial drop, reducing accuracy from 85.2% to 63.0%, which confirms cross-document linking as one of the most critical factors for accurate contradiction detection. Disabling Graph Enrichment also degrades accuracy (73.8%), showing that external knowledge provides useful context, although its effect is secondary to structural linking.

(2) Impact of ER quality. Inaccurate ER results in false positives as naming ambiguities trigger spurious contradictions. Specifically, disabling SBERT and HGCN in graph fusion increases the FPR of entity-matching by 21.2% and decreases detection accuracy by 16.7% (Table 6). This confirms that accurate entity resolution is vital for ensuring the reliability of downstream contradiction detection.

(3) Impact of meta learning components. Table 6 quantifies the impact of three meta-learning components on rule quality and efficiency. (1) Stage 1 (LLM generation): Replacing LLM-driven initialization with a levelwise strategy drops accuracy by 38.0%, proving the LLM's role in capturing complex semantic preconditions. (2) Stage 2 (meta-feedback): Disabling iterative feedback decreases accuracy and F1 by 16.6% and 19.9%, respectively. Thus, few-shot guidance is essential for pattern refinement. (3) Stage 3 (verification/-filtering): Removing this stage may introduce useless rules that fail

Model	CaseHOLD				ContractNLI			
	Acc	P	R	F1	Acc	P	R	F1
DocChecker	0.868	0.879	0.986	0.929	0.878	0.878	1.000	0.935
Longformer-base-4096	0.310	0.347	0.310	0.321	0.712	0.734	0.712	0.716
DeBERTa-v3-base	0.676	0.677	0.676	0.676	0.704	0.733	0.704	0.702
BART-base	0.672	0.680	0.672	0.673	0.508	0.451	0.508	0.477
DistilBERT-base	0.605	0.608	0.605	0.606	0.684	0.699	0.684	0.680

Table 5: Performance on legal QA and document-level NLI

to detect contractions, reducing accuracy to 54.7%. This proves the efficacy of failure counter in pruning invalid candidates via $f_a(\mathcal{D})$.

(4) Efficiency and scalability. Despite added structural modeling, the full model remains efficient (551.3s), beating ablated variants by a large margin. This shows that DocChecker achieves a strong accuracy–efficiency balance and scales well to dense text analysis.

Exp-4: Case study and contradiction analysis. LLMs struggle with long-range dependencies and implicit logic. We present three ContraDoc cases where DocChecker outperforms GPT-5.2.

(1) Content contradiction (ID: 3488771835_6): In the book *Mrs. Frisby*, a character "agrees wholeheartedly" but later "leaves the colony". LLMs miss the mismatch between intent and action. In contrast, DocChecker applies φ_{persp}: $Q[x_0, y_0](S(x_1, s) \wedge S(y_1, s') \wedge \mathcal{M}(x_0.\text{name}, y_0.\text{name}) \wedge \mathcal{M}(x_1.\text{goal}, y_1.\text{action}) \wedge T(s, s', \text{opinion}) \rightarrow \text{content}(x_1.\text{goal}, y_1.\text{action}))$, binding goal ($x_1$) and action ($y_1$) to the same entity to detect the conflict.

(2) Temporal contradiction (ID: 3488771840_9): In *Mrs. Tittlemouse*, cleaning duration shifts from a "fortnight" to "a day". LLMs miss such long-range temporal inconsistencies. DocChecker employs φ_{temp} : $Q[x_0, y_0](\mathcal{M}(x_0.\text{event}, y_0.\text{event}) \wedge x_1.\text{val} \neq y_1.\text{val} \wedge x_1.\text{unit} = y_1.\text{unit} \rightarrow \text{temporal}(x_1.\text{val}, y_1.\text{val}))$, aligning events and comparing values; $x_1.\text{val} \neq y_1.\text{val}$ catches conflicts across >15 sentences.

(3) Factual contradiction (ID: 3488771846_3): In *A Tale of Two Cities*, Darnay is "imprisoned in Paris" yet "dining in London" on the same day. LLMs miss this due to weak spatial consistency. DocChecker uses φ_{space} : $Q[x_0, y_0](\mathcal{M}(x_1.\text{time}, y_1.\text{time}) \wedge x_2.\text{loc} \neq y_2.\text{loc} \wedge T(s, s', \text{factual}) \rightarrow \text{factual}(x_2.\text{loc}, y_2.\text{loc}))$, aligning the entity on a shared timeline to catch impossible co-occurrence.

As limitations, we note that contradictions requiring purely subjective or ambiguous interpretation remain challenging.

Contradiction analysis. To mitigate attention dilution in LLMs over long sequences [89], DocChecker aligns scattered text spans into a unified graph, enabling variable binding for logical rule evaluation. This allows DocChecker to accurately detect intra- and inter-document contradictions on ContraDoc, where LLMs underperform by 15.2% due to long-range dependency failures. While such long-span conflicts account for 28.4% of ContraDoc cases, DocChecker remains robust, highlighting the need for explicit graph-based reasoning in complex inter-document settings.

Exp-5: Incremental study. DocChecker incrementally processes updates of $\Delta N = 100$ documents across the CaseHold and MNLI datasets (Table 7). (a) Graph conversion: By restricting extraction to modified fragments, conversion time on CaseHold drops from 72.3s to 16.2s (a 77.6% latency reduction), and tokens per document fall from 766.8 to 166.7 (a 78.3% token reduction). (b) Rule generation: Isolating affected subgraphs reduces MNLI LLM costs from 8,809.5 to 402.8 tokens per document, with processing time dropping from 1,135.3s to 50.1s. (c) Meta-learning: The feedback loop refines Σ in just 20.6s on MNLI, requiring only 4.3% of the initial discovery time.

Model Configuration	Accuracy (%)	F1-score (%)	Runtime (s)
DocChecker (Full Model)	85.2	92.0	551.3
Ablations			
w/o Graph Join	63.0	77.3	481.1
w/o Graph Enrichment	73.8	85.0	445.7
w/o entity resolution & linking	68.5	71.6	377.6
w/o LLM-based rule generation	47.2	52.4	over 5 hours
w/o Meta-learning feedback	68.6	72.1	390.3
w/o Rule verification	54.7	59.8	414.6

Table 6: Ablation study of key DocChecker components.

Dataset	Components	Full Re-computation		Incremental Update	
		Time	Avg. LLM Cost	Time	Avg. LLM Cost
CaseHold	Graph Conversion	72.3s	766.8 tokens	16.2s	166.7 tokens
	Rule Generation	312.3s	3,118.3 tokens	25.2s	231.0 tokens
	Meta-learning Feedback	195.4s	1,249.8 tokens	14.4s	96.1 tokens
	Total Overhead	580.0s	5,134.9 tokens	55.8s	493.8 tokens
MNLI	Graph Conversion	216.3s	4,538.7 tokens	52.5s	1,055.6 tokens
	Rule Generation	1,135.3s	8,809.5 tokens	50.1s	402.8 tokens
	Meta-learning Feedback	480.0s	4,909.5 tokens	20.6s	217.7 tokens
	Total Overhead	1,831.6s	18,257.7 tokens	123.2s	1,676.1 tokens

Table 7: Efficiency and Cost of Incremental Maintenance

Summary. We find the following. (1) *Effectiveness.* DocChecker achieves SOTA accuracy on 15 datasets, *e.g.,* 93.1% F_1 on SNLI and 91.4% on ContraDoc. It excels at document-level NLI (87.8% *Acc* on ContractNLI), outperforming DeBERTa-v3. RICOs (φ_{persp}, φ_{temp}, φ_{space}) resolve complex content, temporal and factual contradictions that LLMs fail to capture. (2) *Robustness.* DocChecker achieves 76.81% intra- and 86.96% inter-document accuracy, vs. GPT-4o's 51.89% and 65.06% on ContraDoc. It attains near-perfect DSCD (0.992 binary, 0.962 evidence F_1). Graph fusion handles inter-document conflicts, where LLMs lag by 15.2%. (3) *Efficiency and scalability.* DocChecker scales linearly with document length (1k–5k tokens) and graph size (100–500 nodes). Parallelization reduces ContraDoc runtime from 7,410.11s to 974.44s (16 processors). In DSCD, DocChecker incurs minimal overhead (0.13s, 0.0 tokens), avoiding LLM costs in Tasks 2 and 3. Incremental maintenance cuts latency by 77.6%, updating rules in 4.3% of initial time. (4) *Ablation findings.* Graph Join boosts accuracy by 22.2%, while graph enrichment improves F1-score by 7.0%. Disabling ER raises FPR by 21.2% and lowers accuracy by 16.7%. Replacing LLM-based rule generation with a levelwise strategy reduces accuracy by 38.0%. In addition, removing meta-feedback lowers accuracy and F1-score by 16.6% and 19.9%, respectively. Eliminating verification/filtering further drops accuracy to 54.7%. These results highlight the importance of graph, ER and meta-learning components. (5) *Impact of prompts (Appendices G.3–G.5 [1]).* Even with SPO-link, CoVe, or counterfactual auditing, LLMs trail DocChecker by 22.19%–44.67%.

8 CONCLUSION

We propose graph-based RICOs, implemented as DocChecker framework, to detect contradictions across documents. By integrating graph analytics with learning-based rule discovery (Learner) and parallel detection (Detector), DocChecker effectively captures intra- and inter-document contradictions. Our experiments on real-world data verify that DocChecker are highly accurate and scalable.

Future work includes (a) extending DocChecker to *cross-modal contradiction detection* (*e.g.,* text-vision/tabular) via tractable multi-modal RICOs, and (b) enhancing *robustness to LLM noise and hallucinations* through cross-model verification and consistency checks.

REFERENCES

[1] 2026. Code, datasets and full version. https://anonymous.4open.science/r/Doc4LLM-CE2E/.

[2] Serge Abiteboul, Richard Hull, and Victor Vianu. 1995. *Foundations of Databases*. Addison-Wesley.

[3] Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altenschmidt, Sam Altman, and Shyamal Anadkat. 2023. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774* (2023). https://arxiv.org/abs/2303.08774

[4] João Paulo Aires and Felipe Meneguzzi. 2017. Norm Conflict Identification Using Deep Learning. In *AAMAS Workshops*. 194–207.

[5] Abdulaziz Alamri. 2016. *The Detection of Contradictory Claims in Biomedical Abstracts*. Ph. D. Dissertation. University of Sheffield.

[6] Abdulaziz Alamri and Mark Stevenson. 2016. A Corpus of Potentially Contradictory Research Claims from Cardiovascular Research. *Journal of Biomedical Semantics* 7 (2016), 36.

[7] Alan Ansell, Felipe Bravo-Marquez, and Bernhard Pfahringer. 2021. PolyLM: Learning about Polysemy through Language Modeling. In *Conference of the European Chapter of the Association for Computational Linguistics: Main Volume*. 563–574.

[8] Anthropic. 2024. The Claude 3 Model Family: Opus, Sonnet, Haiku. https://www-cdn.anthropic.com/de8ba9b01c9ab7cbabf5c33b80b7bbc618857627/Model_Card_Claude_3.pdf Technical Report.

[9] Marcelo Arenas, Leopoldo Bertossi, and Jan Chomicki. 1999. Consistent Query Answers in Inconsistent Databases. In *PODS*. 68–79.

[10] Sören Auer, Christian Bizer, Georgi Kobilarov, Jens Lehmann, Richard Cyganiak, and Zachary Ives. 2007. DBpedia: A nucleus for a web of open data. In *international semantic web conference*. Springer, 722–735.

[11] Jinze Bai, Shuai Bai, Yunfei Chu, Zeyu Cui, Kai Dang, Xiaodong Deng, Yang Fan, Wenbin Ge, Yu Han, and Fei Huang. 2023. Qwen Technical Report. *arXiv preprint arXiv:2309.16609* (2023). https://arxiv.org/abs/2309.16609

[12] Xianchun Bao, Zian Bao, Qingsong Duan, Wenfei Fan, Hui Lei, Daji Li, Wei Lin, Peng Liu, Zhicong Lv, Mingliang Ouyang, Jiale Peng, Jing Zhang, Runxiao Zhao, Shuai Tang, Shuping Zhou, Yaoshu Wang, Qiyuan Wei, Min Xie, Jing Zhang, Xin Zhang, Runxiao Zhao, and Shuping Zhou. 2024. Rock: Cleaning Data by Embedding ML in Logic Rules. In *SIGMOD (industrial track)*.

[13] Iz Beltagy, Matthew E. Peters, and Arman Cohan. 2020. Longformer: The Long-Document Transformer. *arXiv preprint arXiv:2004.05150* (2020). https://arxiv.org/abs/2004.05150

[14] Christopher M. Bishop and Nasser M. Nasrabadi. 2006. *Pattern Recognition and Machine Learning*. Springer.

[15] Samuel R. Bowman, Gabor Angeli, Christopher Potts, and Christopher D. Manning. 2015. A Large Annotated Corpus for Learning Natural Language Inference. In *EMNLP*. 632–642.

[16] Hung-Ting Chen, Michael Zhang, and Eunsol Choi. 2022. Rich Knowledge Sources Bring Complex Knowledge Conflicts: Recalibrating Models to Reflect Conflicting Evidence. In *Conference on Empirical Methods in Natural Language Processing*. 2292–2307.

[17] Yuhao Chen, Yuanjie Lyu, Shuochen Liu, Chao Zhang, Junhui Lv, and Tong Xu. 2025. Think Wider, Detect Sharper: Reinforced Reference Coverage for Document-Level Self-Contradiction Detection. In *EMNLP*. 1273–1288.

[18] Hyung Won Chung, Le Hou, Shayne Longpre, Barret Zoph, Yi Tay, William Fedus, Yunxuan Li, Xuezhi Wang, Mostafa Dehghani, Siddhartha Brahma, and Jacob Devlin. 2024. Scaling Instruction-Finetuned Language Models. *JMLR* 25, 70 (2024), 1–53.

[19] Peter Coates and Frank Breitinger. 2023. Identifying document similarity using a fast estimation of the Levenshtein Distance based on compression and signatures. *arXiv preprint arXiv:2307.11496* (2023). https://arxiv.org/abs/2307.11496

[20] Mark Coddington and Logan Molyneux. 2023. When sources contradict: The epistemological functions of contradiction in news texts. *Journalism Studies* 24, 10 (2023), 1316–1333.

[21] Cleo Condoravdi, Dick Crouch, Valeria De Paiva, Reinhard Stolle, and Daniel Bobrow. 2003. Entailment, Intensionality and Text Understanding. In *NAACL*. 38–45.

[22] Gao Cong, Wenfei Fan, Floris Geerts, Xibei Jia, and Shuai Ma. 2007. Improving Data Quality: Consistency and Accuracy. In *VLDB*. 315–326.

[23] Ido Dagan, Oren Glickman, and Bernardo Magnini. 2005. The PASCAL Recognising Textual Entailment Challenge. In *Machine Learning Challenges Workshop*. Springer, 177–190.

[24] Marie-Catherine de Marneffe, Anna N. Rafferty, and Christopher D. Manning. 2008. Finding Contradictions in Text. In *ACL*. The Association for Computer Linguistics, 1039–1047.

[25] Tobias Deußer, Maren Pielka, Lisa Pucknat, Basil Jacob, Tim Dilmaghani, Mahdis Nourimand, Bernd Kliem, Rüdiger Loitz, Christian Bauckhage, and Rafet Sifa. 2023. Contradiction Detection in Financial Reports. In *NLDL*.

[26] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2019. BERT: Pre-Training of Deep Bidirectional Transformers for Language Understanding. In *NAACL*. 4171–4186.

[27] Xin Dong, Evgeniy Gabrilovich, Geremy Heitz, Wilko Horn, Ni Lao, Kevin Murphy, Thomas Strohmann, Shaohua Sun, and Wei Zhang. 2014. Knowledge Vault: A Web-Scale Approach to Probabilistic Knowledge Fusion. In *ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. 601–610.

[28] Yilun Du, Shuang Li, Antonio Torralba, Joshua B. Tenenbaum, and Igor Mordatch. 2024. Improving Factuality and Reasoning in Language Models Through Multiagent Debate. In *International Conference on Machine Learning*.

[29] David Easley, Jon Kleinberg, et al. 2010. *Networks, Crowds, and Markets: Reasoning about a Highly Connected World*. Vol. 1. Cambridge university press Cambridge.

[30] Darren Edge, Ha Trinh, Newman Cheng, Joshua Bradley, Alex Chao, Apurva Mody, Steven Truitt, Dasha Metropolitansky, Robert Osazuwa Ness, and Jonathan Larson. 2024. From local to global: A graph RAG approach to query-focused summarization. *arXiv preprint arXiv:2404.16130* (2024). https://arxiv.org/abs/2404.16130

[31] Wenfei Fan, Wenzhi Fu, Ruochun Jin, Ping Lu, and Chao Tian. 2022. Discovering Association Rules from Big Graphs. *PVLDB* 15, 7 (2022), 1479–1492.

[32] Wenfei Fan, Wenzhi Fu, Ruochin Jin, Ping Lu, and Chao Tian. 2023. Making It Tractable to Catch Duplicates and Conflicts in Graphs. *Proc. ACM Manag. Data* 1, 1 (2023), 86:1–86:28.

[33] Wenfei Fan, Floris Geerts, Xibei Jia, and Anastasios Kementsietsidis. 2008. Conditional functional dependencies for capturing data inconsistencies. *TODS* 33, 2 (2008), 6:1–6:48.

[34] Wenfei Fan, Floris Geerts, Nan Tang, and Wenyuan Yu. 2014. Conflict resolution with data currency and consistency. *J. Data and Information Quality* 5, 1-2 (2014), 6:1–6:37.

[35] Wenfei Fan, Ruochun Jin, Muyang Liu, Ping Lu, Chao Tian, and Jingren Zhou. 2020. Capturing Associations in Graphs. *PVLDB* 13, 11 (2020), 1863–1876.

[36] Wenfei Fan, Muyang Liu, Shuhao Liu, and Chao Tian. 2024. Capturing More Associations by Referencing External Graphs. *PVLDB* 17, 6 (2024), 1173–1186.

[37] Wenfei Fan, Muyang Liu, Ping Lu, and Chao Tian. 2023. Making It Tractable to Catch Duplicates and Conflicts in Graphs. *ACM Trans. on Database Systems* (2023).

[38] Wenfei Fan, Xueli Liu, Ping Lu, and Chao Tian. 2020. Catching Numeric Inconsistencies in Graphs. *TODS* 45, 2 (2020), 1–47.

[39] Wenfei Fan and Ping Lu. 2019. Dependencies for Graphs. *ACM TODS* 44, 2 (2019), 1–40.

[40] Wenfei Fan, Ping Lu, and Chao Tian. 2020. Unifying Logic Rules and Machine Learning for Entity Enhancing. *Sci. China Inf. Sci.* 63, 7 (2020).

[41] Wenfei Fan, Ping Lu, Chao Tian, and Jingren Zhou. 2019. Deducing Certain Fixes to Graphs. *PVLDB* 12, 7 (2019), 752–765.

[42] Wenfei Fan and Chao Tian. 2022. Incremental graph computations: Doable and undoable. *ACM Transactions on Database Systems (TODS)* 47, 2 (2022), 1–44.

[43] Wenfei Fan, Chao Tian, Yanghao Wang, and Qiang Yin. 2021. Parallel Discrepancy Detection and Incremental Detection. *PVLDB* 14, 8 (2021), 1351–1364.

[44] Wenfei Fan, Chao Tian, Ruiqi Xu, Qiang Yin, Wenyuan Yu, and Jingren Zhou. 2021. Incrementalizing graph algorithms. In *Proc. ACM Manag. Data*. 459–471.

[45] Wenfei Fan, Yinghui Wu, and Jingbo Xu. 2016. Functional dependencies for graphs. In *SIGMOD*.

[46] Wenfei Fan, Jingbo Xu, Yinghui Wu, Wenyuan Yu, Jiaxin Jiang, Bohan Zhang, Zeyu Zheng, Yang Cao, and Chao Tian. 2017. Parallelizing Sequential Graph Computations. In *SIGMOD*.

[47] Anna Fariha, Ashish Tiwari, Alexandra Meliou, Arjun Radhakrishna, and Sumit Gulwani. 2021. CoCo: Interactive Exploration of Conformance Constraints for Data Understanding and Data Cleaning. In *SIGMOD*. ACM, 2706–2710.

[48] Tom Fawcett. 2006. An introduction to ROC analysis. *Pattern recognition letters* 27, 8 (2006), 861–874.

[49] Matthias Feurer, Aaron Klein, Katharina Eggensperger, Jost Tobias Springenberg, Manuel Blum, and Frank Hutter. 2015. Efficient and Robust Automated Machine Learning. In *NeurIPS*. 2962–2970.

[50] Lampros Flokas, Weiyuan Wu, Yejia Liu, Jiannan Wang, Nakul Verma, and Eugene Wu. 2022. Complaint-Driven Training Data Debugging at Interactive Speeds. In *SIGMOD*. 369–383.

[51] Etienne Fortier-Dubois and Domenic Rosati. 2023. Using Contradictions Improves Question Answering Systems. In *ACL*. 827–840.

[52] Michael Garey and David Johnson. 1979. *Computers and Intractability: A Guide to the Theory of NP-Completeness*. W. H. Freeman and Company.

[53] Gemini Team and Google. 2023. Gemini: A Family of Highly Capable Multimodal Models. *arXiv preprint arXiv:2312.11805* (2023). https://arxiv.org/abs/2312.11805

[54] Danilo Giampiccolo, Bernardo Magnini, Ido Dagan, and William B. Dolan. 2007. The Third PASCAL Recognizing Textual Entailment Challenge. In *ACL*. 1–9.

[55] Justin Gilmer, Samuel S. Schoenholz, Patrick F. Riley, Oriol Vinyals, and George E. Dahl. 2017. Neural Message Passing for Quantum Chemistry. In *ICML*. 1263–1272.

[56] Joseph E. Gonzalez, Yucheng Low, Haijie Gu, Danny Bickson, and Carlos Guestrin. 2012. PowerGraph: Distributed Graph-Parallel Computation on Natural Graphs. In *OSDI*. 17–30.

[57] Rihan Hai, Christos Koutras, Andra Ionescu, Ziyu Li, Wenbo Sun, Jessie van

Schijndel, Yan Kang, and Asterios Katsifodimos. 2023. Amalur: Data Integration Meets Machine Learning. In *ICDE*. IEEE, 3729–3739.
[58] Will Hamilton, Zhitao Ying, and Jure Leskovec. 2017. Inductive Representation Learning on Large Graphs. In *NeurIPS*, Vol. 30. 1025–1035.
[59] Sanda Harabagiu, Andrew Hickl, and Finley Lacatusu. 2006. Negation, contrast and contradiction in text processing. In *AAAI*, Vol. 6. 755–762.
[60] Pengcheng He, Jianfeng Gao, and Weizhu Chen. 2023. DeBERTaV3: Improving DeBERTa using ELECTRA-Style Pre-Training with Gradient-Disentangled Embedding Sharing. In *International Conference on Learning Representations, ICLR*. OpenReview.net.
[61] Alireza Heidari, Joshua McGrath, Ihab F Ilyas, and Theodoros Rekatsinas. 2019. HoloDetect: Few-Shot Learning for Error Detection. In *SIGMOD*. 829–846.
[62] Andrew Hickl, John Williams, Jeremy Bensley, Kirk Roberts, Bryan Rink, and Ying Shi. 2006. Recognizing Textual Entailment with LCC's GROUNDHOG System. In *The Second PASCAL Challenges Workshop*, Vol. 18. 1–4.
[63] Benjamin Hilprecht, Christian Hammacher, Eduardo Souza dos Reis, Mohamed Abdelaal, and Carsten Binnig. 2023. DiffML: End-to-end Differentiable ML Pipelines. In *DEEM@SIGMOD*. ACM, 7:1–7:7.
[64] Johannes Hoffart, Mohamed Amir Yosef, Ilaria Bordino, Hagen Fürstenau, Manfred Pinkal, Marc Spaniol, Bilyana Taneva, Stefan Thater, and Gerhard Weikum. 2011. Robust Disambiguation of Named Entities in Text. In *EMNLP*. 782–792.
[65] Cheng Hsu, Cheng-Te Li, Diego Saez-Trumper, and Yi-Zhan Hsu. 2021. Wikicontradiction: Detecting Self-Contradiction Articles on Wikipedia. In *BigData*. 427–436.
[66] Weihua Hu, Bowen Liu, Joseph Gomes, Marinka Zitnik, Percy Liang, Vijay Pande, and Jure Leskovec. 2020. Strategies for Pre-training Graph Neural Networks. In *International Conference on Learning Representations*. https://openreview.net/forum?id=HJlWWJSFDH
[67] Ziniu Hu, Yuxiao Dong, Kuansan Wang, and Yizhou Sun. 2020. Heterogeneous Graph Transformer. In *The Web Conference 2020*. 2704–2710.
[68] Liusheng Huang, Huaping Chen, Xun Wang, and Guoliang Chen. 2000. A fast algorithm for mining association rules. *Journal of Computer Science and Technology* 15, 6 (2000), 619–624.
[69] Pengcheng Jiang, Lang Cao, Ruike Zhu, Minhao Jiang, Yunyi Zhang, Jimeng Sun, and Jiawei Han. 2025. RAS: Retrieval-And-Structuring for Knowledge-Intensive LLM Generation. *CoRR* abs/2502.10996 (2025). https://doi.org/10.48550/arXiv.2502.10996
[70] Cheng Jiayang, Chunkit Chan, Qianqian Zhuang, Lin Qiu, Tianhang Zhang, Tengxiao Liu, Yangqiu Song, Yue Zhang, Pengfei Liu, and Zheng Zhang. 2024. ECON: On the Detection and Resolution of Evidence Conflicts. In *Conference on Empirical Methods in Natural Language Processing*. 7816–7844.
[71] Di Jin, Sijia Liu, Yang Liu, and Dilek Hakkani-Tur. 2022. Improving Bot Response Contradiction Detection via Utterance Rewriting. In *SIGDIAL*. 605–614.
[72] Nikhil Kandpal, Haikang Deng, Adam Roberts, Eric Wallace, and Colin Raffel. 2023. Large language models struggle to learn long-tail knowledge. In *International conference on machine learning*. PMLR, 15696–15707.
[73] Bojan Karlas, Peng Li, Renzhi Wu, Nezihe Merve Gürel, Xu Chu, Wentao Wu, and Ce Zhang. 2020. Nearest Neighbor Classifiers over Incomplete Information: From Certain Answers to Certain Predictions. *CoRR* abs/2005.05117 (2020). ArXiv:2005.05117 https://arxiv.org/abs/2005.05117
[74] Thomas N. Kipf and Max Welling. 2017. Semi-Supervised Classification with Graph Convolutional Networks. In *International Conference on Learning Representations (ICLR)*.
[75] Yuta Koreeda and Christopher D. Manning. 2021. ContractNLI: A Dataset for Document-Level Natural Language Inference for Contracts. In *Findings of the Association for Computational Linguistics: EMNLP 2021*. 1907–1919.
[76] Sanjay Krishnan, Michael J. Franklin, Ken Goldberg, and Eugene Wu. 2017. BoostClean: Automated Error Detection and Repair for Machine Learning. *CoRR* abs/1711.01299 (2017).
[77] Sanjay Krishnan, Jiannan Wang, Eugene Wu, Michael J. Franklin, and Ken Goldberg. 2016. ActiveClean: Interactive Data Cleaning For Statistical Modeling. *PVLDB* 9, 12 (2016), 948–959.
[78] Sanjay Krishnan and Eugene Wu. 2019. AlphaClean: Automatic Generation of Data Cleaning Pipelines. *CoRR* abs/1904.11827 (2019). http://arxiv.org/abs/1904.11827
[79] Clyde P. Kruskal, Larry Rudolph, and Marc Snir. 1990. A Complexity Theory of Efficient Parallel Algorithms. *Theoretical Computer Science* 71, 1 (1990), 95–132.
[80] Piroska Lendvai, Isabelle Augenstein, Kalina Bontcheva, and Thierry Declerck. 2016. Monolingual Social Media Datasets for Detecting Contradiction and Entailment. In *LREC*. 4602–4605.
[81] Piroska Lendvai and Uwe Reichel. 2016. Contradiction Detection for Rumorous Claims. In *The Workshop on Extra-Propositional Aspects of Meaning in Computational Linguistics (ExProM)*. 31–40.
[82] Mike Lewis, Yinhan Liu, Naman Goyal, Marjan Ghazvininejad, Abdelrahman Mohamed, Omer Levy, Veselin Stoyanov, and Luke Zettlemoyer. 2020. BART: Denoising Sequence-to-Sequence Pre-Training for Natural Language Generation, Translation, and Comprehension. In *ACL*. 7871–7880.
[83] Chuqin Li, Xi Niu, Ahmad Al-Doulat, and Noseong Park. 2018. A Computational Approach to Finding Contradictions in User Opinionated Text. In *ASONAM*. 351–356.
[84] Jierui Li, Vipul Raheja, and Dhruv Kumar. 2024. ContraDoc: Understanding Self-Contradictions in Documents with Large Language Models. In *Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*. 6509–6523.
[85] Peng Li, Zhiyi Chen, Xu Chu, and Kexin Rong. 2023. DiffPrep: Differentiable Data Preprocessing Pipeline Search for Learning over Tabular Data. *Proc. ACM Manag. Data* 1, 2 (2023), 183:1–183:26.
[86] Peng Li, Xi Rao, Jennifer Blase, Yue Zhang, Xu Chu, and Ce Zhang. 2021. CleanML: A Study for Evaluating the Impact of Data Cleaning on ML Classification Tasks. In *ICDE*. IEEE, 13–24.
[87] Aixin Liu, Bei Feng, Bing Xue, Bingxuan Wang, Bochao Wu, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chenyu Zhang, and Chong Ruan. 2024. DeepSeek-V3 Technical Report. *arXiv preprint arXiv:2412.19437* (2024). https://arxiv.org/abs/2412.19437
[88] Hanmeng Liu, Leyang Cui, Jian Liu, and Yue Zhang. 2021. Natural language inference in context-investigating contextual reasoning over long texts. In *AAAI*, Vol. 35. 13388–13396.
[89] Nelson F Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. 2024. Lost in the middle: How language models use long contexts. *Transactions of the association for computational linguistics* 12 (2024), 157–173.
[90] Yinhan Liu, Myle Ott, Naman Goyal, Jingfei Du, Mandar Joshi, Danqi Chen, Omer Levy, Mike Lewis, Luke Zettlemoyer, and Veselin Stoyanov. 2019. RoBERTa: A Robustly Optimized BERT Pretraining Approach. *arXiv preprint arXiv:1907.11692* (2019). https://arxiv.org/abs/1907.11692
[91] Zifan Liu, Zhechun Zhou, and Theodoros Rekatsinas. 2020. Picket: Self-supervised Data Diagnostics for ML Pipelines. *CoRR* abs/2006.04730v1 (2020). https://arxiv.org/abs/2006.04730v1
[92] Shayne Longpre, Kartik Perisetla, Anthony Chen, Nikhil Ramesh, Chris DuBois, and Sameer Singh. 2021. Entity-Based Knowledge Conflicts in Question Answering. In *Conference on Empirical Methods in Natural Language Processing*. 7052–7063.
[93] Bill MacCartney, Trond Grenager, Marie-Catherine de Marneffe, Daniel Cer, and Christopher D. Manning. 2006. Learning to Recognize Features of Valid Textual Entailments. In *ACL*. 41–48.
[94] Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegreffe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, Shashank Gupta, Bodhisattwa Prasad Majumder, Katherine Hermann, Sean Welleck, Amir Yazdanbakhsh, and Peter Clark. 2023. Self-refine: Iterative refinement with self-feedback. *Advances in neural information processing systems* 36 (2023), 46534–46594.
[95] Mohammad Mahdavi, Ziawasch Abedjan, Raul Castro Fernandez, Samuel Madden, Mourad Ouzzani, Michael Stonebraker, and Nan Tang. 2019. Raha: A Configuration-Free Error Detection System. In *SIGMOD*. 865–882.
[96] Stanislav Malyshev, Markus Krötzsch, Larry González, Julius Gonsior, and Adrian Bielefeldt. 2018. Getting the Most Out of Wikidata: Semantic Technology Usage in Wikipedia's Knowledge Graph. In *ISWC*. 376–394.
[97] Puneet Mathur, Gautam Kunapuli, Riyaz Bhat, Manish Shrivastava, Dinesh Manocha, and Maneesh Singh. 2022. DocInfer: Document-level natural language inference using optimal evidence selection. In *EMNLP*. 809–824.
[98] Christian Meilicke, Melisachew Wudage Chekol, Manuel Fink, and Heiner Stuckenschmidt. 2020. Reinforced Anytime Bottom-Up Rule Learning for Knowledge Graph Completion. *arXiv preprint arXiv:2004.04412* (2020). https://arxiv.org/abs/2004.04412
[99] Roser Morante and Caroline Sporleder. 2012. Modality and Negation: An Introduction to the Special Issue. *Comput. Linguist.* 38, 2 (2012), 223–260.
[100] Yixin Nie, Mary Williamson, Mohit Bansal, Douwe Kiela, and Jason Weston. 2021. I Like Fish, Especially Dolphins: Addressing Contradictions in Dialogue Modeling. In *ACL*. 1699–1713.
[101] OpenAI. 2025. OpenAI GPT-5: A Comprehensive Technical Overview. https://cirra.ai/articles/gpt-5-technical-overview
[102] Shirui Pan, Linhao Luo, Yufei Wang, Chen Chen, Jiapu Wang, and Xindong Wu. 2024. Unifying Large Language Models and Knowledge Graphs: A Roadmap. *TKDE* 36, 7 (2024), 3580–3599.
[103] James Pustejovsky, José M. Castano, Robert Ingria, Roser Sauri, Robert J. Gaizauskas, Andrea Setzer, Graham Katz, and Dragomir R. Radev. 2003. TimeML: Robust Specification of Event and Temporal Expressions in Text. *New Directions in Question Answering* 3 (2003), 28–34.
[104] ABM Ashikur Rahman, Saeed Anwar, Muhammad Usman, Irfan Ahmad, and Ajmal Mian. 2025. DefAn: Definitive Answer Dataset for LLM Hallucination Evaluation. *Information* 16, 11 (2025), 937.
[105] Qiu Ran, Yankai Lin, Peng Li, Jie Zhou, and Zhiyuan Liu. 2019. NumNet: Machine Reading Comprehension with Numerical Reasoning. In *Conference on Empirical Methods in Natural Language Processing and the International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*. 2474–2484.
[106] Alexander Ratner, Stephen H. Bach, Henry R. Ehrenberg, Jason Alan Fries, Sen

Wu, and Christopher Ré. 2017. Snorkel: Rapid Training Data Creation with Weak Supervision. *PVLDB* 11, 3 (2017), 269–282.

[107] Nils Reimers and Iryna Gurevych. 2019. Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. In *EMNLP-IJCNLP*. 3980–3990.

[108] Graciela Rosemblat, Marcelo Fiszman, Dongwook Shin, and Halil Kilicoglu. 2019. Towards a Characterization of Apparent Contradictions in the Biomedical Literature Using Context Analysis. *JBI* 98 (2019), 103275.

[109] Victor Sanh, Lysandre Debut, Julien Chaumond, and Thomas Wolf. 2019. DistilBERT, a Distilled Version of BERT: Smaller, Faster, Cheaper and Lighter. *arXiv preprint arXiv:1910.01108* (2019). https://arxiv.org/abs/1910.01108

[110] Farzaneh Sarafraz. 2012. *Finding Conflicting Statements in the Biomedical Literature*. Ph. D. Dissertation. The University of Manchester.

[111] Michael Schlichtkrull, Thomas N. Kipf, Peter Bloem, Rianne Van Den Berg, Ivan Titov, and Max Welling. 2018. Modeling Relational Data with Graph Convolutional Networks. In *ESWC*. 593–607.

[112] Tal Schuster, Sihao Chen, Senaka Buthpitiya, Alex Fabrikant, and Donald Metzler. 2022. Stretching Sentence-pair NLI Models to Reason over Long Documents and Clusters. In *Findings of the Association for Computational Linguistics: EMNLP 2022*. 394–412.

[113] Özge Sevgili, Artem Shelmanov, Mikhail Arkhipov, Alexander Panchenko, and Chris Biemann. 2022. Neural Entity Linking: A Survey of Models Based on Deep Learning. *Semantic Web* 13, 3 (2022), 527–570.

[114] Wei Shen, Jianyong Wang, and Jiawei Han. 2014. Entity Linking with a Knowledge Base: Issues, Techniques, and Solutions. *TKDE* 27, 2 (2014), 443–460.

[115] Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. 2023. Reflexion: Language Agents with Verbal Reinforcement Learning. *NeurIPS* 36 (2023), 8634–8652.

[116] Zhaochen Su, Zecheng Tang, Xinyan Guan, Lijun Wu, Min Zhang, and Juntao Li. 2022. Improving Temporal Generalization of Pre-Trained Language Models with Lexical Semantic Change. In *EMNLP*. 6380–6393.

[117] Zhaochen Su, Jun Zhang, Xiaoye Qu, Tong Zhu, Yanshu Li, Jiashuo Sun, Juntao Li, Min Zhang, and Yu Cheng. 2024. ConflictBank: A Benchmark for Evaluating the Influence of Knowledge Conflicts in LLMs. In *NeurIPS*, Vol. 37. 103242–103268.

[118] Melanie Subbiah, Faisal Ladhak, Akankshya Mishra, Griffin Thomas Adams, Lydia Chilton, and Kathleen McKeown. 2024. StorySumm: Evaluating Faithfulness in Story Summarization. In *EMNLP*. 9988–10005.

[119] Katia P. Sycara. 1993. Machine learning for intelligent support of conflict resolution. *Decision Support Systems* 10, 2 (1993), 121–136.

[120] Ki Hyun Tae, Yuji Roh, Young Hun Oh, Hyunsu Kim, and Steven Euijong Whang. 2019. Data Cleaning for Accurate, Fair, and Robust Models: A Big Data - AI Integration Approach. In *DEEM@SIGMOD*. ACM, 5:1–5:4.

[121] James Thorne, Andreas Vlachos, Christos Christodoulopoulos, and Arpit Mittal. 2018. FEVER: a Large-scale Dataset for Fact Extraction and VERification. In *Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long Papers)*. 809–819.

[122] Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, and Shruti Bhosale. 2024. The Llama 3 Herd of Models. *arXiv preprint arXiv:2407.21783* (2024). arXiv:2407.21783 [cs.AI] https://arxiv.org/abs/2407.21783

[123] Leslie G. Valiant. 1990. A Bridging Model for Parallel Computation. *Commun. ACM* 33, 8 (1990), 103–111.

[124] Petar Veličković, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Liò, and Yoshua Bengio. 2018. Graph Attention Networks. arXiv:1710.10903 [stat.ML] https://arxiv.org/abs/1710.10903

[125] Larysa Visengeriyeva and Ziawasch Abedjan. 2018. Metadata-driven error detection. In *SSDBM*. 1:1–1:12.

[126] Denny Vrandečić. 2012. Wikidata: A New Platform for Collaborative Data Collection. In *International Conference on World Wide Web*. 1063–1064.

[127] Alexander Wan, Eric Wallace, and Dan Klein. 2024. What Evidence Do Language Models Find Convincing? arXiv:2402.11782 [cs.CL] https://arxiv.org/abs/2402.11782

[128] Keheng Wang, Feiyu Duan, Peiguang Li, Sirui Wang, and Xunliang Cai. 2025. LLMs Know What They Need: Leveraging a Missing Information Guided Framework to Empower Retrieval-Augmented Generation. In *COLING*. 2379–2400.

[129] Yike Wang, Shangbin Feng, Heng Wang, Weijia Shi, Vidhisha Balachandran, Tianxing He, and Yulia Tsvetkov. 2024. Resolving Knowledge Conflicts in Large Language Models. arXiv:2310.00935 [cs.CL] https://arxiv.org/abs/2310.00935

[130] Yizhong Wang, Yeganeh Kordi, Swaroop Mishra, Alisa Liu, Noah A. Smith, Daniel Khashabi, and Hannaneh Hajishirzi. 2023. Self-Instruct: Aligning Language Models with Self-Generated Instructions. In *ACL*. 13484–13508.

[131] Adina Williams, Nikita Nangia, and Samuel Bowman. 2018. A Broad-Coverage Challenge Corpus for Sentence Understanding Through Inference. In *Conference of the North American chapter of the association for computational linguistics: human language technologies, volume 1 (long papers)*. 1112–1122.

[132] Weiyuan Wu, Lampros Flokas, Eugene Wu, and Jiannan Wang. 2020. Complaint-driven Training Data Debugging for Query 2.0. In *SIGMOD*. ACM, 1317–1334.

[133] Jian Xie, Kai Zhang, Jiangjie Chen, Renze Lou, and Yu Su. 2023. Adaptive Chameleon or Stubborn Sloth: Revealing the Behavior of Large Language Models in Knowledge Conflicts. In *ICLR*.

[134] Wenhan Xiong, Thien Hoang, and William Yang Wang. 2017. DeepPath: A Reinforcement Learning Method for Knowledge Graph Reasoning. In *Conference on Empirical Methods in Natural Language Processing*. 564–573.

[135] Haike Xu, Zongyu Lin, Yizhou Sun, Kai-Wei Chang, and Piotr Indyk. 2024. SparseCL: Sparse Contrastive Learning for Contradiction Retrieval. arXiv:2406.10746 [cs.CL] https://arxiv.org/abs/2406.10746

[136] Rongwu Xu, Brian Lin, Shujian Yang, Tianqi Zhang, Weiyan Shi, Tianwei Zhang, Zhixuan Fang, Wei Xu, and Han Qiu. 2024. The Earth Is Flat Because...: Investigating LLMs' Belief Towards Misinformation via Persuasive Conversation. In *ACL*. 16259–16303.

[137] Zhilin Yang, Zihang Dai, Yiming Yang, Jaime Carbonell, Russ R. Salakhutdinov, and Quoc V. Le. 2019. XLNet: Generalized Autoregressive Pretraining for Language Understanding. *NeurIPS* 32 (2019).

[138] Wenpeng Yin, Dragomir Radev, and Caiming Xiong. 2021. DocNLI: A Large-Scale Dataset for Document-Level Natural Language Inference. In *Findings of the Association for Computational Linguistics: ACL-IJCNLP 2021*. 4913–4922.

[139] Zhangdie Yuan and Andreas Vlachos. 2023. Zero-Shot Fact-Checking with Semantic Triples and Knowledge Graphs. *CoRR* abs/2312.11785 (2023).

[140] Fabio Massimo Zanzotto, Marco Pennacchiotti, and Alessandro Moschitti. 2007. Shallow Semantic in Fast Textual Entailment Rule Learners. In *ACL*. 72–77.

[141] Urchade Zaratiana, Nadi Tomeh, Pierre Holat, and Thierry Charnois. 2024. GLiNER: Generalist Model for Named Entity Recognition Using Bidirectional Transformer. In *Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*. 5364–5376.

[142] Jiawei Zhang, Haopeng Zhang, Congying Xia, and Li Sun. 2020. Graph-Bert: Only Attention is Needed for Learning Graph Representations. (2020). arXiv:2001.05140 [cs.LG] https://arxiv.org/abs/2001.05140

[143] Yue Zhang, Yafu Li, Leyang Cui, Deng Cai, Lemao Liu, Tingchen Fu, Xinting Huang, Enbo Zhao, Yu Zhang, Yulong Chen, Longyue Wang, Anh Tuan Luu, Wei Bi, Freda Shi, and Shuming Shi. 2025. Siren's Song in the AI Ocean: A Survey on Hallucination in Large Language Models. *Computational Linguistics* 51, 4 (2025), 1373–1418.

[144] Chujie Zheng, Jinfeng Zhou, Yinhe Zheng, Libiao Peng, Zhen Guo, Wenquan Wu, Zheng-Yu Niu, Hua Wu, and Minlie Huang. 2022. CDConv: A Benchmark for Contradiction Detection in Chinese Conversations. In *EMNLP*. 18–29.

[145] Lucia Zheng, Neel Guha, Brandon R. Anderson, Peter Henderson, and Daniel E. Ho. 2021. When Does Pretraining Help? Assessing Self-Supervised Learning for Law and the CaseHOLD Dataset of 53,000+ Legal Holdings. In *International Conference on Artificial Intelligence and Law*. 159–168.

A COMPARING RICOS WITH PRIOR RULES

Below we compare RICOs with previous graph rules. To highlight the expressivity of RICOs, we further compare them with recent representative graph cleaning rules (GCRs) [37]. GCRs take the form $Q[x_0, y_0](X \rightarrow p_0)$, where the consequent p_0 is an equality predicate and other predicates are limited to equality, constants, similarity or ER functions: $p ::= x.A = y.B \mid z.A = c \mid \mathcal{M}(x.\bar{A}, y.\bar{B})$.

(1) Expressivity. RICOs explicitly account for linguistic variation and ambiguity through substring bindings and textual classifiers, bridging structured graph reasoning with unstructured natural language text, which is not addressed in prior graph rule frameworks.

Unlike earlier graph rules such as GARs [35] and GCRs [32, 37], which operate on graphs where only vertices carry properties, RICOs are defined on property graphs where both vertices and edges may have attributes. They further support: (a) edge predicates ($e_x.A \oplus e_y.B$, $e_z.A \oplus c$, $\mathcal{M}(e_x.\bar{A}, e_y.\bar{B})$); (b) textual classifiers $T(s, s', \tau)$; and (c) substring bindings $S(e_x, s)$. Thus, RICOs detect contradictions on both entities (vertices) and relationships (edges), whereas GARs/GCRs capture only entity-level conflicts.

More specifically, compared with GCRs, RICOs provide four advancements for textual contradiction detection: (1) NLP predicates. Instead of rigid equality or basic ML similarity, RICOs introduce textual classifiers $T(s, s', \tau)$ and type predicates $\tau(x.A, y.B)$. Leveraging models such as SBERT, NumNet, and LLMs (*e.g.,* GPT-5.2), they capture semantic conflicts (*e.g.,* negations, causal inconsistencies) beyond simple value mismatches. (2) Edge-level reasoning. Unlike GCRs that focus on vertices, RICOs integrate edge attributes into reasoning, which captures relational contradictions. (3) Structural inference. By implementing graph-based models (*e.g.,* GraphSAGE and HGT) as $\mathcal{M}$ predicates, RICOs capture contextual dependencies to infer implicit contradictions emerging from multi-entity interactions. (4) Rich comparison logic. Beyond GCR's equality consequence (p_0), RICOs support a range of operators $\oplus \in \{=, \neq, <, \leq, >, \geq\}$ and textual contradiction types τ, which are essential for identifying numeric and temporal conflicts.

(2) Pattern structure. Like GCRs, RICOs adopt extended star-shaped dual patterns, enabling efficient pairwise comparison of entity (or relationship) features. In contrast, GFDs [45] and GARs [35] are defined on generic graph patterns, which offer generality but are intractable for valuation finding and contradiction detection.

B INCREMENTAL MAINTENANCE

This section presents incremental processes for graph conversion and fusion, RICO discovery, and contradiction detection.

B.1 Incremental Maintenance of Graphs

To handle dynamic document collections without costly full recomputations, DocChecker employs an incremental mechanism. Following the principles of *localizable incremental computation* [42, 44], we derive an incremental algorithm $\mathcal{A}'$ from our graph conversion and fusion algorithm $\mathcal{A}$. The strategy is to trace the computational dependencies of $\mathcal{A}$ and identify its *affected area* (AFF), defined as the set of status variables invalidated by updates. Instead of executing $\mathcal{A}$ blindly to over the entire corpus, $\mathcal{A}'$ restricts the execution of $\mathcal{A}$'s exact operators exclusively to the bounded AFF. This derives a localized graph update ΔG and maintains the global graph by treating $G \oplus \Delta G$ as G, such that it produces identical results to a full re-computation with strictly bounded costs.

Incremental graph conversion and enrichment. By identifying the AFF upon the document updates $\Delta\mathcal{D}$ and their graph mappings ΔG, $\mathcal{A}'$ restricts $\mathcal{A}$'s extraction operators exclusively to the changed inputs. We categorize the computation of ΔG into three scenarios based on the affected inputs. (a) For additions (ΔG^+), where new documents or text fragments are inserted, $\mathcal{A}'$ directly applies the sequential pipeline of $\mathcal{A}$ (*i.e.,* NER, LLM mapping, and KG enrichment) exclusively to the new text, extracting new entities to generate ΔG^+. (b) For deletions (ΔG^-), where text is removed, $\mathcal{A}'$ completely bypasses the costly LLM extraction operators of $\mathcal{A}$. Instead, it exploits provenance tracking (linking text spans to graph primitives) to instantly identify and mark obsolete entities and edges as ΔG^-. (c) Regarding updates (ΔG^u) to existing fragments, $\mathcal{A}'$ triggers $\mathcal{A}$'s semantic extraction operators strictly on the semantically shifted parts to compute attribute modifications. By restricting extraction to the AFF, $\mathcal{A}'$ reduces the complexity from $O(NL)$ in Section 5.1 to $O(|\Delta\mathcal{D}| \cdot L)$, ensuring maintenance costs are relatively bounded by the update size and independent of N.

Incremental graph fusion. To derive the incremental fusion step, we analyze the structural dependencies of $\mathcal{A}$'s global similarity join and HGNN aggregation. Since $\mathcal{A}$ computes node embeddings via a localized d-hop receptive field ($d = 2$), an update in ΔG only invalidates the embedding status variables within a strict d-hop radius, which constitutes the AFF for this stage. Thus, $\mathcal{A}'$ replaces the $O(n \log n)$ global re-fusion of $\mathcal{A}$ with targeted $\oplus$ operations on these specific variables. First, for handling additions (ΔG^+), $\mathcal{A}'$ isolates the new nodes and their d-hop neighbors, re-evaluating $\mathcal{A}$'s HGNN equations solely on these nodes to compute new embeddings ($\mathbf{h}_u$), followed by querying the vector index to establish *new* ϵ-edges. Second, when handling deletions (ΔG^-), $\mathcal{A}'$ prunes the invalidated topology by checking the connectivity of the deleted node u: if u is a direct endpoint of an ϵ-edge, $\mathcal{A}'$ removes that specific ϵ-edge from the global graph G; otherwise, it only prunes u's link relationship within the alignment structure while preserving the existing ϵ-edges that connect other surviving coreferent nodes. Finally, for handling updates (ΔG^u), the structural existence of the ϵ-edges remains intact. Algorithm $\mathcal{A}'$ merely feeds the updated attributes back into $\mathcal{A}$'s HGNN equations to refresh the embeddings of the modified nodes and their d-hop neighbors across the ϵ-edges. The total incremental fusion complexity is bounded to $O(|V_{\Delta G}| \cdot \log n)$, where $|V_{\Delta G}|$ is the number of updated nodes. This achieves marginal overhead for dynamic updates since $|V_{\Delta G}| \ll n$.

B.2 Incremental RICO Discovery

To handle evolving data streams, Learner avoids costly full recomputation. Instead, the incremental variant $\text{Learner}_{\text{Inc}}$ maintains the rule set Σ by confining updates to the affected area (AFF) induced by ΔG. For deletions (ΔG^-) and updates (ΔG^u), $\text{Learner}_{\text{Inc}}$ circumvents global metric re-evaluation by maintaining localized sufficient statistics (*e.g.,* match counts). It just computes the delta of these counts within the AFF to update rule coverage (f_c) and accuracy (f_a) in $O(|\Delta G|)$ time, dynamically pruning or specializing rules that decay below the thresholds α and β. For graph additions (ΔG^+), $\text{Learner}_{\text{Inc}}$ reduces redundant LLM calls. It first applies the existing

Σ to the new subgraphs, re-activating the LLM-driven discovery exclusively for novel structural contradictions $\mathbb{Q}_{\text{new}}$ that remain uncovered by Σ. Concurrently, the meta-learning feedback loop incorporates generalized patterns from the updates into the few-shot prompt buffer as continuous training signals. By integrating these localized updates, $\text{Learner}_{\text{Inc}}$ incrementally adapts to recent data distributions, bounding the maintenance cost to ΔG while preserving high performance with marginal overhead.

B.3 Incremental Contradiction Detection

To ensure consistent reasoning as documents evolve, algorithm Detector extends the localizable computation framework through incremental re-chasing. Rather than re-evaluating the full rule set Σ globally, it isolates an active subset $\Sigma_\Delta \subseteq \Sigma$ whose preconditions intersect with the updates ΔG. The affected area (AFF) is formally defined as the union of localized subgraphs triggered by ΔG and their cascading RICO dependency chains. Detector restricts the execution of the chase operator exclusively to this AFF, and propagates new facts until a local stable fixpoint is reached.

Specifically, while entity/edge additions in ΔG may trigger new chase sequences to derive further facts, entity/edge deletions or attribute updates in entities/edges can cause interruptions in existing deduction paths by invalidating previously held facts. In such cases, Detector identifies the point of divergence and re-computes the affected dependency chain until a new termination state is reached. This localized deduction is guaranteed to be equivalent to a full global re-chase, *i.e.,* $\text{chase}(G \oplus \Delta G, \Sigma) = G \oplus \text{chase}(\text{AFF}, \Sigma_\Delta)$, due to the Church-Rosser property of the RICO chase. Specifically, the confluence and monotonicity of RICOs ensure that the final fixpoint is unique and invariant for any facts outside the logical ripple effect of ΔG. Consequently, contradiction labels $[x.A]_\otimes$ are toggled only within the AFF, explicitly bounding the incremental reasoning complexity to $O(|\text{AFF}_{\text{chase}}|)$ and ensuring that computational costs scale strictly with the update size rather than the entire graph.

C PROOF OF THEOREM 1

Proof: Given a graph G, a document collection $\mathcal{D}$, and a set Σ of RICOs, validation asks whether $(G, \mathcal{D}) \models \Sigma$, *i.e.,* for every $\varphi = Q[x_0, y_0](X \to q_0) \in \Sigma$, there exists no valuation h of φ such that $h \models X$, but $h \not\models q_0$. It suffices to validate a single φ in PTIME.

Predicate evaluation. Each predicate in RICO is polynomially checkable. Attribute predicates $z.A \oplus c$ and $x.A \oplus y.B$ require constant-time lookups and scalar comparison. Binding $S(x, s)$ performs indexed span lookup in at most linear time. Textual and ML predicates (T, $\mathcal{M}$, τ) have PTIME cost relative to $|G|$ and $|\mathcal{D}|$, following standard bounded-input assumptions in dependency validation [32, 39]. Thus, every atomic predicate in X and q_0 is decidable in PTIME.

Pattern matching and violation checking. Instead of enumerating all subgraphs of Q, pattern matching follows the PTIME strategy in [37]. For each rule $\varphi \in \Sigma$, the algorithm iterates over vertex pairs $(u, v) \in V_G \times V_G$ as candidates for (x_0, y_0). It first checks unary predicates on x_0 and y_0, reducing the search space to $O(|V_G|^2)$ root candidates. If satisfied, matching extends to leaf variables (x_i, y_j) and incident edges (e_{x_i}, e_{y_j}). RICO 's structural constraints then ensure tractability: by Restriction (1) of the definition of RICOs (Section 3.2), binary predicates appear only across the two stars; by Restriction (2), each leaf and its incident edge participate in *at most one* binary predicate. Consequently, verifying whether a valuation satisfies $X \wedge \neg q_0$ reduces to independent checks over neighbors of u and v, effectively a linear scan or hash-join over adjacency lists, rather than an exponential search over leaf combinations [37].

Complexity. Let $n = |V_G|$. For each rule $\varphi \in \Sigma$, the algorithm examines $O(n^2)$ candidate root pairs (u, v). For each pair, verifying the binary predicates requires scanning the neighbors of u and v. Since each leaf participates in at most one binary predicate, checking k binary predicates reduces to k independent neighbor checks, costing $O(k \cdot \deg(G)^2)$ in the worst case. Thus, the total complexity is

$$O(|\Sigma| \cdot n^2 \cdot \deg(G)^2).$$

Note that the exponential factor $\deg(G)^{|\bar{x}|+|\bar{y}|}$ that appears in generic graph pattern matching is avoided because leaf-pair valuations are decoupled. Hence, RICO validation is in PTIME. □

D COMPLEXITY ANALYSIS OF DocChecker

Next, we analyze the complexity of key steps in DocChecker.

D.1 Complexity of Graph Conversion

Graph conversion comprises three steps: (1) NER-based context extraction, (2) LLM-based graph generation, and (3) KG enrichment. Let N be the number of documents; L_D the average document length; L_P and L_G the prompt and generated-graph lengths; and $|\mathcal{E}|$ the number of entities to enrich. The total cost is $O(NL)$, where $L = L_D + L_P + L_G + |\mathcal{E}|$. Specifically, NER costs $O(NL_D)$, graph generation costs $O\big(N(L_P + L_G)\big)$, and enrichment costs $O(N|\mathcal{E}|)$. The documents are independent and thus are processed in parallel, running in $O(NL/K)$ on K processors under balanced load.

D.2 Complexity of Graph Fusion

Let $n = |V_{\text{total}}|$ and $\mathcal{E}_{\text{total}}$ be the node and edge sets of the final graph G. The combined *graph join* (entity alignment) and *graph fusion* (aggregation and enrichment) steps run in $O(n \log n + n + |\mathcal{E}_{\text{total}}| + q)$ time, where $q \le n$ is the number of KG queries. Blocking (*e.g.,* hash or LSH) reduces alignment from $O(n^2)$ to $O(n \log n)$, and fusion performs a linear scan of all nodes and edges plus q constant-time lookups. Since $q = O(n)$, the total complexity becomes $O(n \log n + |\mathcal{E}_{\text{total}}|)$, excluding system-level overhead such as network latency.

E PROOF OF PROPOSITION 3

We prove this based on the locality of operators within $\mathcal{A}$:

(1) Invariance of the complement ($\mathcal{D} \setminus AFF$). Let O be the set of deterministic operators in the sequential pipeline $\mathcal{A}$. Each operator $o \in O$ (*e.g.,* NER, LLM mapping, HGNN fusion) possesses a bounded receptive field. For any data primitive $d \in \mathcal{D}$ that lies outside the AFF, its provenance traces back exclusively to unchanged partitions of the document set. In the conversion stage, the LLM extraction for D_i is independent of D_j ($i \neq j$). In the fusion stage, an embedding $\mathbf{h}_u$ remains invariant if its d-hop neighborhood $\mathcal{N}^d(u)$ does not intersect with the update $\Delta\mathcal{D}$. Since $\mathcal{A}$ is deterministic, the states of these elements are guaranteed to remain identical to those produced by a full re-computation on $\mathcal{D} \oplus \Delta\mathcal{D}$.

(2) Equivalence within the affected area (AFF). For elements within the AFF, $\mathcal{A}'$ is derived by projecting the global operators of $\mathcal{A}$ onto

the localized update $\Delta\mathcal{D}$. For additions, $\mathcal{A}'$ applies the exact sequential operators of $\mathcal{A}$ to the new fragments. For deletions and updates, provenance tracking identifies invalidated spans, ensuring that removing or refreshing these specific nodes is functionally equivalent to re-processing the updated documents and differencing the graph states. The HGNN aggregation is a linearizable localized operator. By re-evaluating aggregation equations $\mathbf{h}_u^{(l+1)} = \sigma(\sum \mathbf{h}_v^{(l)})$ specifically for $u \in$ AFF, $\mathcal{A}'$ replicates the embedding shifts. The subsequent ϵ-edge adjustments use the same thresholding as $\mathcal{A}$, such that the updated topology aligns with global results.

Since algorithm $\mathcal{A}'$ restricts the exact operators of $\mathcal{A}$ to the minimal sufficient set of variables (AFF) while preserving all invariant states, the final integrated result $\mathcal{D} \oplus \mathcal{A}'(\mathcal{D}, \Delta\mathcal{D})$ is point-wise equivalent to the full re-computation $\mathcal{A}(\mathcal{D} \oplus \Delta\mathcal{D})$. □

F CONTRADICTION DETECTION

We next provide a detailed explanation of the contradiction detection and document marking procedures of DocChecker. We focus on the chase process in contradiction detection, prove its Church-Rosser property, and present the parallel detection algorithm. We also describe how detected contradictions are mapped back to the original documents for document marking.

F.1 The Chase Process

Facts. To track facts for contradictions, for each attribute $x.A$ (resp. $e_x.A$), we maintain relations $[x.A]_\otimes$ (resp. $[e_x.A]_\otimes$), where $\otimes$ ranges over $=, \neq, <, \leq, >, \geq$. Each $[x.A]_\otimes$ (resp. $[e_x.A]_\otimes$) consists of attributes $y.B$ (resp. $e_y.B$) or constants c such that $x.A \otimes y.B$ (resp. $e_x.A \otimes e_y.B$) and $x.A \otimes c$ (resp. $e_x.A \otimes c$) are either in the initial ground truth Γ_0 (see below) or generated during the chase.

In particular, if $x.A$ (resp. $e_x.A$) is in contradiction, then $\tau(x.A, y.B)$ (resp., $\tau(e_x.A, e_y.B)$) may include a contradiction type (*i.e.,* casual, content, etc.) as a special value for label.

We ensure that each $[x.A]_=$ (resp. $[e_x.A]_=$) forms an equivalence relation, *i.e.,* it is reflexive, symmetric, and transitive, while each $[x.A]_\otimes$ (resp. $[e_x.A]_\otimes$) for any other $\otimes$ is reflexive and transitive. Moreover, each $[x.A]_\otimes$ (resp. $[e_x.A]_\otimes$) is *valid*, *i.e.,* no two constants within the same class conflict; for example, $[x.A]_=$ (resp. $[e_x.A]_=$) cannot contain constants c_1 and c_2 with $c_1 \neq c_2$.

Ground truth. The initial fact set, denoted Γ_0, is collected and validated by users, domain experts or crowd-sourcing.

The chase. A *chase step* of Graph G (converted from documents $\mathcal{D}$) with a set Σ_f of RICOs at a set $\mathcal{F}$ of facts is defined as:

$$\mathcal{F} \Rightarrow_{(\varphi,h)} \mathcal{F}'.$$

Here $\varphi : Q[x_0, y_0](X \to q_0)$ is a rule in Σ_f and h is a valuation of φ such that the following conditions are satisfied:

(a) All predicates in X are *validated* by $\mathcal{F}$: if p is $x.A \otimes y.B$, then $h(y).B \in [h(x).A]_\otimes$ for $[h(x).A]_\otimes$ in $\mathcal{F}$; similarly for $e_x.A \otimes e_y.B$. If p is $x.A \otimes c$, then $c \in [h(x).A]_\otimes$; similarly for $e_x.A \otimes c$. If p is $\mathcal{M}(x.\bar{A}, y.\bar{B})$, then $\mathcal{M}$ predicts true at $h(x).\bar{A}$ and $h(y).\bar{B}$, and $h(x).A \in [x.A]_{\text{eq}}$, $h(y).B \in [y.B]_{\text{eq}}$ for $A \in \bar{A}$ and $B \in \bar{B}$; similarly for all $\mathcal{M}(e_x.\bar{A}, e_y.\bar{B})$. If p is $\tau(x.A, y.B)$ or $\tau(e_x.A, e_y.B)$, then τ is in $[x.A]_\otimes$ or $[e_x.A]_\otimes$. If p is $T(s, s', \tau)$, then τ is in $[h(x).A]_\otimes$ when s and s' are derived from $h(x).A$ and $h(y).B$, and τ is in $[h(e_x).A]_\otimes$ when s and s' are from $h(e_x).A$ and $h(e_y).B$.

(b) The consequence p_0 extends $\mathcal{F}$ to $\mathcal{F}'$, by adding c, $h(y).B$, $h(e_y).B$ or τ to $[h(x).A]_\otimes$ in $\mathcal{F}'$ if p_0 is $x.A \otimes c$, $x.A \otimes y.B$, $e_x.A \otimes e_y.B$, $\tau(x.A, y.B)$ or $\tau(e_x.A, e_y.B)$, respectively, where τ is a specific contradiction type (*e.g.,* causal) in $\tau(x.A, y.B)$ or $\tau(e_x.A, e_y.B)$. That is, the chase expands ground truth with validated facts in the process.

A *chasing sequence* ξ of G by (Σ, Γ_0) is a sequence

$$\mathcal{F}_0, \ldots, \mathcal{F}_n,$$

where $\mathcal{F}_0$ is Γ_0 and for each $i \in [1, n]$, there exist φ in Σ and valuation h of φ such that $\mathcal{F}_{i-1} \Rightarrow_{(\varphi,h)} \mathcal{F}_i$ is a valid chase step, *i.e.,* $\mathcal{F}_i$ is valid.

The sequence ξ *terminates* when no more rules in Σ can be applied to further extend $\mathcal{F}_n$ with new facts. Such a sequence is referred to as a *terminal chasing sequence*. When a sequence terminates, we refer to the set $\mathcal{F}_n$ of facts as *the result of* ξ.

F.2 The Church-Rosser Property of Chase

Termination and consistency guarantees. Our chase procedure conducts deep contradiction detection, as each chase step may trigger the next step with new facts. Moreover, under the assumption that all rules in Σ and initial facts in Γ_0 are correct, and that ML and textual predicates are accurate, no incorrect fact can enter the final result $\mathcal{F}_k$, which can be proved by induction on chase steps.

We further show that the chase process over graph G is Church-Rosser, *i.e.,* the final result of any terminal chasing sequence ξ is unique regardless of the order in which rules are applied.

Proposition 5: *The chase process with rules Σ is Church-Rosser.* □

Proof: The proof proceeds in two steps.

(1) *All chasing sequences are finite.* Each chase step adds a new fact to a finite set of attributes of vertices and edges. Since the number of nodes and edges in graph G is finite, and each attribute can only be validated once to a consistent set of values or types, the number of possible valid chase steps is bounded.

Specifically, for any terminal chasing sequence $\xi = (\mathcal{F}_0, \ldots, \mathcal{F}_n)$ of G under (Σ, Γ_0), we observe that: (a) Each fact assigns a constant c (from Γ_0 or derived from models) to an attribute $x.A$ or $e_x.A$, which can happen at most $O(|\Gamma_0||G|)$ times. (b) Each fact may equate two attributes (*e.g.,* $x.A = y.B$, or $e_x.A = e_y.B$), which is bounded by $O(|G|^2)$, as the number of such unordered pairs is finite. (c) Each contradiction type label (*e.g.,* $\tau(x.A, y.B)$ or $\tau(e_x.A, e_y.B)$) is added at most once per pair of attributes and contradiction type τ, again bounded by $O(|G|^2)$. Thus, the total number of distinct and valid facts is bounded by a polynomial in $|G|$ and ξ terminates.

(2) *All terminal chasing sequences converge to the same result.* Assume by contradiction that two terminal sequences $\xi_1 = (\mathcal{F}_0, \ldots, \mathcal{F}_{n_1})$ and $\xi_2 = (\mathcal{F}'_0, \ldots, \mathcal{F}'_{n_2})$ over the same graph G and rules Σ produce different fact sets $\mathcal{F}_{n_1} \neq \mathcal{F}'_{n_2}$. Then there exists a chase step $\mathcal{F}'_i \Rightarrow_{(\varphi,h)} \mathcal{F}'_{i+1}$ in ξ_2 such that some new fact (say, $h(x).A \oplus c$ or $h(x).A \oplus h(y).B$ or $\tau(h(x).A, h(y).B)$) appears in $\mathcal{F}'_{i+1}$ but does not appear in $\mathcal{F}_{n_1}$. However, by the validity of rules and consistency of ML and textual classifiers, the rule φ is still applicable to $\mathcal{F}_{n_1}$ via the same valuation h. Therefore, we can extend $\mathcal{F}_{n_1}$ with the same fact, which contradicts the assumption that $\mathcal{F}_{n_1}$ is terminal. Moreover, once the ML and textual models $\mathcal{M}$ and T are trained and validated, they produce consistent outputs across chase sequences. Hence, any rule φ applied in ξ_2 would also apply in ξ_1 under the same data

and valuation, leading to convergence.

Thus, all terminal chasing sequences yield the same fact set. □

F.3 Parallel Detection Algorithm

We next develop Detector, a parallel algorithm for checking contradictions, and shows its parallel scalability [79].

Given a document collection $\mathcal{D}$, a unified graph G, and a RICO set Σ, Detector detects all intra- and inter-document contradictions. It follows the bulk synchronous parallel (BSP) model [123] with one master and K worker processors. The global graph G is partitioned across workers for parallel evaluation. Let G_D denote the subgraph induced by document $D \in \mathcal{D}$. The partitioning strategy satisfies:

- Each document-level graph G_D is kept intact within a single partition to enable efficient local rule execution whenever possible. Large graphs are further decomposed by clustering (dual) star-shaped substructures to preserve logical coherence.
- For inter-document conflict detection (*e.g.,* entity co-reference contradictions), document pairs with potential overlap are assigned to the same partition when possible.

Following [56], each worker node receives a graph fragment $G^{(i)}$ and executes rule checks in Σ locally over its vertices and edges. The master node coordinates execution, uploads tasks via SSH, and aggregates the violation reports returned by workers.

Conflict rules and local evaluation. We discover a RICO rule set Σ capturing diverse contradiction types. Each rule is implemented as a local validator over a graph partition. Each worker independently evaluates all applicable rules on its assigned fragment, serializing the results into a violation report that contains the offending node IDs, violated rule IDs, and relevant context.

Example 7: Consider the following scenario over three documents:

- Doc1: John Smith is born in 1980 and is the father of Mary.
- Doc2: John Smith is recorded as a child of Mary born in 1975.
- Doc3: John Smith is listed as 25 years old in 2025.

The following contradictions are automatically flagged:

- *Temporal*: A cross-check between Doc1 and Doc2 shows that the birth year (1980) postdates his childhood status (1975).
- *Numeric*: The Master node joins Doc1 and Doc3 to identify that the derived age (45) contradicts the recorded 25.
- *Content*: The fusion of Doc1 and Doc2 induces a role conflict, as the same person cannot be both *parent* and *child* of Mary.

Following our partitioning strategy, Doc1 and Doc2 share strong entity overlap (John Smith and Mary) and are thus co-located within the same partition. This allows the local worker to independently detect the temporal and content contradictions without any inter-node data shipping. Conversely, assume Doc3 resides in a different partition. Instead of transmitting raw graphs, the respective workers independently evaluate their local graphs and serialize only the lightweight relevant context (*e.g.,* node IDs and age attributes) into violation reports. The master node then synthesizes these concise reports to detect the inter-partition numeric contradiction. □

Complexity and scalability. Let $|\mathcal{D}|$ be the number of documents and $|\Sigma|$ the number of RICO rules. We analyze the complexity of Detector as follows. (1) *Local evaluation.* For a large document graph G_D, instead of a full scan, Detector employs f_{index} to extract only the subgraphs relevant to the candidate patterns in Σ. Thus, the local cost is $O(\text{avg}(|G_{sub}|) \cdot |\Sigma|)$, where $|G_{sub}| \ll |G_D|$ represents the localized context of a potential contradiction. This ensures that the evaluation remains efficient even as document length grows. (2) *Master coordination.* Input distribution and result aggregation across K workers cost $O(K)$ in the worst case. (3) *Parallel efficiency.* Since checks are rule- and graph-local, the procedure is naturally parallel. With K worker processes, the expected time is

$$T(n, K) = O\left(\frac{|\mathcal{D}|}{K} \cdot |\Sigma| + \log K\right).$$

Thus the runtime scales approximately inversely with K: as the number of processors increases, the cost decreases nearly linearly, demonstrating the parallel scalability in the sense of [79].

F.4 Document Marking

Conflict indexing. To enable conflict indexing from contradictions in G_D back to their textual contexts in D, each node/edge is annotated with a `text_span` field during graph conversion (Section 5.1), recording its location in D. This is done using two strategies: (1) *Exact matching:* direct string matching of entity and relation mentions via regular expressions; and (2) *Fuzzy matching:* when exact matches fail, a windowed search using sequence similarity (*e.g.,* Levenshtein ratio [19]) selects the most likely text span.

We define a function $f_{\text{index}}(u) \to [s_u, e_u]$ that maps each $u \in G_D$ (node or edge) to a text span $[s_u, e_u]$ in the source document, where s_u and e_u represent the start and offset indices in the source document D, respectively. This enables later stages to trace graph components back to their textual origin for contradiction localization and context retrieval.

To map graph-level contradictions back to text: (1) we extract all triples $T = \{(h, r, t)\}$ from the raw text and filter them by the user-specified entity set $\mathcal{E}$, restricting detection to relevant entities; (2) for each retained triple (h, r, t), we apply f_{index} to locate the sentences containing the head and tail entities, providing precise textual anchors; and (3) we localize logical or semantic conflicts identified by RICOs (Appendix F.3) within both the triples and their associated text spans. If a statement in D conflicts with multiple claims, all corresponding spans are jointly marked, supporting multi-point tracing and relational analysis.

Example 8: Consider the entity "Scrooge" in a text, from which the following triples are extracted: (Scrooge, is_generous_to, poor) and (Scrooge, is_stingy_to, poor). RICOs similar to those in Example 2 identify a logical contradiction between these two claims. Then, an example of the document annotation using $f_{\text{index}}(u) \to [s_u, e_u]$ is:

- *Triple 1:* `(Scrooge) -[is generous to]-> (poor)`
  - Head (s_u): "Scrooge has always been generous to the poor during Christmas."
  - Tail (e_u): "The poor often receive gifts from Scrooge."
- *Triple 2:* `(Scrooge) -[is stingy to]-> (poor)`
  - Head (s_u): "Despite rumors, Scrooge is known to be stingy with his wealth."
  - Tail (e_u): "The poor rarely see any charity from Scrooge."

Thus, we successfully map the contradictions detected by the RICOs in the attributed graph G_D back to their corresponding textual contexts (Head and Tail) in the original document D. □

Complexity analysis. The overall complexity of the algorithm is $O\left(\frac{N}{M} \cdot C_{\text{model}} + T \cdot N\right)$ time, where N is the total number of tokens, M is the chunk size, T is the number of extracted triples, and C_{model} is the cost per model inference. The pipeline proceeds as follows. (1) *Triple extraction:* the text is divided into $\frac{N}{M}$ chunks, each triggering one model call, yielding $O\left(\frac{N}{M} \cdot C_{\text{model}}\right)$. (2) *Entity matching and localization:* based on regex and sentence segmentation, costing $O(N)$. (3) *Fuzzy matching:* at worst $O(T \cdot N)$ across T triples, though windowing and thresholding reduce cost in practice. (4) *Contradiction detection:* it scales linearly with the number of chunks, with low overhead. Overall, the method scales well with input size and remains efficient even for long and complex documents.

G LLM PROMPTS

We present LLM prompts used in various stages of DocChecker.

G.1 LLM Prompt for Graph Generation

The prompt for graph generation is as follows:

Graph Generation Prompt for GPT-5.2

(1) Role and task
You are an expert knowledge graph extractor. Your task is to convert the provided document context into a precise JSON property graph (G_D). The graph must only focus on the relationships between the specified target entities based on the given context excerpts.

(2) Input data
- **Target entities ($\mathcal{E}$)**: A user-specified set of entities to be included as vertices in the graph.
- **Relevant contexts**: Specific text excerpts from the original document (D) describing relationships.

(3) Input content (example)
- **Target entities**: `Elon Musk, SpaceX, X (Twitter), Hawthorne, CA`
- **Context excerpts**:
  ```
  (1) Elon Musk, the CEO of SpaceX.
  (2) SpaceX (founded in 2002), acquired X in 2022.
  (3) SpaceX has its headquarters in Hawthorne, CA.
  ```

Note: Context excerpts are based on the original document text.

(4) Output specification
Generate the property graph G_D as a single JSON object with `"vertices"` and `"edges"` arrays.
- **Vertices**: Required fields: `id`, `label`, `name`, `attributes`.
- **Edges**: Required fields: `source`, `target`, `label`, `attributes`.

(5) Desired output format
- **JSON structure**:
```
{ "vertices": [
{"id": "v1", "label": "...", "name": "...", "attributes": {...}}, ...
],
"edges": [
{"source": "vA", "target": "vB", "label": "...", "attributes": {...}}, ... ] }
```

(6) Expected output example
- **Generated JSON**:
```
{ "vertices": [
{"id": "v1", "label": "Person", "name": "Elon Musk", "attributes": {"Role": "CEO"}},
{"id": "v2", "label": "Company", "name": "SpaceX", "attributes": {"Founded": 2002}}
],
"edges": [
{"source": "v1", "target": "v2", "label": "IS_CEO_OF", "attributes": {}} ] }
```

G.2 LLM Prompt for Pruning Search Space and Generating RICO Predicates

The LLM Prompts are as follows:

Contradiction Analyzer and Rule Predicate Generator Prompt

(1) Role and task
You are a highly specialized Contradiction Analyzer and Rule Predicate Generator. Your task is to review a set of Dual Star Patterns ($\mathbb{Q}$) extracted from a knowledge graph, identify those containing a contradiction, classify the primary contradiction type, and propose relevant `RICO` predicates for rule mining.

(2) Input format
You will receive dual star patterns in JSON format. Each dual star (Q_i) consists of two star structures linked by a `same_entity` edge, including their multi-hop neighbors and full attributes.

(3) Required output format
Return a single JSON list containing only the identified contradictory patterns ($\mathbb{Q}_s$). For each Q_i', include:
- `"contradiction_type"`: One of the 7 types defined below.
- `"confidence_score"`: A float from 0.0 to 1.0.
- `"candidate_predicates"`: A list of 3–5 relevant predicates (*e.g.,* `["HasAttribute(X, Y)", "HasValue(X, V)"]`).

(4) Constraint
The analysis must strictly adhere to the provided definitions. If a pattern is not contradictory or does not clearly fit a type, omit it from the output.

(5) Contradiction type definitions
- **1. Factual**: Requires external world knowledge to verify (external KBs needed).
- **2. Content**: Caused by altering one or more attributes of an event or entity.
- **3. Causal**: Arising from a mismatch between cause and effect.
- **4. Negation**: Resulting from negating original sentences (antonyms or "not").
- **5. Relation**: Mutually exclusive relationships between entities (reversed or mismatched).
- **6. Numeric**: Mismatched numbers or values outside an acceptable range.
- **7. Temporal**: Caused by updates over time affecting temporal order.

G.3 LLM Prompt for Baselines in Sensitivity Analysis Tasks

The LLM prompts for sensitivity analysis in Exp-1 are as follows. First, we designed the prompt with a greater emphasis on graph structures (such as SPO format, link building, etc.).

Graph-Augmented Sensitivity Analysis Prompt (GPT-4o Baseline)

(1) Role and task You are a high-precision logical auditor. Your task is

to detect contradictions across different contextual spans (local, intra-document, inter-document) and domains (News, stories, encyclopedia) within the provided text.

(2) **Structural reasoning strategy (SPO + Linking)**
To ensure accuracy, you must follow a three-step structural analysis:
- **Entity normalization**: Identify all entity mentions. Map different surface forms (*e.g.,* "Elon", "CEO of SpaceX") to a single unique `Entity_ID`.
- **SPO extraction**: For every entity, extract fact-triples in `(Subject, Predicate, Object)` format. You must record the paragraph index or source document for each triple.
- **Unit normalization**: Convert all non-standard values (*e.g.,* "a fortnight") into standard units (*e.g.,* "14 days") before comparison.

(3) **Input data**
You will be provided with a set of documents $\mathcal{D}$. Some contradictions may be buried across distant paragraphs (long-range dependencies).

(4) **Required output format (JSON)**
Return a JSON object for each identified contradiction including:
- "`position_analysis`": `{"span": "Local/Intra/Inter", "distancing_sentences": N}`.
- "`conflict_triples`": A pair of triples sharing the same `Entity_ID` but having mismatched `Object` values.
- "`contradiction_type`": One of the 7 types (numeric, temporal, relational, negation, factual, causal, perspective).
- "`final_judgment`": "`contradictory`" or "`consistent`".

(5) **Few-Shot guidance (example)**
Context: (Para 1) "John is 45 years old." ... (Para 20) "John is 65."
Reasoning: Same `Entity_ID` (John); Same `Predicate` (Age); Conflicting `Object` (45 vs 65).
Result: `{"type": "Numeric", "span": "Intra-document", "judgment": "Contradictory"}`.

To supplement structure-centric SPO extraction, we then introduced a chain-of-verification (CoVe) strategy. While SPO-style prompts excel at static attribute matching, CoVe forces the LLM to perform a multi-stage audit: first anchoring initial facts, then tracking state-transitions, and finally evaluating the mutual exclusivity of propositions. This ensures a robust evaluation of the LLM's reasoning upper bound in long-range narratives.

Multi-Stage Logical Consistency Auditing Prompt (GPT-4o Baseline)

(1) **Role and task** You are a senior logical auditor. Your mission is to detect "hidden" contradictions within long-range document contexts by performing a state-transition audit. You must focus on whether an entity's state, intent, or factual grounding undergoes an impossible or un-explained rupture.

(2) **Core strategy: Multi-stage auditing (CoVe)**
Execute the following reasoning steps to minimize false negatives:
- **Stage 1: Fact anchoring**: Identify and list the "Ground Truth" facts established in the first 20% of the document (e.g., character goals, initial locations, primary values).
- **Stage 2: Sequential state tracking**: Monitor these anchored facts throughout the remaining 80% of the text. Note every instance where the entity's state is updated or mentioned.
- **Stage 3: Mutual exclusivity check**: For each state update, verify if it is logically compatible with the "ground truth" or previously stated facts. Apply the law of non-contradiction: A and $\neg A$ cannot both be true at the same time and in the same sense.

(3) **Contradiction heuristics**
Focus on these high-probability conflict zones:
- **Intent-action gap**: Does the final action (Para Z) negate the explicitly stated intent (Para A)?
- **Temporal non-sequitur**: Does event B occur before event A, despite A being its necessary cause?
- **Numeric divergence**: Are there different quantitative descriptions for the same measurement?

(4) **Output specification (JSON)**
- "`logical_anchors`": The initial facts identified.
- "`state_transitions`": Tracking how the fact changed or was reiterated.
- "`detected_rupture`": The exact point where the logic broke.
- "`verifiable_evidence`": A pair of contradictory quotes.

Impact of LLM Prompts. We evaluated GPT-4o using two enhanced prompts to test its reasoning upper bound. (a) Graph-augmented (SPO): By enforcing entity normalization, GPT-4o improves Type MR but still trails DocChecker (77.91%) by 44.67%, as SPO matching fails on noisy surface forms. (b) Multi-stage CoVe: Utilizing state-transition tracking to mitigate long-range decay, this boosts intra-document accuracy by 2.42%; however, its inter-document performance (64.77%) remains 22.19% below DocChecker (86.96%).

G.4 LLM Prompt for Baselines in DSCD Tasks

We then evaluated tailored LLM prompts for the DSCD baselines in Exp-1. First, to bridge the gap between unstructured text and formal logic, we designed the SPO-Link prompt. This strategy mimics the graph-based reasoning of DocChecker by forcing the LLM to perform explicit entity anchoring and triple-based alignment. By transforming long-range dependencies into a structured comparison of subject-predicate-object (SPO) attributes, we evaluate whether LLMs can overcome their inherent architectural limitations in maintaining cross-paragraph consistency.

SPO-Link-based DSCD Prompt (Task Focus: 3a, 3b, 3c)

(1) **Role and task**
You are a graph-based logical auditor. Your task is to perform binary contradiction detection and retrieve the Top-3 evidence sentences by constructing a mental knowledge graph.

(2) **Structural strategy: SPO-linking**
Follow these steps to bridge long-range dependencies:
- **Entity anchoring**: Identify and link all coreferent mentions of the same entity across the entire document. Assign a unique ID to ensure consistency.
- **SPO extraction**: Convert the text into a list of triples: `(Subject, Predicate, Object, Sentence_ID)`.
- **Graph alignment**: Systematically compare objects for the same Subject-Predicate pair. Any mutually exclusive values (*e.g.,*, Location: London vs. Paris) must be flagged.

(3) **Output specification (JSON)**
- "`is_contradictory`": Boolean (true/false).
- "`top_3_evidence_IDs`": The IDs of the sentences forming the detected clash.
- "`clash_triples`": The specific mismatched SPO pairs found during alignment.

To further investigate why LLMs often struggle with evidence localization (Task 3b/c), we then employed the counterfactual au-

diting prompt. Instead of simple pattern matching, this approach requires the model to attempt a logical reconciliation between conflicting sentences. If a hypothesis that makes both sentences true fails, the pair is flagged as definitive evidence. This "self-audit" mechanism specifically targets the Evidence Hit Rate (EHR) by filtering out superficial semantic overlaps.

Counterfactual Auditing DSCD Prompt (Task Focus: EHR / 3c)

(1) **Role and task**
You are a senior evidence auditor. You must decide if a document is contradictory and provide verifiable evidence. You focus on the "reconciliation failure" between sentence pairs.
(2) **Reasoning protocol: Counterfactual self-audit**
To identify high-salience evidence, perform the following:
- **Potential conflict identification**: Screen all sentence pairs that describe the same entity state or event attribute.
- **Reconciliation attempt**: For a suspect pair (s_i, s_j), try to hypothesize a scenario where both can be true.
- **Failure ranking**: If the reconciliation fails logically (violates text facts or physical laws), rank this pair as high-confidence evidence.

(3) **Joint decision constraint**
Only output "`is_contradictory`": `true` if you can pinpoint at least one pair of sentences with a failed reconciliation. This ensures a high Evidence Hit Rate (EHR).
(4) **Output specification (JSON)**
- "`is_contradictory`": Boolean (true/false).
- "`reconciliation_failure_reasoning`": Why the two sentences cannot coexist.
- "`top_3_verifiable_evidence`": The exact text of the most contradictory sentences.

Impact of LLM Prompts. We evaluated Claude-3.7 via two tailored strategies. (a) SPO-Link prompt: It enforces triple alignment to bridge long-range dependencies; while Task 1's F_1 improves by 4.3%, evidence localization (F_1) remains 26.9% lower than DocChecker (0.731 vs. 1.000) due to imprecise entity anchoring. (b) Counterfactual auditing: It employs a self-audit to filter semantic overlaps. While this raises Claude's EHR by 1.4%, its joint performance $J(D) \wedge V(E)$ in Task 3 (0.434) is just nearly half of DocChecker's (0.914).

G.5 LLM Prompt for Baselines in Case Study

In Exp-4, to evaluate whether LLMs can mimic the structured reasoning of DocChecker's rules ($\varphi_{\text{persp}}, \varphi_{\text{temp}}, \varphi_{\text{space}}$), we designed the SPO-graph linking prompt. This strategy compels the model to decompose the narrative into a formal knowledge graph, explicitly linking entities to their specific goal, action, and intent nodes. By forcing the alignment of attributes (e.g., "spring cleaning" $\rightarrow$ "duration") across distant paragraphs, we test the model's ability to detect structural mismatches that are otherwise obscured by fluid prose.

SPO-Graph Linking Prompt (Structural Consistency)

(1) **Role**: You are a Knowledge Graph Architect.
(2) **Graph construction**:
- **Entity anchoring**: Identify all core entities and assign each a unique `Entity_ID` to bridge long-range dependencies.
- **Attribute mapping**: Extract every mentioned `Intent`, `Goal`, `Action`, and `Value` associated with these IDs.
- **Normalization**: Convert all non-standard temporal or numeric descriptors into a unified metric system to ensure comparability.

(3) **Conflict detection**: Compare all attributes linked to the same `Entity_ID`. Flag a contradiction if two nodes of the same category (e.g., Location or Duration) contain mutually exclusive information, regardless of their distance in the text.
(4) **Output**: Return a JSON object listing the `Entity_ID`, the `Mismatched_Attributes`, and the corresponding `Sentence_IDs`.

To address complex contradictions beyond static attributes, we then employed a "semantic tension & intent auditing" prompt. While structural mapping aligns entities, this strategy evaluates narrative integrity through the lens of character-driven causality. It specifically targets "logical ruptures" by auditing the psychological tension between a character's stated intent and their actualized behavior. A primary example is the unresolved conflict in Of Human Bondage, where an initial goal of attending Oxford is abruptly superseded by a forced stay in Germany, which substantially undermines the document's factual consistency.

Semantic Tension Auditing Prompt (Narrative Integrity)

(1) **Role**: You are a narrative consistency auditor.
(2) **Audit protocol**:
- **Stance-action consistency**: Audit whether a character's expressed internal state (opinion, agreement, or intent) is logically compatible with their recorded external behavior.
- **Evolutionary rupture check**: Trace the "initial intent" of key entities. Identify any "subsequent actions" that contradict this foundation, especially if the text attempts to present the change as an original state.
- **Contextual verification**: Scan for discrepancies where the factual premises of an event (time, place, or scale) are altered between different mentions.

(3) **Reasoning requirement**: For each detected conflict, analyze the "semantic tension"—explain why the two states cannot coexist within a coherent and realistic narrative.
(4) **Output**: Return the `Contradiction_Type`, the `Conflicting_Quotes`, and a detailed `Tension_Analysis`.

Impact of LLM Prompts. We re-evaluated these cases using SPO-graph linking and semantic tension auditing (see Exp-4) for more details). (a) For temporal conflicts, although SPO-graph forces the extraction of "fortnight" and "a day," GPT-4o fails to link them to the same "spring cleaning" event across 15+ sentences, whereas RICO φ_{temp} maintains a persistent graph-based anchor. (b) For perspective and motivation, semantic tension improves intent awareness, but lacks the multi-node alignment of RICO φ_{persp}. Consequently, the LLM forcefully reconciles the conflict, wrongly treating a factual error as a natural change in the character's mind.

H LEARNING RICOS IN PARALLEL

Parallel algorithm PLearner is shown as Algorithm 2. We present its implementation details as follows.

Rule generation stage parallelism. To avoid lock contention while preserving global consistency, PLearner adopts an asynchronous producer-consumer design with a lock-free coordination layer (line

Algorithm 2: Parallelized Algorithm PLearner

Input: Same as Algorithm 1
Output: Same as Algorithm 1

1 Reuse lines 1-11 from Algorithm 1 ;
2 **while** $(f_c(\Sigma, C) \leq \alpha)$ ***and*** (FailCount < 3) **do**
3 $\Sigma_{\text{new}} \leftarrow \emptyset$;
4 **Distribute** the set of potentially inconsistent structures $\mathbb{Q}_s$ across K processing units;
5 **foreach** *structure* Q'_i *in the distributed set* ***processed simultaneously*** **do**
6 The processing unit uses the LLM to generate rule proposals Σ_i ;
7 Filter proposals: keep only rules Σ'_i that correctly identify contradictions in the ground truth C;
8 **Collect** Σ'_i into Σ_{new} (ensure safe aggregation);
9 $\Sigma' \leftarrow \Sigma' \cup \Sigma_{\text{new}}$;
10 Parallelize Generalize predicates in Σ';
11 Use generalized Σ' as few-shot prompt for LLM to summarize patterns and guide next rule generation;
12 **if** $(\text{iter} + 1) \bmod k = 0$ **then**
13 Partition corpus $\mathcal{D}$ into M disjoint fragments $\{\mathcal{G}_1, \ldots, \mathcal{G}_M\}$;
14 **Parallel Distribute** fragments to K workers ;
15 **foreach** *fragment* $\mathcal{G}_m$ ***processed in parallel*** **do**
16 Apply current rules Σ' to local subgraph $\mathcal{G}_m$;
17 Compute local statistics (hits, accuracy, coverage) ;
18 **Reduce** local statistics to obtain global $f_a(\mathcal{D}_{\text{clean}})$ and $f_c(\Sigma, C)$;
19 **if** $f_a(\mathcal{D}_{\text{clean}}) > f_a(\mathcal{D})$ **then**
20 $\Sigma \leftarrow \Sigma \cup \Sigma'$;
21 FailCount $\leftarrow 0$;
22 **else**
23 FailCount $\leftarrow$ FailCount $+ 1$;
24 $\Sigma' \leftarrow \emptyset$
25 **return** Σ;

4). Initially, the controller distributes the contradictory patterns across units (line 5). Each worker acts as a producer (line 6), pushing its generated rule set Σ'_i into a thread-safe high-concurrency queue (line 7). A dedicated collector thread consumes these batches and performs atomic, hash-based deduplicated unions to construct Σ_{new} (line 8). PLearner aggregates all discovered Σ_{new} into a unified Σ' upon completion of all worker processes (line 9). This decouples high-latency LLM generation from low-latency aggregation, ensuring consistent global state under high throughput. To balance workload across heterogeneous patterns, a dynamic task-stealing mechanism ensures all K workers complete iterations synchronously.

To ensure autonomous scalability in these steps, PLearner uses a *Token-Bucket* controller to manage traffic. (1) *Smart re-scheduling.* Instead of a simple exponential backoff, which can waste significant time by forcing workers into long, idle waits, our workers perform a *task swap*. When a rate-limit error occurs, the current task is temporarily sidelined, and the worker immediately switches to other local graph processing tasks that do not need the API. (2) *Dynamic flow control.* It monitors how often workers are being blocked and automatically adjusts the speed of requests. This keeps the pipeline moving at the maximum safe speed, avoiding the "stop-and-start" cycles that typically kill performance in large-scale runs.

Verification parallelism. PLearner parallelizes this stage while keeping meta-learning feedback as a synchronized global update. First, predicates in Σ' are generalized in parallel (line 10), and the generalized Σ' is used as few-shot prompts for LLM pattern summarization (line 11). Then, PLearner distributes the verification stage of Learner (lines 12–25), the most costly component, across K workers.

PLearner adopts graph-centric parallel model following GRAPE [46]: (1) *Partitioning phase.* The corpus $\mathcal{D}$ and attributed graph are partitioned into M disjoint fragments $\{\mathcal{G}_1, \ldots, \mathcal{G}_M\}$ (line 13) and dispatched to K workers while preserving local structure (line 14). (2) *Local match phase.* Each worker independently applies Σ' to its fragment (lines 15–17), leveraging dual-star symmetry to detect contradiction instances locally, without cross-worker communication. (3) *Global consolidation.* Partial statistics are aggregated via a single-step global reduction (line 18) to compute f_a and f_c. If accuracy improves, rules are committed to Σ (line 20).

By decoupling fragment-level contradiction matching from global metric computation, this design avoids the high synchronization overhead typically associated with iterative graph processing, yielding near-linear scalability with respect to the number of processors used and correspondingly, corpus size (see below).

The proof of Proposition 4 is as follows.

Proof: Let T_{seq} and T_{par} denote the execution time of the sequential Learner and the parallel PLearner with K workers, respectively. We show that PLearner achieves near-linear speedup as the number of candidate patterns $|\mathbb{Q}_s|$ increases, *i.e.,* its parallel scalability.

Sequential cost. The cost of Learner is dominated by rule generation:

$$T_{\text{seq}} \approx \sum_{i=1}^{|\mathbb{Q}_s|} t_{\text{llm}}(i) + T_{\text{other}},$$

where $t_{\text{llm}}(i)$ is the latency of the LLM call for pattern Q'_i, and T_{other} is the remaining overhead (*e.g.,* pruning and aggregation).

Parallel decomposition. PLearner evenly distributes the $|\mathbb{Q}_s|$ patterns across K workers. The parallel rule generation cost is

$$T_{\text{gen_par}} = \max_{j \in [1,K]} \sum_{i \in \text{Worker}_j} t_{\text{llm}}(i).$$

Assuming roughly uniform LLM latency and $|\mathbb{Q}_s| \gg K$, the workload is balanced, we have the following:

$$T_{\text{gen_par}} \approx \frac{1}{K} \sum_{i=1}^{|\mathbb{Q}_s|} t_{\text{llm}}(i).$$

Overhead. Predicate generalization is naturally parallel and incurs no inter-worker communication. Aggregating Σ'_i into Σ_{new} adds only $O(\log K)$ synchronization, negligible compared with t_{llm}.

Scalability. As $|\mathbb{Q}_s|$ grows, $T_{\text{other}}/T_{\text{seq}} \to 0$, and thus

$$S = \frac{T_{\text{seq}}}{T_{\text{par}}} \approx \frac{T_{\text{seq}}}{T_{\text{seq}}/K} = K.$$

Hence, PLearner is parallelly scalable relative to Learner, assuming LLM rate limits are not the bottleneck. □

I CONSTRUCTING THE GOLD STANDARD C

To evaluate rule correctness against human annotations, we define a transformation $\Phi : \mathcal{D} \to C$ that maps textual ground truth into

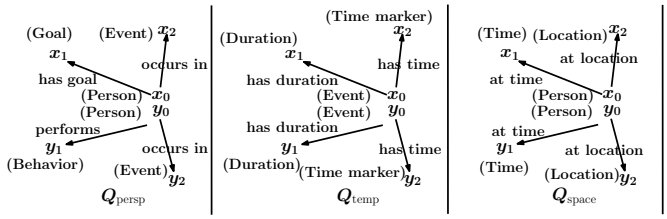


Figure 5: Dual patterns of RICOs in case study

structured verification instances. For each document $d \in \mathcal{D}$, the dataset provides $\langle T_d, E_d, L_{\text{type}}, P_{\text{plug}} \rangle$, where T_d is the full text, E_d is the evidence span, L_{type} is the contradiction category, and P_{plug} records the perturbation (*e.g.,* `Replace`). The construction of a gold instance $c \in C$ proceeds in three steps.

(1) Entity–attribute extraction. Following Section 5.1, we extract entities $\mathcal{E}$ and attributes $\mathcal{A}$ appearing in E_d. Each fact $f = (s, p, o)$ anchors s and o to nodes in the enriched graph G^+.

(2) Predicate mapping. During the transformation Φ, each annotated label L_{type} and perturbation P_{plug} is mapped to a predicate $P \in \mathsf{P}$, where $\mathsf{P} = \mathsf{P}_{\text{bind}} \cup \mathsf{P}_{\text{text}} \cup \mathsf{P}_{\text{ML}} \cup \mathsf{P}_{\text{comp}} \cup \mathsf{P}_{\text{type}}$. Here (1) $\mathsf{P}_{\text{bind}} = \{S(x, s), S(e_x, s)\}$ binds substrings to vertices or edges; (2) $\mathsf{P}_{\text{text}} = \{T(s, s', \tau)\}$ catches contradiction types τ via pretrained NLP/NLI/LLM models; (3) $\mathsf{P}_{\text{ML}} = \{\mathcal{M}(x.\bar{A}, y.\bar{B}), \mathcal{M}(e_x.\bar{A}, e_y.\bar{B})\}$ encodes learned boolean relations on attributes; (4) $\mathsf{P}_{\text{comp}} = \{=, \neq, <, \leq, >, \geq\}$ applies to scalar comparisons (*e.g.,* $x.A \oplus y.B$ or $z.A \oplus c$); and (5) $\mathsf{P}_{\text{type}} = \{\tau(x.A, y.B), \tau(e_x.A, e_y.B)\}$ enforces contradiction types τ. Thus, Φ maps $(L_{\text{type}}, P_{\text{plug}})$ to predicates in P (*e.g.,* numeric contradictions map to $\mathsf{P}_{\text{comp}} \cup \mathsf{P}_{\text{type}}$).

(3) Contradiction modeling. The set P_{plug} determines inconsistency boundaries. For example, if $P_{\text{plug}} =$ `Replace`, any value $v' \neq v_{\text{orig}}$ on the same attribute path constitutes a contradiction.

Example 9: Consider ContraDoc instance `3488771835_6`, where E_d = "One rat, Jenner, agreed wholeheartedly with The Plan," annotated as a Content contradiction (*Perspective/Opinion* and *Emotion/Mood/Feeling*) with perturbation $P_{\text{plug}} =$ `Replace`. The transformation Φ constructs the gold instance $c \in C$ as follows:

(1) Extraction: DocChecker extracts the subject *Jenner* and object *The Plan*, anchoring them to graph nodes, and identifies the original sentiment attribute as $v_{\text{orig}} =$ Positive.

(2) Mapping: The Content contradiction label (L_{type}) is deterministically mapped to a type constraint predicate $P \in \mathsf{P}_{\text{type}}$, specifically formulating $\tau_{\text{content}}(x.\text{sentiment}, y.\text{sentiment})$ to represent the expected semantic conflict.

(3) Modeling: Because $P_{\text{plug}} =$ `Replace` enforces semantic inversion, the inconsistency boundary is formalized such that any detected sentiment $v' \neq v_{\text{orig}}$ (*e.g.,* a negative or conflicting opinion) triggers the contradiction.

Consequently, C encodes this structured constraint ($c =$ {Subj: Jenner, Obj: The Plan, Expected: $v' \neq$ Positive}), providing a fixed reference for computing rule accuracy metrics f_a and f_c (Algorithm 2, lines 12–25). □

J ADDITIONAL EXPERIMENTAL DETAILS

J.1 Results of Additional Experiments

We also conduct the following experiments.

Sentence-level NLI. This task tests premise–hypothesis classification, a benchmark for fine-grained semantic understanding. As shown in Table 8, DocChecker performs best, reaching 87.0% Acc ($F_1 = 0.931$) on SNLI and 77.0% Acc ($F_1 = 0.870$) on MNLI. While RoBERTa-base and XLNet-base-cased are competitive on MNLI (71.6% and 69.4% Acc), they still lag behind DocChecker. The 17.2% Acc margin over RoBERTa-base on SNLI further confirms that DocChecker effectively captures core semantic relations.

Case study. Figure 5 depicts the dual patterns of the three RICO rules: φ_{persp}, φ_{temp} and φ_{space} in the case study of Exp-4 (Section 7).

J.2 Details of Experimental Datasets

We provide details of the experimental datasets below.

Domain of datasets. We categorize the 15 datasets into three groups based on structural complexity and domain characteristics:

(1) Document-level & real-world contradiction. Dataset ContraDoc targets cross-document conflicts; ECON evaluates consistency in event-centric narratives; Knowledge Conflict captures discrepancies between model knowledge and external evidence; Real-life Contradiction Corpus contains naturally occurring contradictions from news and social media; and ConTRoL benchmarks reasoning-intensive contradiction detection.

(2) Specialized & domain-specific contradiction. ContractNLI and CaseHOLD focus on legal consistency and reasoning; StorySumm evaluates inconsistencies between stories and their summaries; and DOCNLI addresses large-scale document-level inference.

(3) Standard NLI & linguistic probing. SNLI/MNLI benchmark general domain inference; RTE-3 is a classic entailment challenge set; MCTest-NLI derives contradiction cases from machine comprehension; binary-FEVER-NLI targets fact-verification conflicts; and Negation probing sets isolate logical negation phenomena.

Statistical summary and contradiction distribution. We summarize dataset scale, ground truth labeling, and contradiction-type distribution across all 15 datasets. For QA datasets, ground truth derives from correct answers, with alternative candidates treated as contextual contradictions. For entailment/non-entailment datasets, we use LLM annotation with expert verification to identify contradiction types within non-entailment instances; entailment and neutral cases are treated as non-contradictory negatives.

For NLI/QA datasets consisting of sentence pairs or samples, we randomly group every m pairs/samples into a single document in order to reduce I/O overhead while increasing the density and complexity of contradiction types within each document. The value of m for each NLI/QA dataset is reported below. This allows us to rigorously evaluate detection capabilities within complex documents containing diverse, long-context contradiction.

The binary-FEVER-NLI dataset [138] is a large-scale fact-

Model	SNLI				MNLI			
	Acc	P	R	F1	Acc	P	R	F1
DocChecker	0.870	0.879	0.989	0.931	0.770	0.899	0.844	0.870
BERT-base-uncased	0.671	0.675	0.671	0.671	0.600	0.603	0.600	0.599
RoBERTa-base	0.698	0.706	0.698	0.698	0.716	0.716	0.716	0.715
XLNet-base-cased	0.709	0.710	0.709	0.709	0.694	0.693	0.694	0.693

Table 8: Performance on sentence-Level NLI tasks

checking benchmark adapted from FEVER, containing 221,361 samples (203,152 train/8,209 dev/10,000 test). Labels are binary (*entailment* vs. *not-entailment*), with *not-entailment* (41.28%) merging original contradiction and neutral categories to evaluate non-supportive evidence detection. As a Wikipedia-based verification task, its contradiction profile is dominated by *factual* and *relational* conflicts, with moderate *content*, *temporal*, and *numeric* mismatches, and relatively fewer *causal* and *negation* cases. We batch these samples into 222 documents of 1,000 samples each.

The CaseHOLD dataset [145] is a large-scale legal benchmark with 53,000 multiple-choice problems derived from U.S. federal and state court opinions. Each question requires selecting the correct legal holding from five options (one ground truth and four expert-crafted distractors). Unlike standard NLI, the "not-entailment" candidates are often semantically plausible yet legally inconsistent, making CaseHOLD a demanding test of domain-specific legal reasoning. Our categorical analysis shows that *factual* and *causal* conflicts dominate, reflecting the role of complex precedent and legal dependency structures, while *content* and *temporal* contradictions also appear frequently. In contrast, *numeric*, *relational*, and *negation* contradictions are relatively rare, as the primary challenge lies in resolving high-level judicial logic rather than surface-level linguistic or numeric inconsistencies. We partition these samples into 530 documents, each containing 100 problems.

ConTRoL [88] is a specialized benchmark for contrastive reasoning over long documents, consisting of 8,325 expert-annotated samples derived from Wikipedia. Formulated primarily as a multiple-choice task, it requires models to cross-reference dispersed thematic segments within the same document to resolve conflicts. The ground truth is determined through logical synthesis across these disjoint evidence chains, making ConTRoL a rigorous test of long-range reasoning and cross-paragraph coherence. Our categorical analysis shows that *casual* and *content* contradictions dominate (approximately 65% combined), reflecting the emphasis on semantic consistency and topic alignment. These are followed by notable *factual* and *temporal* ones, which arise from precise cross-block comparisons. In contrast, simple *negation* and *numeric* inconsistencies are less prevalent, as the dataset prioritizes higher-level causal and logical integration over surface-level linguistic cues. The samples are organized into 417 documents with 20 samples per document.

ContraDoc [84] is a high-quality benchmark for evaluating self-contradiction detection in long-form documents (300–2,200 tokens). It contains 891 documents across three domains (Wikipedia, News, and Story Summaries), with binary ground truth: 449 contradictory (pos) and 442 non-contradictory (neg) samples. To assess contextual difficulty, contradictions are categorized by span: intra-sentence (73 documents), local (within 4 sentences; 220 documents), and global (beyond 4 sentences; 155 documents). Our analysis indicates that *numeric* and *negation* contradictions are most frequent and relatively easier to detect due to their objective nature. These are followed by *factual*, *content*, *casual* and *temporal* inconsistencies, which typically appear in the global scope. In contrast, *relational* contradictions are less common and harder to identify, as they require deeper logical synthesis over extended context.

ContractNLI [75] is a specialized legal benchmark comprising 607 real-world non-disclosure agreements (NDAs) and 10,319 hypothesis-premise pairs. It adopts a three-way ground truth: *contradiction* (0), *entailment* (1), and *neutral* (2) and poses greater difficulty than general NLI due to complex legal syntax and strict logical constraints. Our analysis shows that *content* and *casual* conflicts dominate for such documents, typically arising from prohibited actions or conditional exclusions within clauses. These are followed by *factual* and *temporal* inconsistencies, often involving discrepancies in confidentiality duration or scope. *Relational* and *numeric* contradictions appear at moderate levels (*e.g.,* damages or notice periods), while *negation* conflicts are rare, reflecting the prescriptive nature of contractual language rather than descriptive narrative logic. We group the samples into 104 documents, where each document contains 100 hypothesis-premise pairs.

DOCNLI [138] is a large document-level NLI benchmark of 942,000 samples, constructed by reformulating five heterogeneous NLP tasks (including ANLI and SQuAD) into inference settings. It pairs full-document premises with hypotheses of varying granularity to evaluate long-range reasoning. The ground truth is provided in both binary (*entailment* vs. *not-entailment*) and three-way formulations. We find that *causal*, *content* and *factual* contradictions dominate, reflecting its summarization-oriented construction. *Relational* and *temporal* inconsistencies also appear frequently and typically require cross-paragraph evidence alignment. In contrast, *numeric* and *negation* conflicts are comparatively rare, representing the most challenging logical categories within the corpus. These samples are divided into 471 documents, each comprising 2,000 samples.

ECON [70] is a specialized benchmark for evaluating event-centric consistency in long-form narratives. It contains approximately 12,000 samples assessing whether a generated story remains logically coherent with its established event history. The ground truth is binary, distinguishing between *consistent* (entailment) and *inconsistent* (contradiction) event flows. Our categorical analysis of the contradiction types shows that *factual* and *numeric* conflicts are most dominant, reflecting the dataset's core focus on the logical "effects" of events and their chronological ordering within a narrative. *Casual*, *relational* and *temporal* inconsistencies appear next in frequency, often arising when the logical consequences of a clause are violated or when the duration of legal obligations (such as confidentiality periods) deviates from the agreed-upon timeframe. In contrast, *negation* and *content* contradictions are relatively rare, as the challenge centers on maintaining high-level narrative coherence rather than detecting surface-level discrepancies. We construct 120 documents by batching 100 samples into each document.

Knowledge Conflict [129] is a diagnostic benchmark designed to evaluate how LLMs resolve discrepancies between internal parametric knowledge and external contextual evidence. It

contains over 2,000 samples spanning diverse domains and entities, constructed via two synthetic mechanisms: *substitution* (in-domain entity replacement) and *shuffle* (entity reordering). The ground truth is multi-faceted, requiring models to (i) detect the existence of conflicts, (ii) localize the contradictory segments, and (iii) articulate divergent viewpoints generated from parametric versus contextual knowledge. Our categorical analysis of contradiction types indicates that *factual* and *temporal* conflicts dominate, reflecting the dataset's emphasis on manipulating entity properties and the specific time-sensitive states of entities via substitution. *Content* and *numeric* discrepancies appear with moderate frequency due to the knowledge-intensive nature of the corpus. By contrast, *negation*, *causal*, and *relational* contradictions are relatively rare, since the generation paradigm primarily targets semantic entity rewrites rather than logical or chronological inversions. The dataset is reorganized into 200 documents containing 10 samples each.

The binary-MCTest-NLI dataset [138] is a narrative-driven benchmark adapted from the MCTest machine reading comprehension dataset, containing 8,000 samples that are split into 4,800 for training, 2,400 for testing, and 800 for development. The task follows a binary classification scheme (*entailment* vs. *non-entailment*), with a fixed label distribution of 75% non-entailment (6,000 samples) and 25% entailment (2,000 samples). Our categorical analysis of the seven contradiction types within the Non-entailment class indicates that *content*, *factual* and *casual* discrepancies are most prevalent for the benchmark, reflecting the story-centric nature of the corpus where inconsistencies commonly arise from character actions and entity attributes. *Numeric* and *temporal* conflicts follow in frequency, often stemming from narrative event tracking. By contrast, *negation* and *relational* contradictions are less frequent, as the primary challenge emphasizes lexical coherence and story-level reasoning rather than explicit logical or complex entity-to-entity interdependencies. Each document is constructed from a batch of 20 samples, resulting in 400 documents in total.

MNLI (Multi-Genre Natural Language Inference) [131] is a large-scale, multi-domain benchmark containing 412,313 samples across diverse text genres. The ground truth adopts a three-way classification (*entailment*, *neutral*, *contradiction*), with contradiction instances constituting approximately 32.53% (134,116 samples). Our analysis shows that *casual* and *factual* conflicts are most prevalent, reflecting the frequent occurrence of premise-driven effects and entity-based contradictions across genres. These are followed by *negation* and *content* ones, which highlight MNLI's emphasis on linguistic diversity and direct semantic opposition. In contrast, *numeric*, *temporal* and *relational* contradictions are present but less dominant, as the dataset primarily targets broad natural language understanding rather than specialized mathematical logic, chronological reasoning, or complex entity linking. We create 825 documents by grouping problems in batches of 500.

Negation (RTE-Negation) [24] is a specialized diagnostic corpus constructed by perturbing the RTE2 benchmark with explicit negative markers. It contains 899 samples partitioned into a development set (102 pairs) and a test set (797 pairs), with a binary ground truth (*YES*/*NO*) and a nearly balanced label distribution (50.17% contradictions; 451 samples). We find that *factual* dominates (>80%), reflecting the dataset's design focus on testing explicit negative facts. *Numeric* and *negation* ones appear as relatively simple contradiction cases, while *content* and *relational* conflicts occur moderately through world-knowledge contrasts. Conversely, *causal* and *temporal* types are rare, as the dataset targets direct semantic inversion, not complex event-flow reasoning.

Real-life Contradiction Corpus [24] is a high-quality benchmark of 131 naturally occurring contradiction pairs that were collected from Google News, Wikipedia, Lexis Nexis, and the DARPA GALE Program. Unlike synthetic datasets, all samples are labeled as contradictions (100% positive), providing a focused evaluation of real-world inconsistency phenomena. Our seven-type categorical analysis reveals a diverse distribution: *numeric* (~29.0%) and *factual* (~30.5%) dominate for such documents, followed by *casual* (17.56%). *content* and *relational* conflicts account for ~20% collectively, while *temporal* and *negation* contradictions form the minority, reflecting the nuanced and context-driven nature of naturally occurring inconsistencies in professional and encyclopedic writing.

RTE-3 [54] represents the transition to three-way textual entailment classification, comprising 2,974 sentence pairs in the 3w-4C configuration. The ground truth is labeled as *entailment* (1,520), *contradiction* (340), and *unknown* (1,114), with contradictions accounting for 11.43% of the corpus. The dataset spans four task domains: information extraction (IE), summarization (SUM), information retrieval (IR), and question answering (QA), to evaluate complex logical and commonsense reasoning. We find that *factual* and *content* types dominate (~55%), reflecting entity-role mismatches and semantic overlap common in IE and SUM. *Relational* and *casual* conflicts follow, particularly in IR and QA, where direct evidence verification is required. *Numeric* and *temporal* mismatches are less frequent, while *negation* contradictions are rare, reflecting a focus on structural alignment over explicit logical inversion. We organize the dataset into 744 batched documents, each covering 4 sentence pairs.

SNLI (Stanford Natural Language Inference) [15] is a foundational large-scale NLI benchmark comprising 570,152 sentence pairs derived from image captions. It adopts a three-way classification scheme: *entailment* (0), *neutral* (1), and *contradiction* (2), with contradictions accounting for 33.27% (189,702 samples), maintaining a near-balanced distribution across training, validation, and test sets. Our seven-type categorical analysis shows that *content* and *relational* discrepancies are most dominant, often involving conflicting actions or entity attributes (*e.g.,* "a man sitting" vs. "a man standing"). *Negation* and *factual* inconsistencies are also frequent, reflecting the crowdsourced hypothesis generation process. *Numeric* and *temporal* conflicts appear with moderate frequency, typically regarding counts of entities or event timing, while *causal* contradictions are rare due to the descriptive, caption-based nature of the corpus, which seldom encodes explicit cause-effect relations. We generate 571 documents, each aggregating 1,000 sentence pairs.

StorySumm [118] is a narrative-centric benchmark for long-form fiction consistency, comprising 32 short stories and 96 primary entries, with 580 associated summary paragraphs. The dataset focuses on narrative reliability, with a high ground-truth error rate of 85.17% (494 paragraphs) labeled as contradictory. Samples are categorized by difficulty, and 41.67% are marked as "hard" requir-

ing deep comprehension of character development and narrative arcs. Our seven-type categorical analysis indicates that *temporal*, *causal* and *factual* conflicts are significantly more frequent than in other datasets, reflecting the reliance of storytelling on chronological progression and character motivation. *Content* and *negation* discrepancies are also prominent, often involving misattributed roles or plot details. By contrast, *numeric* and *relational* errors are relatively rare, as story narratives prioritize plot progression and character dialogue over precise quantification or entity linking.